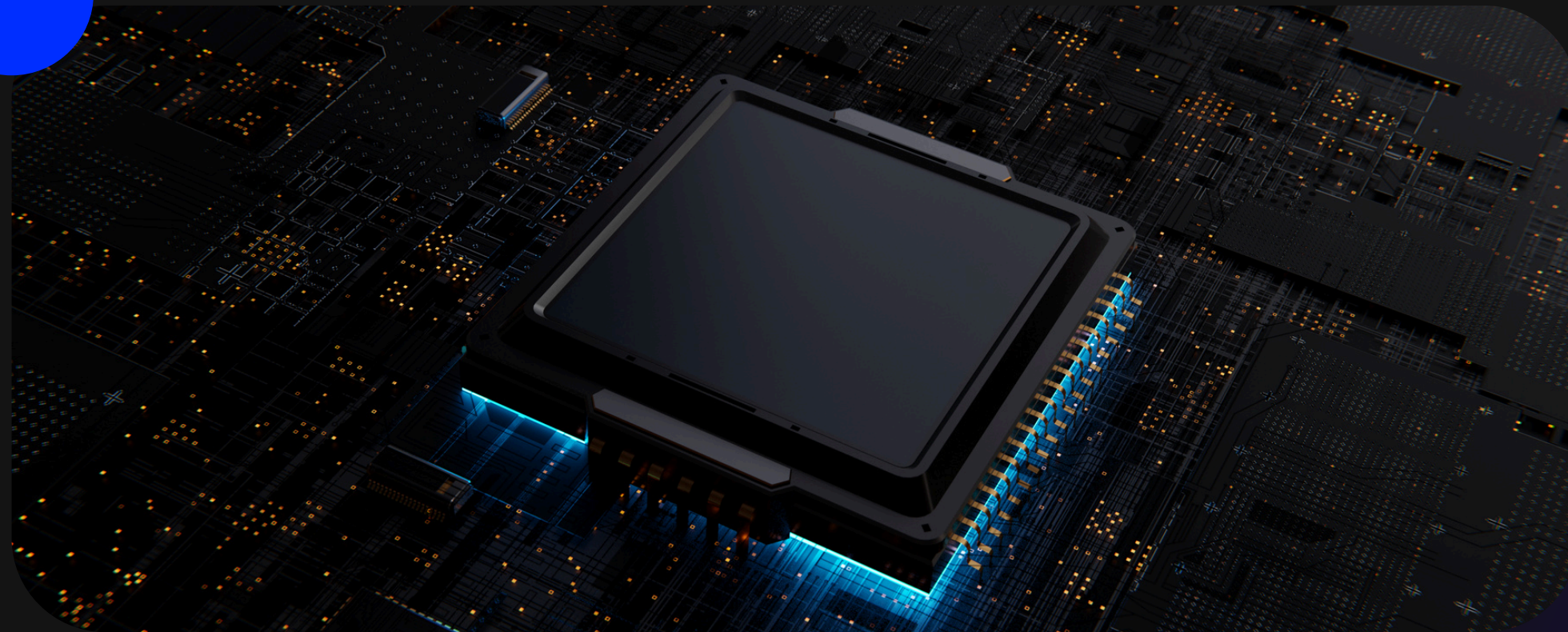


Modelo de Ising



Juan Manuel Bonilla Arce

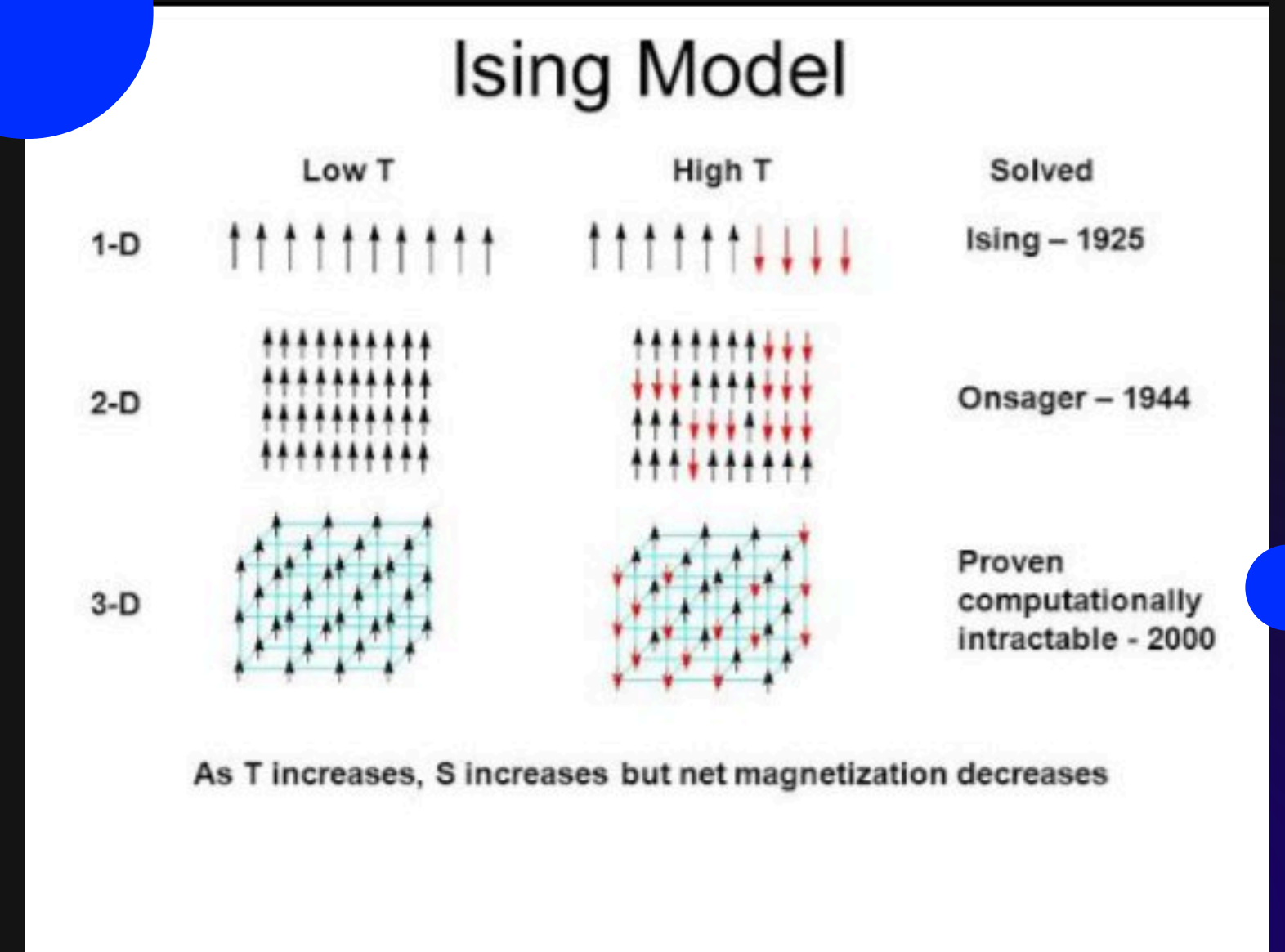
Sebastián Alí Sacasa Céspedes

Sebastián Longhi Pacheco

Diego Vermeer Cubero

Introducción

- Modelo fundamental en física de muchos cuerpos
- Describe magnetismo cuántico y transiciones de fase
- Relevante en simulación cuántica y computación cuántica



Marco teórico

- Hamiltoniano:
 - J: interacción entre espines
 - g: campo transversal
 - Competencia entre orden y fluctuaciones cuánticas
- Dinámica cuántica:
 - Necesita calcular la exponencial del Hamiltoniano
 - Costoso para matrices $2^N \times 2^N$
- Dimensión del espacio Hilbert:
 - Crece exponencialmente
 - Impracticable para N moderado
 - Evolución exacta: imposible de computar

$$\hat{H} = -J \sum_i \sigma_i^z \sigma_{i+1}^z - g \sum_i \sigma_i^x$$

$$\begin{aligned} \frac{d}{dt} |\psi(t)\rangle &= -iH |\psi(t)\rangle \\ |\psi(t)\rangle &= e^{-iHt} |\psi(0)\rangle \end{aligned}$$

$$\dim(\mathcal{H}) = 2^N$$

Marco teórico

$$|z\rangle = \frac{1}{\sqrt{1+|z|^2}}(1, z)^T$$

$$|Z\rangle = \bigotimes_i |z_i\rangle$$

$$G = D^\dagger D$$

$$P = \sum_{m=1}^M |u_m\rangle \langle u_m|$$

$$H_{\text{eff}} = PHP$$

$$|\tilde{\psi}(t)\rangle = P e^{-iH_{\text{eff}}t} P |\psi_0\rangle$$

- Representación continua de espínes (estados coherentes):
 - Permite muestrear configuraciones sin recorrer 2^N estados.
 - Parametrización continua de espínes.
- Subespacio reducido:
 - Se forma D con el conjunto de M estados coherentes
 - Matriz de correlación G de dimensión M .
- Proyección sobre el subespacio:
 - Se construye con los autovectores de G
 - Se reduce de 2^N a M dimensiones.
 - Tenemos que $M \ll 2^N$.
- Hamiltoniano efectivo:
 - Obtenemos la evolución numérica en un espacio comprimido.
 - Esto es computacionalmente mucho más eficiente.

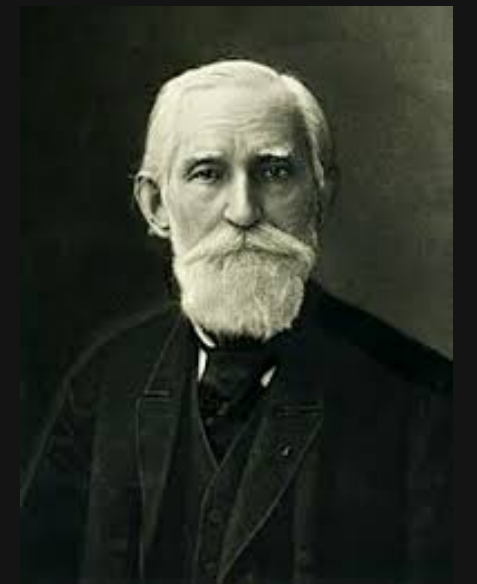
Implementación

Clases principales:

- **SystemConfig:** configuración del sistema → parámetros iniciales, hamiltoniano.

$$\hat{H} = -J \sum_{i=1}^N \hat{\sigma}_i^z \hat{\sigma}_{i+1}^z - g \sum_{i=1}^N \hat{\sigma}_i^x,$$

- **ChebyshevPropagator:** implementa el método de propagación de Chevyshev → Se toman los polinomios de Chevyshev (ortogonales) y se expande la función de onda en esta base.

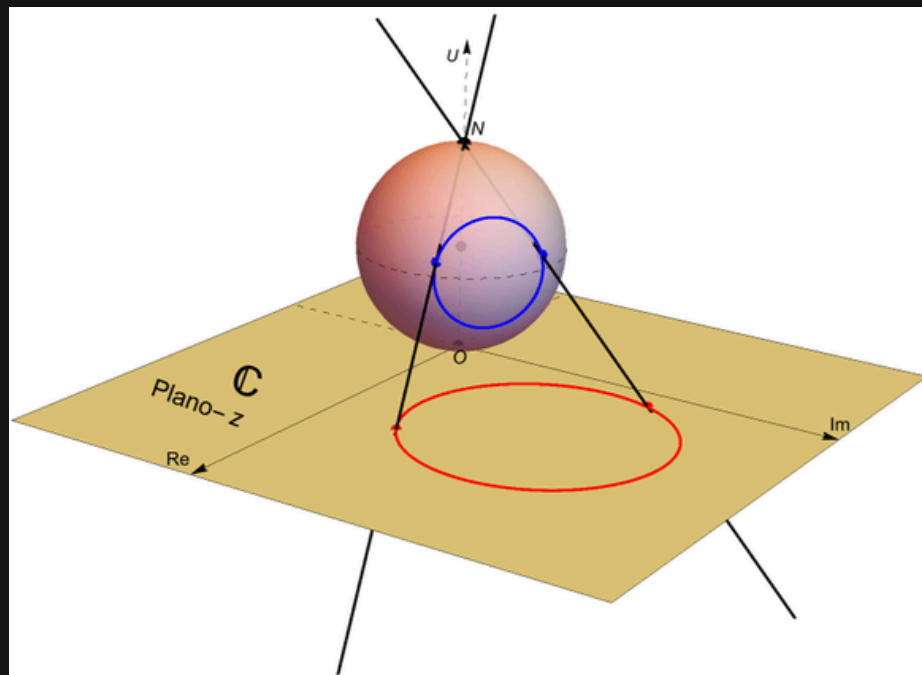


$$a_0 = \frac{1}{\pi} \int_{-1}^1 \frac{f(x)}{\sqrt{1-x^2}} dx \quad y \quad a_k = \frac{2}{\pi} \int_{-1}^1 \frac{f(x) T_k(x)}{\sqrt{1-x^2}} dx \quad \text{donde } k \geq 1$$

Implementación

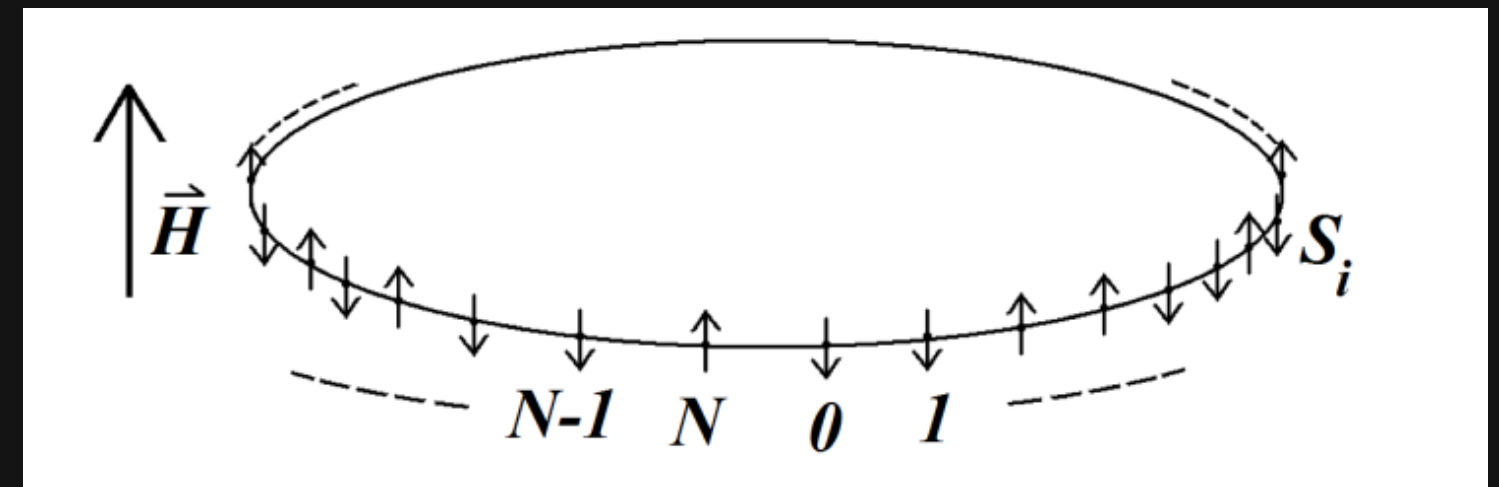
Clases principales:

- **project_and_evolve_chebyshev**: proyecta H y ψ_0 evoluciona con Chebyshev en subespacio reducido. Compara con evolución completa por Chebyshev.



- **RicciFlowOptimizer**: optimiza la implementación con flujo de Ricci.

- **EnergyAnalyzer**: analizador de energía del modelo de Ising, principalmente se evalúa la energía en cada estado, de cada interacción y la densidad de energía local



- **EnergyVisualizer**: visualización de análisis energético

Paralelización

Construcción de estados coherentes

```
#pragma omp parallel for schedule(dynamic)
for (int j = 0; j < S; ++j) {
    VectorXcd single = coherent_state(z_list[j]);
    VectorXcd psi = single;

    for (int i = 1; i < N; ++i) {
        int rows_psi = psi.rows();
        int rows_single = single.rows();
        VectorXcd kron_vec(rows_psi * rows_single);

        for (int r1 = 0; r1 < rows_psi; ++r1) {
            kron_vec.segment(r1 * rows_single, rows_single) = psi(r1) * single;
        }
        psi = kron_vec;
    }
    Data.col(j) = psi;
}
```

Paralelización

Optimización con Flujo de Ricci

```
MatrixXcd H_dense = MatrixXcd(H);  
SelfAdjointEigenSolver<MatrixXcd> eigensolver(H_dense);
```

```
#pragma omp parallel for schedule(dynamic)  
for (int i = 0; i < n_beta; ++i) {  
    double beta = beta_list[i];  
  
    std::vector<double> weights = get_thermal_weights(H, Data, beta);  
    std::vector<double> eigenvalues;  
    MatrixXcd P = build_weighted_projection(Data, config.M_trunc, weights, eigenvalues);  
    double fid = calculate_fidelity(P, psi0);  
  
    results.weights_list[i] = weights;  
    results.fidelities[i] = fid;  
    P_list[i] = P;  
    eigenvalues_list[i] = eigenvalues;  
}
```

Paralelización

Evolución temporal con Chebyshev

```
VectorXcd ChebyshevPropagator::evolve(const VectorXcd& psi0, double t) const {  
    (...)  
    // Recurrencia de Chebyshev  
    VectorXcd T0 = psi0;  
    VectorXcd T1 = H_rescaled * psi0; // matriz * vector  
    VectorXcd psi_t = coeff[0] * T0 + coeff[1] * T1;  
  
    for (int n = 2; n <= M; ++n) {  
        VectorXcd Tn = 2.0 * H_rescaled * T1 - T0; // matriz * vector  
        psi_t += coeff[n] * Tn;  
        T0 = T1;  
        T1 = Tn;  
    }  
  
    return psi_t;  
}
```

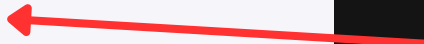
```
#pragma omp parallel for schedule(dynamic)  
for (int i = 0; i < nt; ++i) {  
    double t = t_list[i];  
  
    VectorXcd psi_proj_t = prop_eff.evolve(psi0_proj, t);  
    VectorXcd psi_proj_lifted = P * psi_proj_t;  
    states_proj.col(i) = psi_proj_lifted;  
  
    VectorXcd psi_full_t = prop_full.evolve(psi0, t);  
    states_full.col(i) = psi_full_t;  
}
```

Paralelización

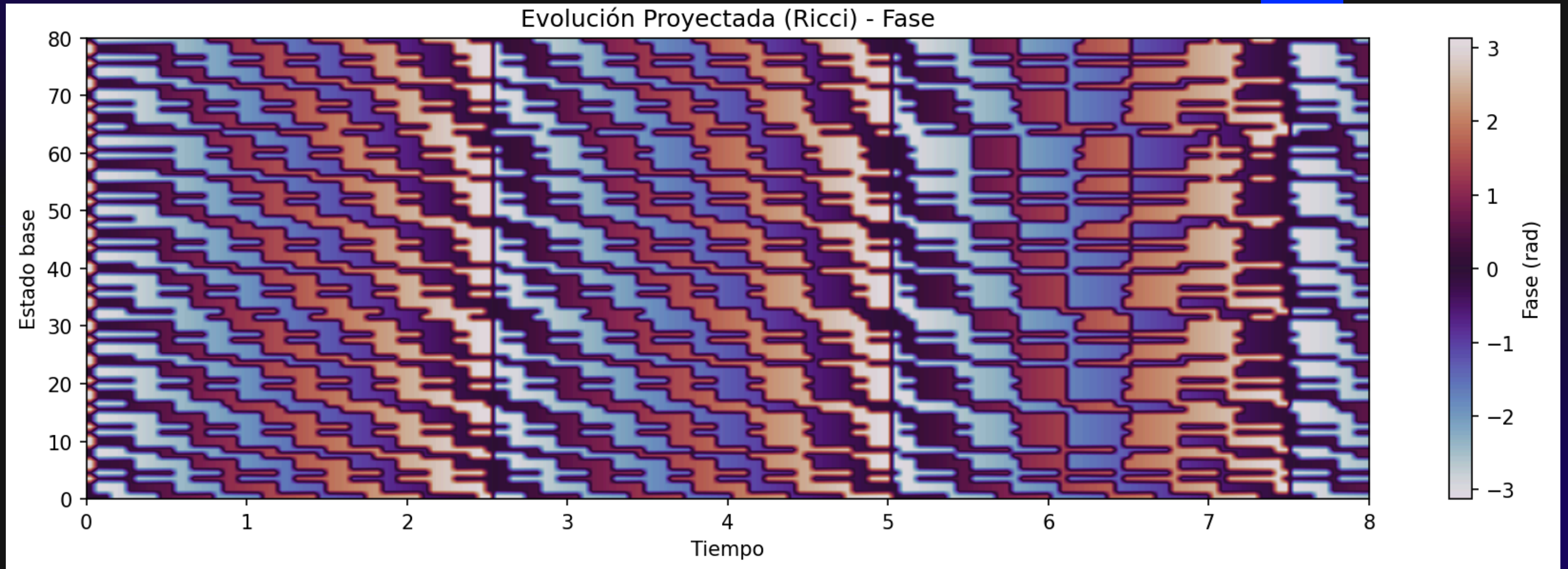
Observables (energía, magnetización, etc)

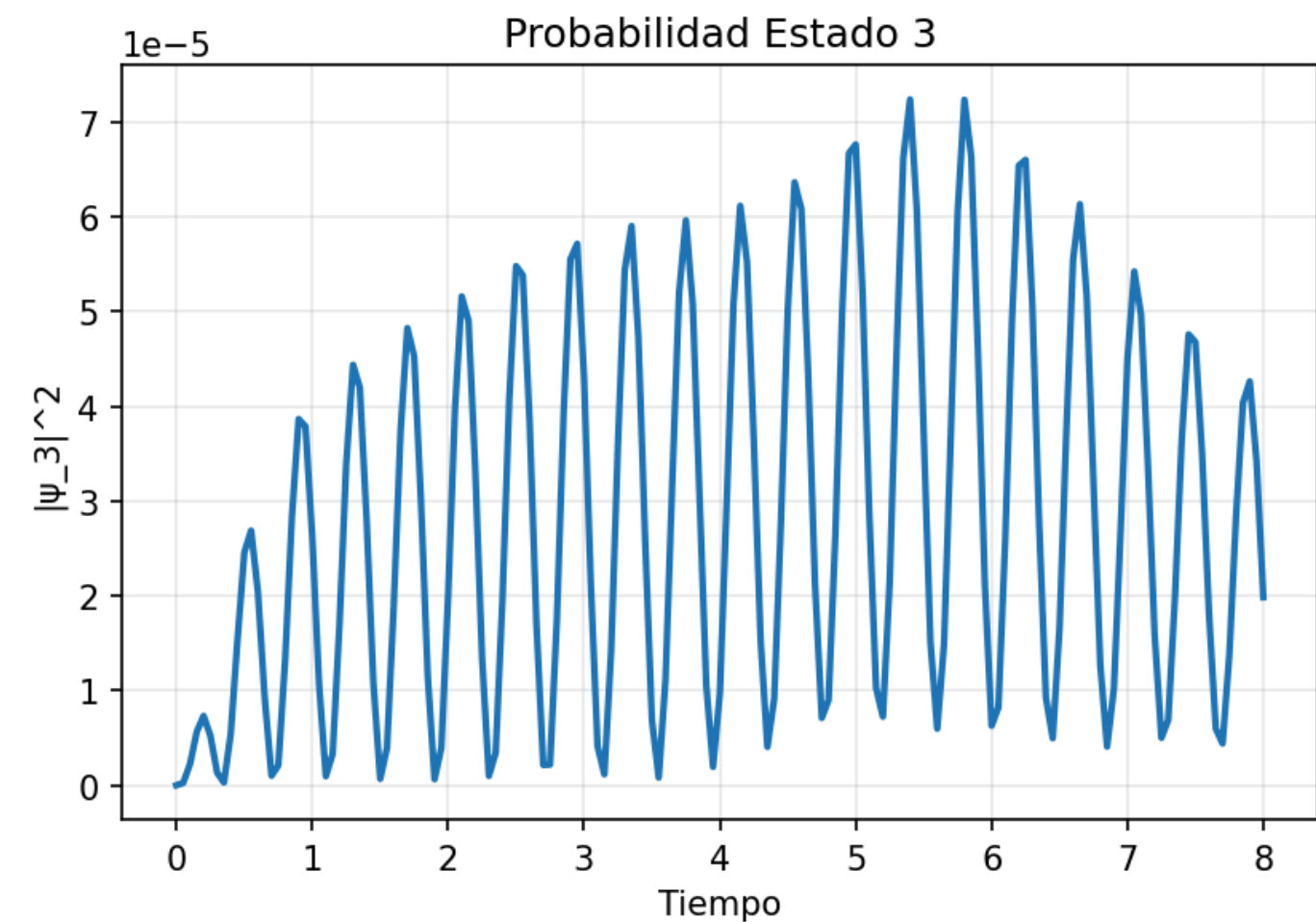
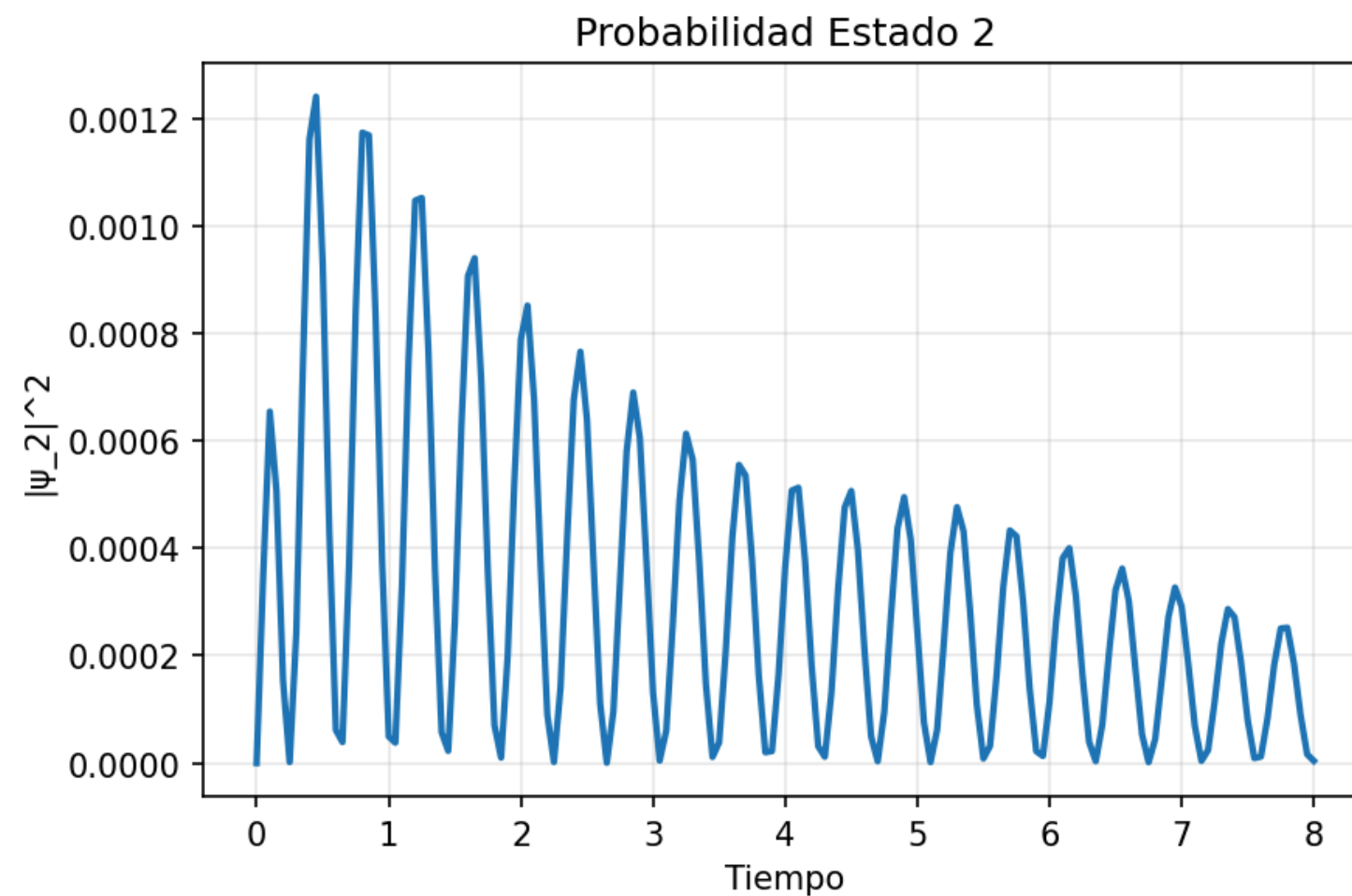
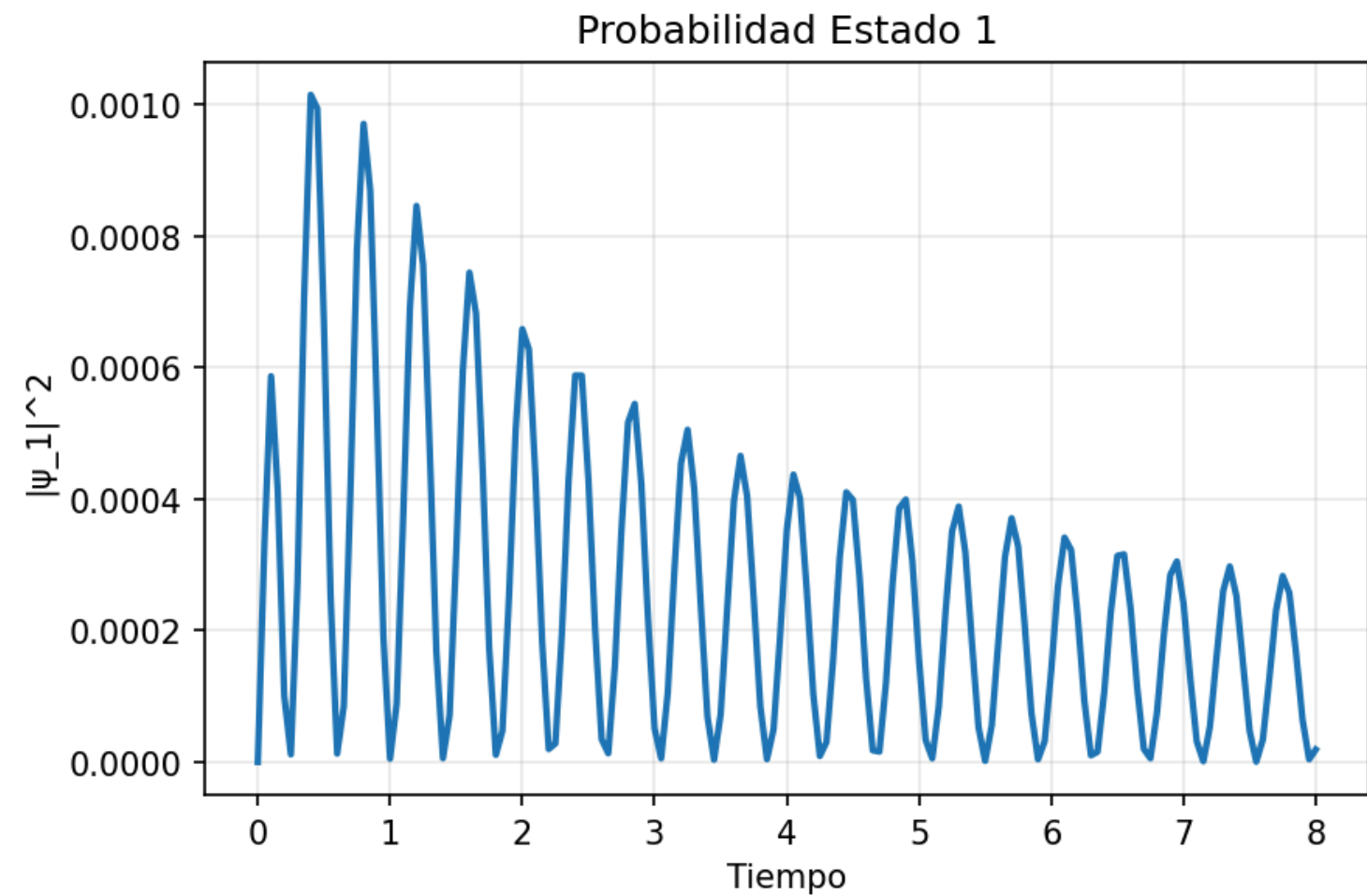
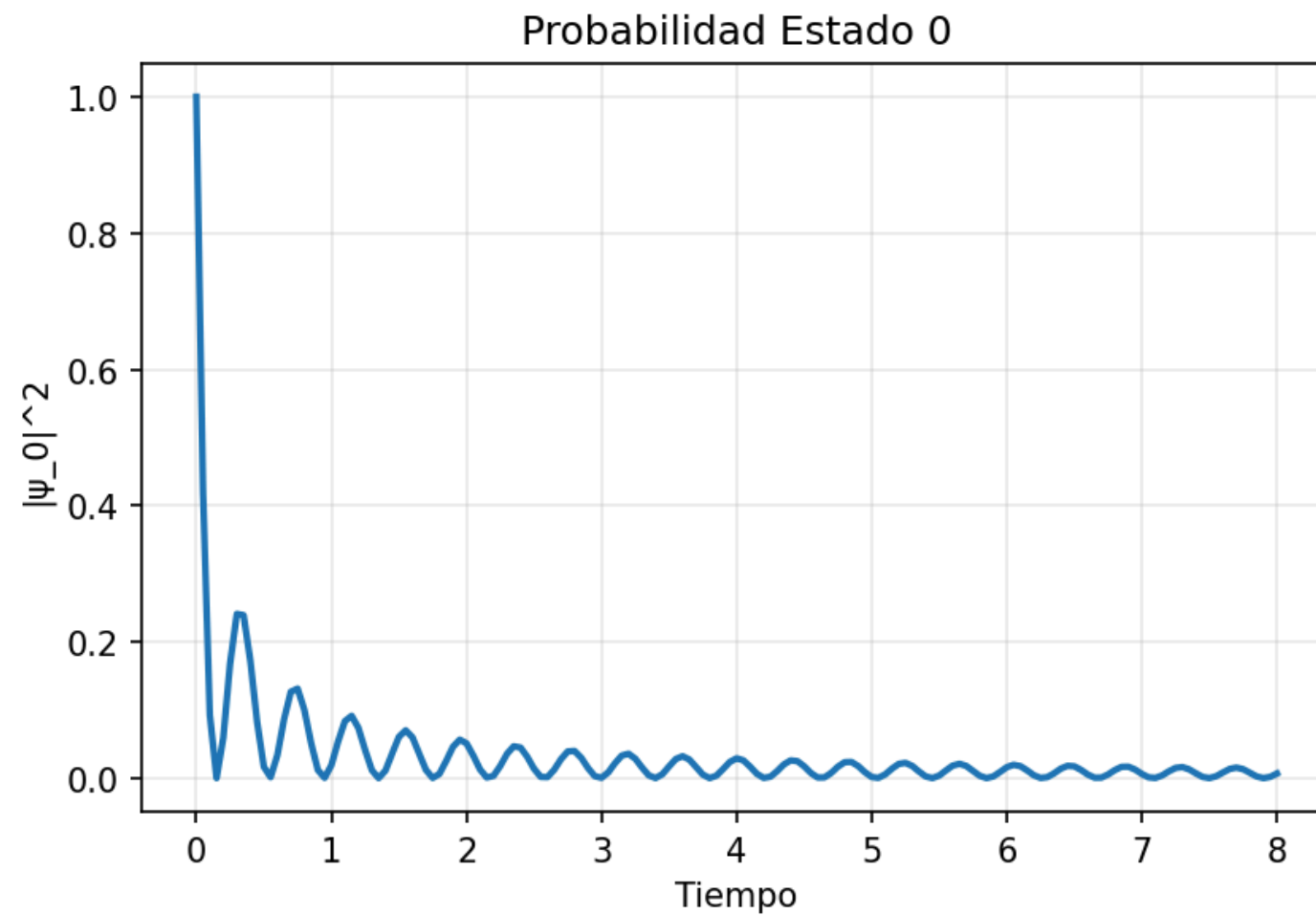
```
// Energía total
#pragma omp parallel for
for (int i = 0; i < nt; ++i) {
    VectorXcd state = states.col(i);
    Complex energy = state.adjoint() * (H_dense * state);
    energies[i] = energy.real();
}

// Densidad local de energía
#pragma omp parallel for collapse(2)
for (int i = 0; i < nt; ++i) {
    for (int j = 0; j < N; ++j) {
        VectorXcd state = states.col(i);
        MatrixXcd h_local_dense = MatrixXcd(h_local_list[j]);
        Complex e_local = state.adjoint() * (h_local_dense * state);
        local_energy(i, j) = e_local.real();
    }
}
```

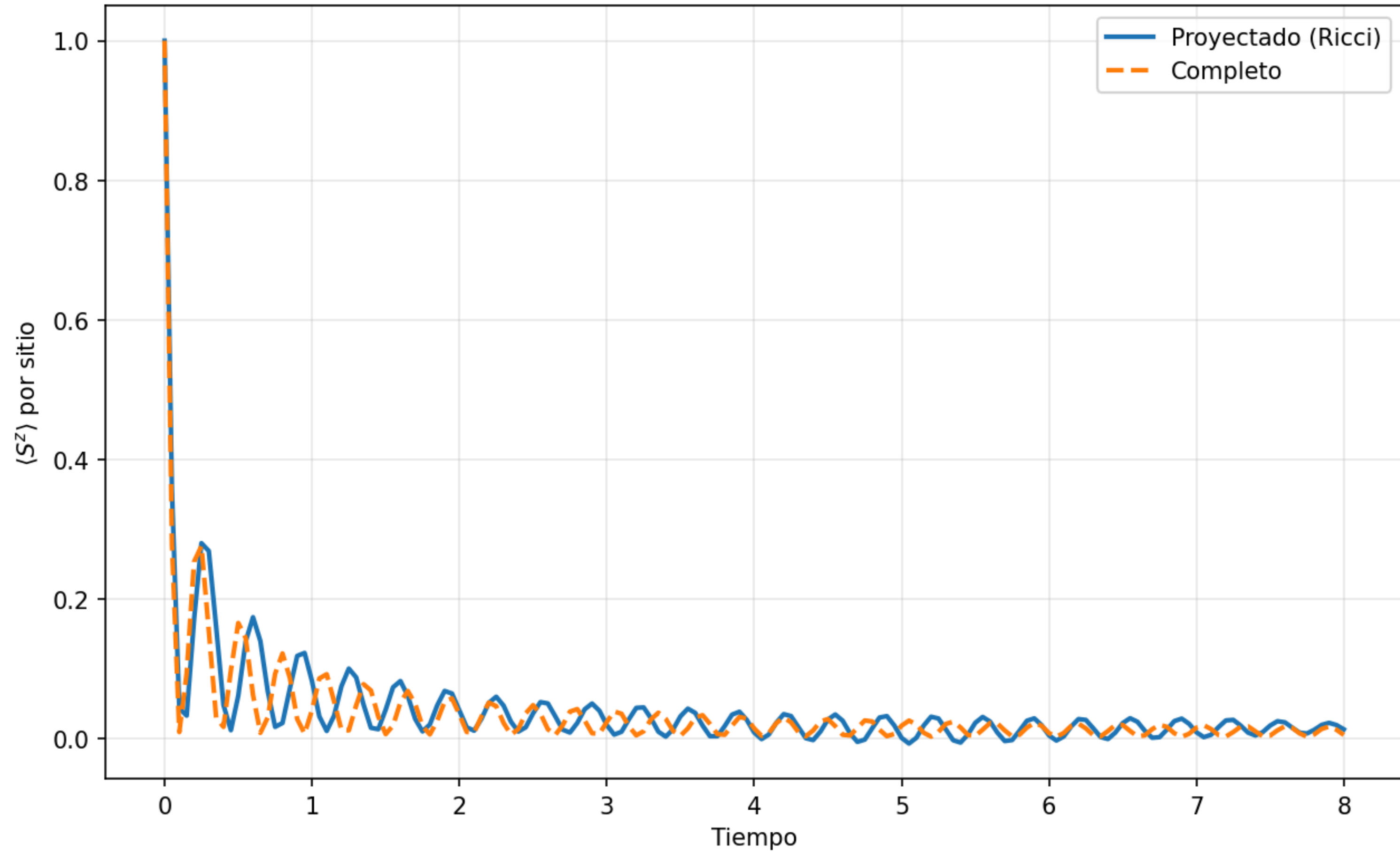

$$\langle \psi(t) | H | \psi(t) \rangle$$

Resultados en Python

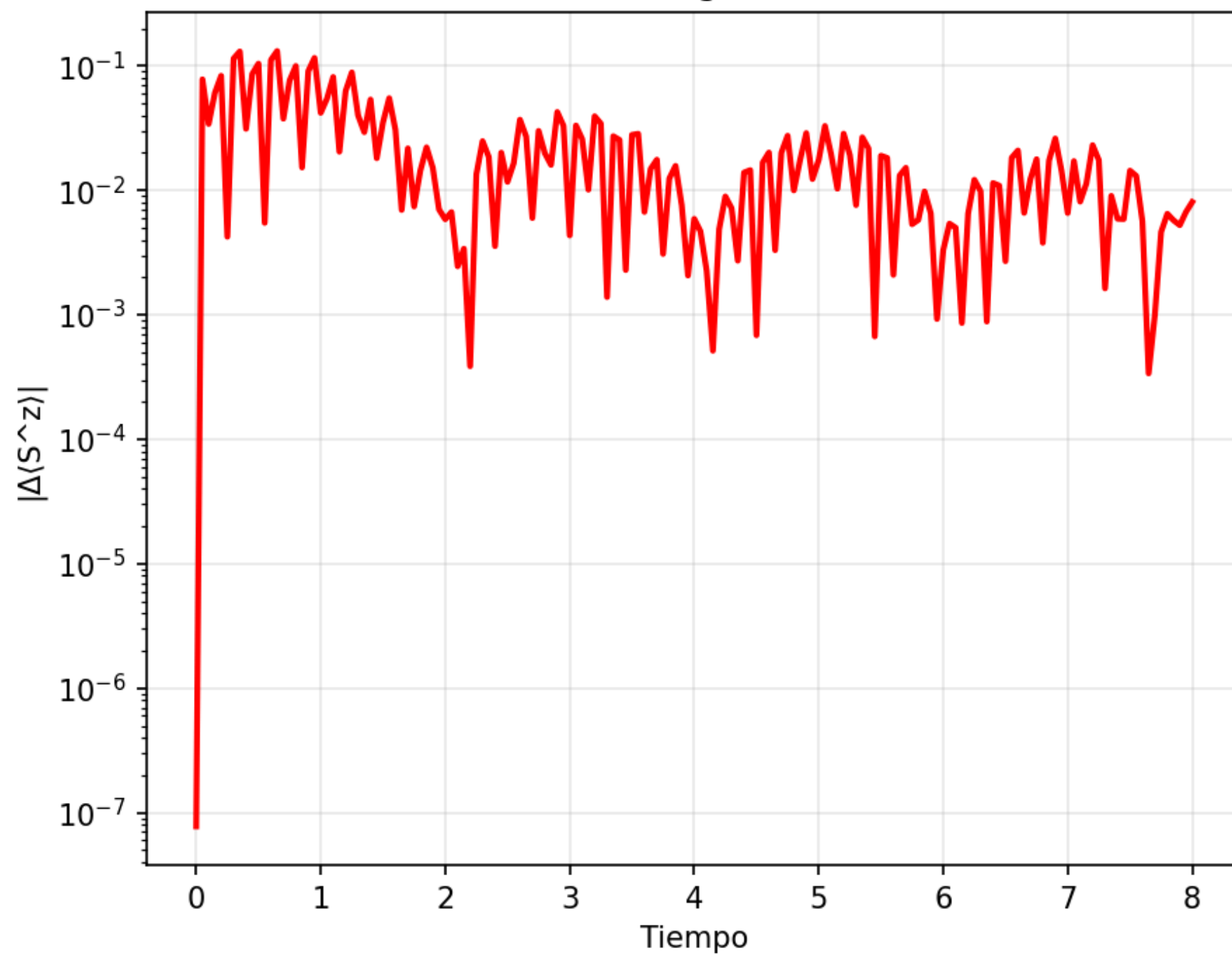




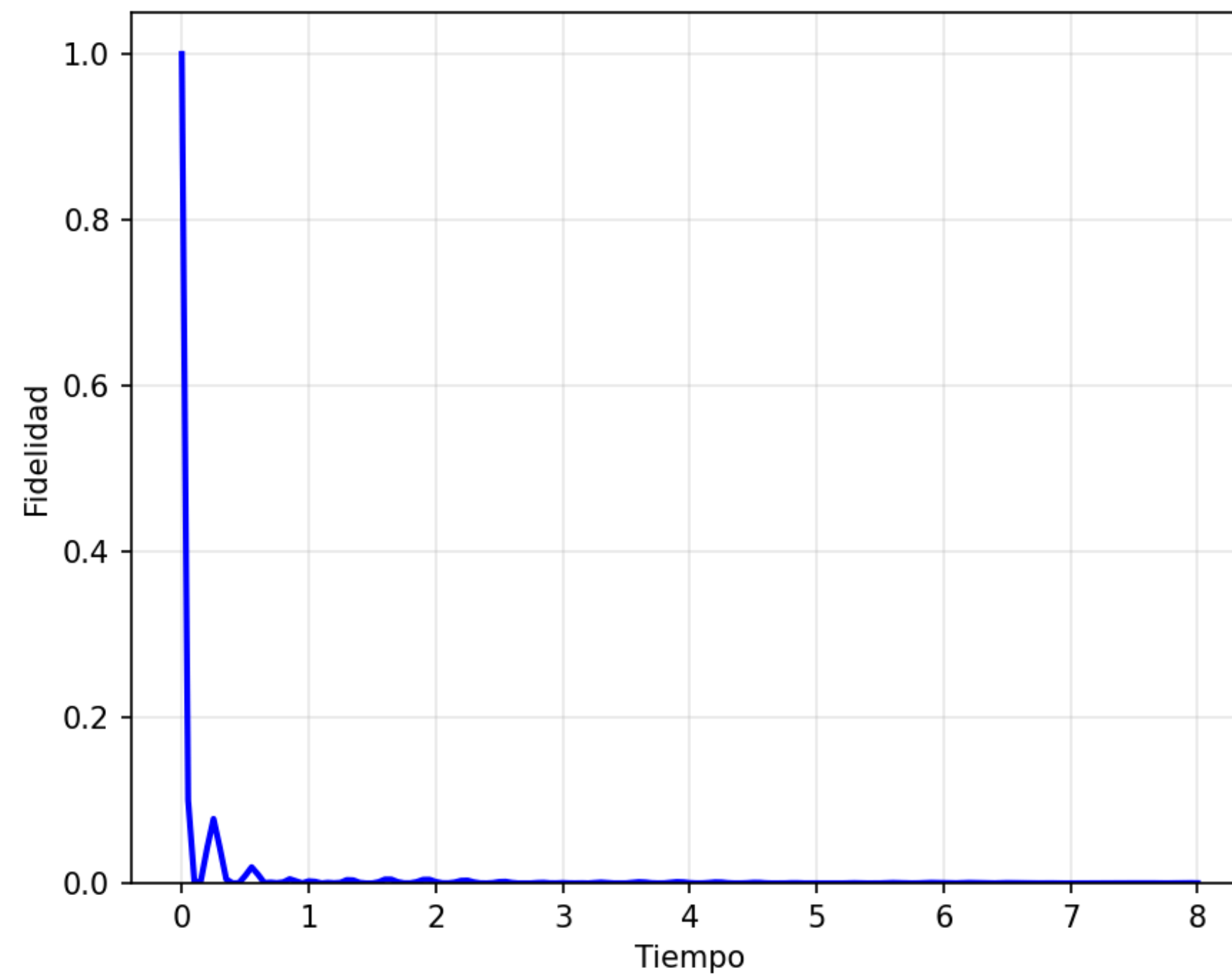
Magnetización: Ricci vs Completo

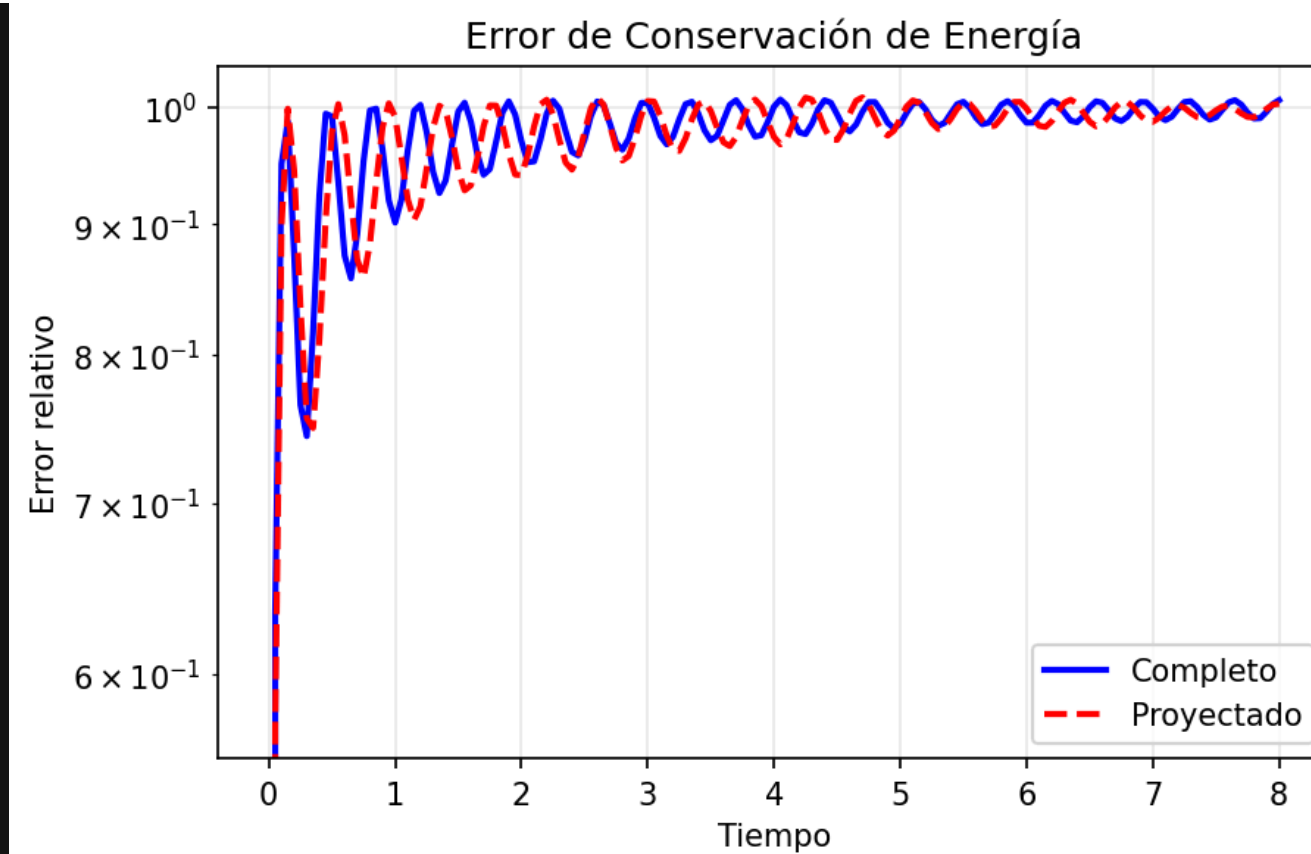
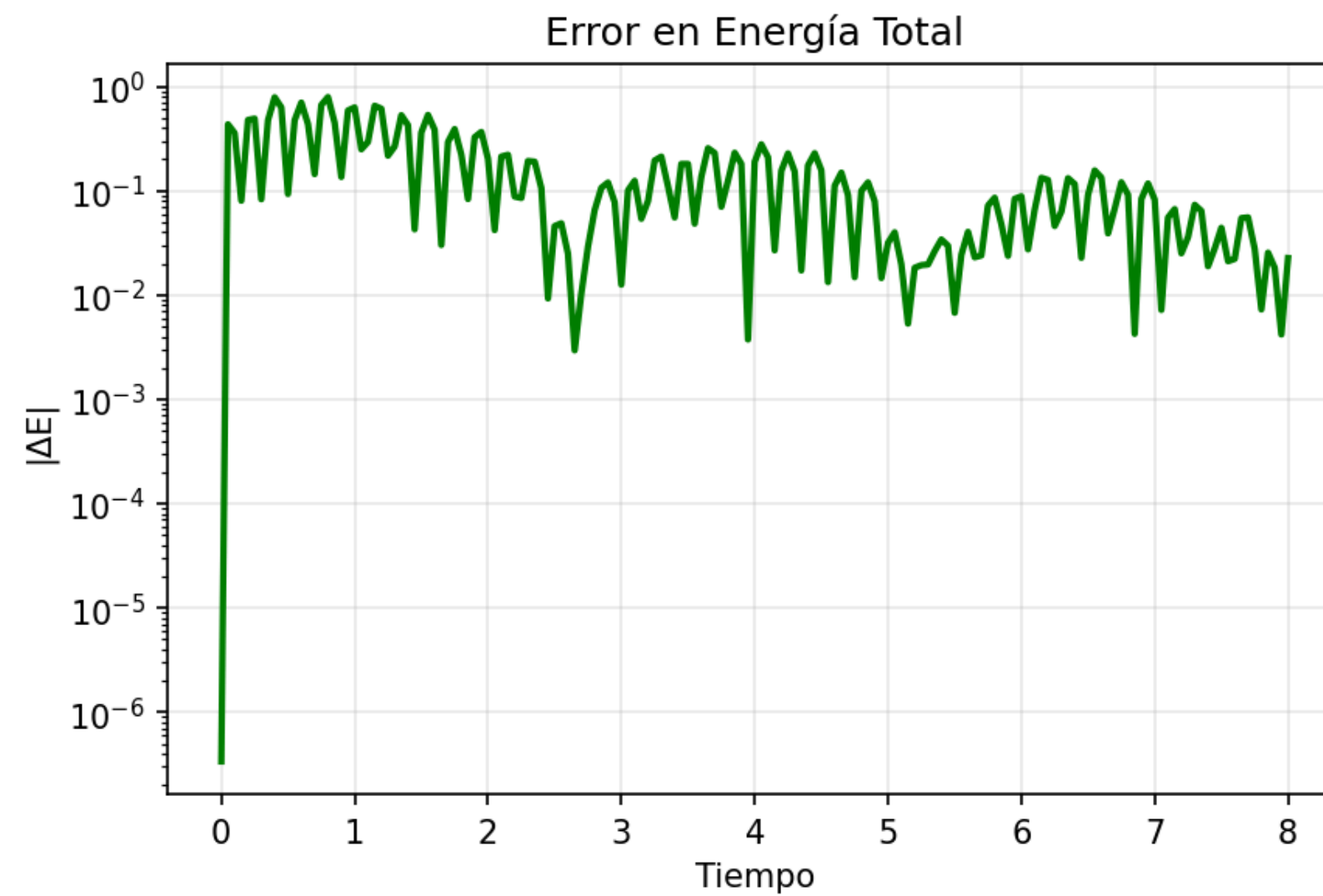
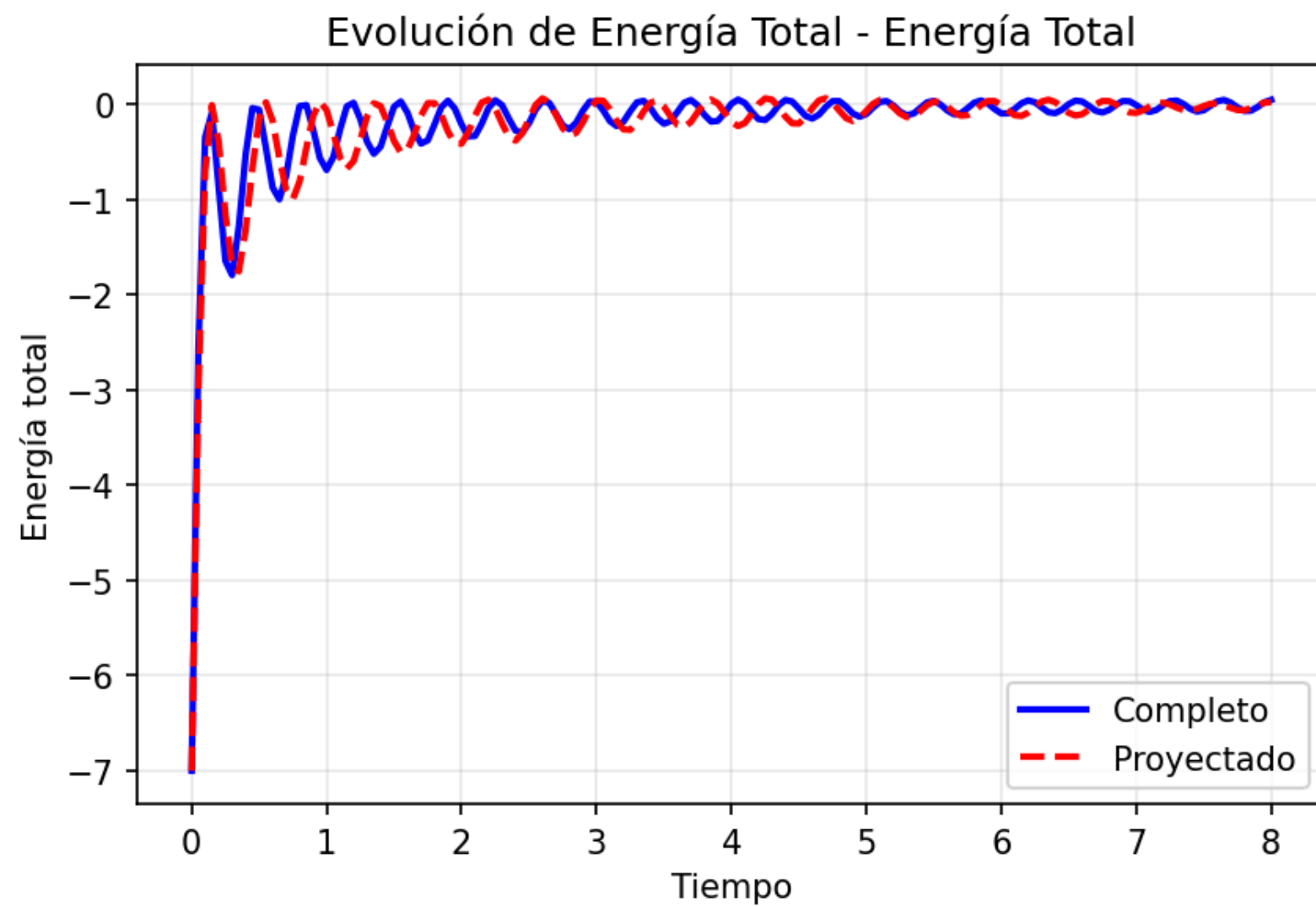


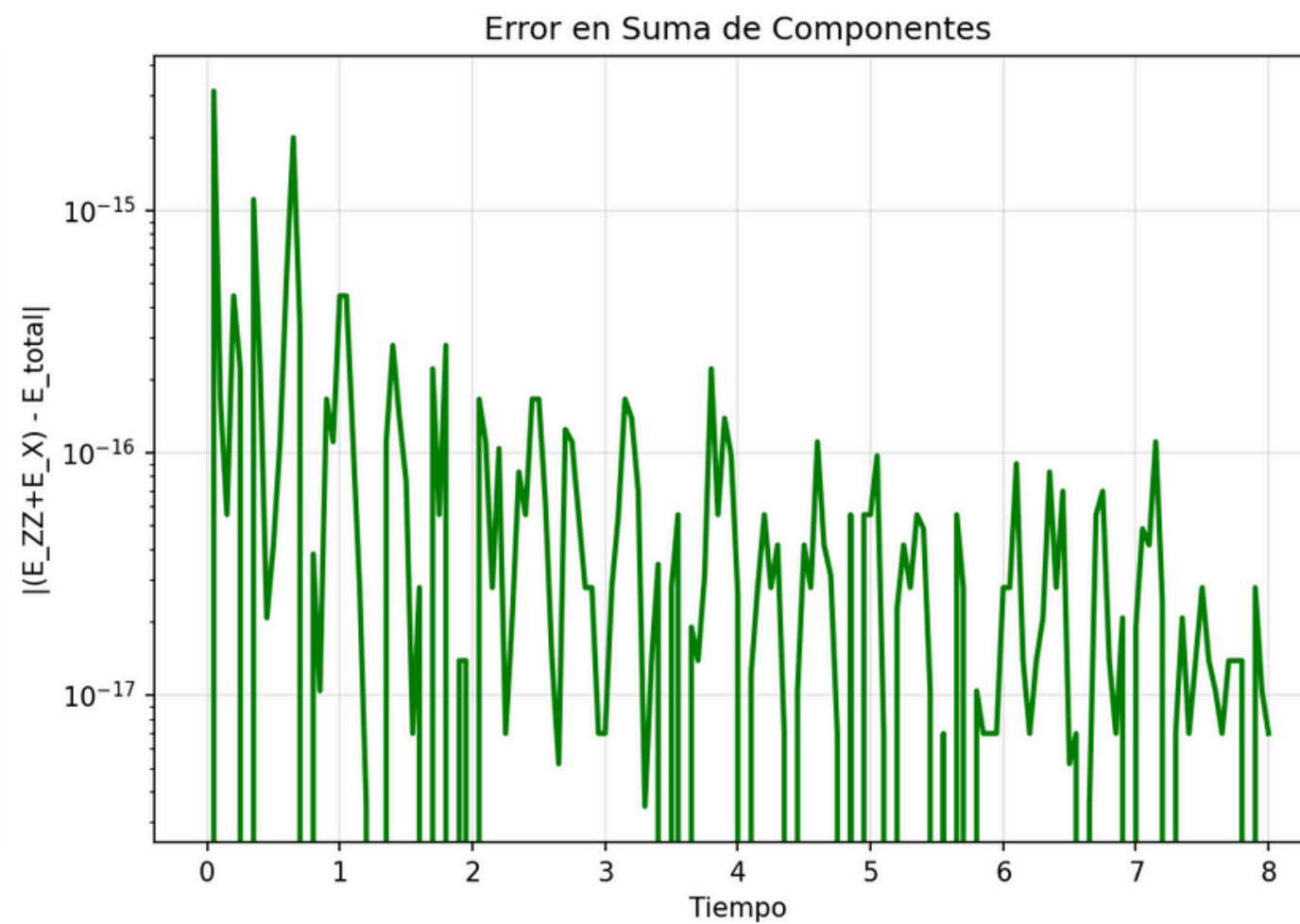
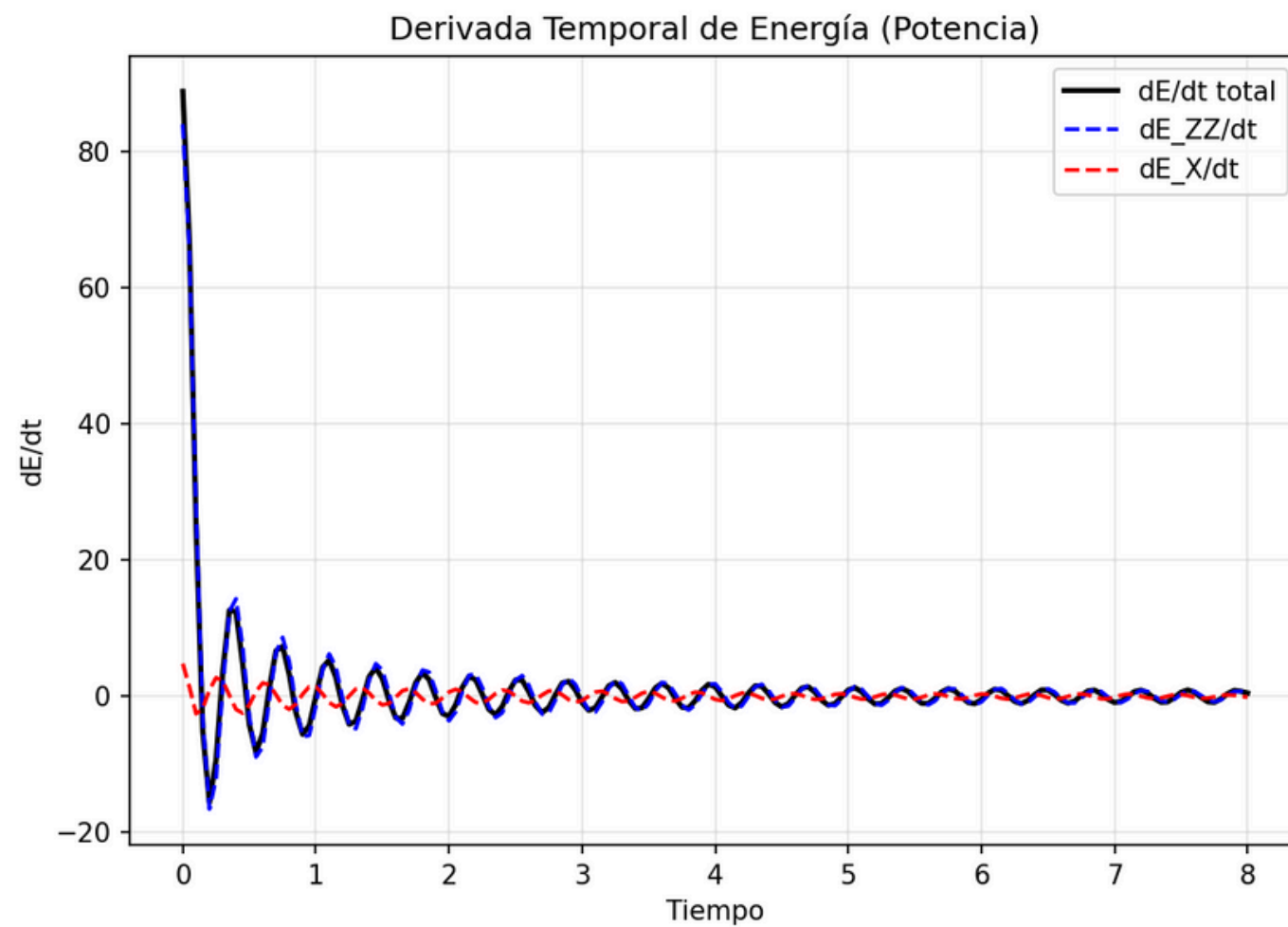
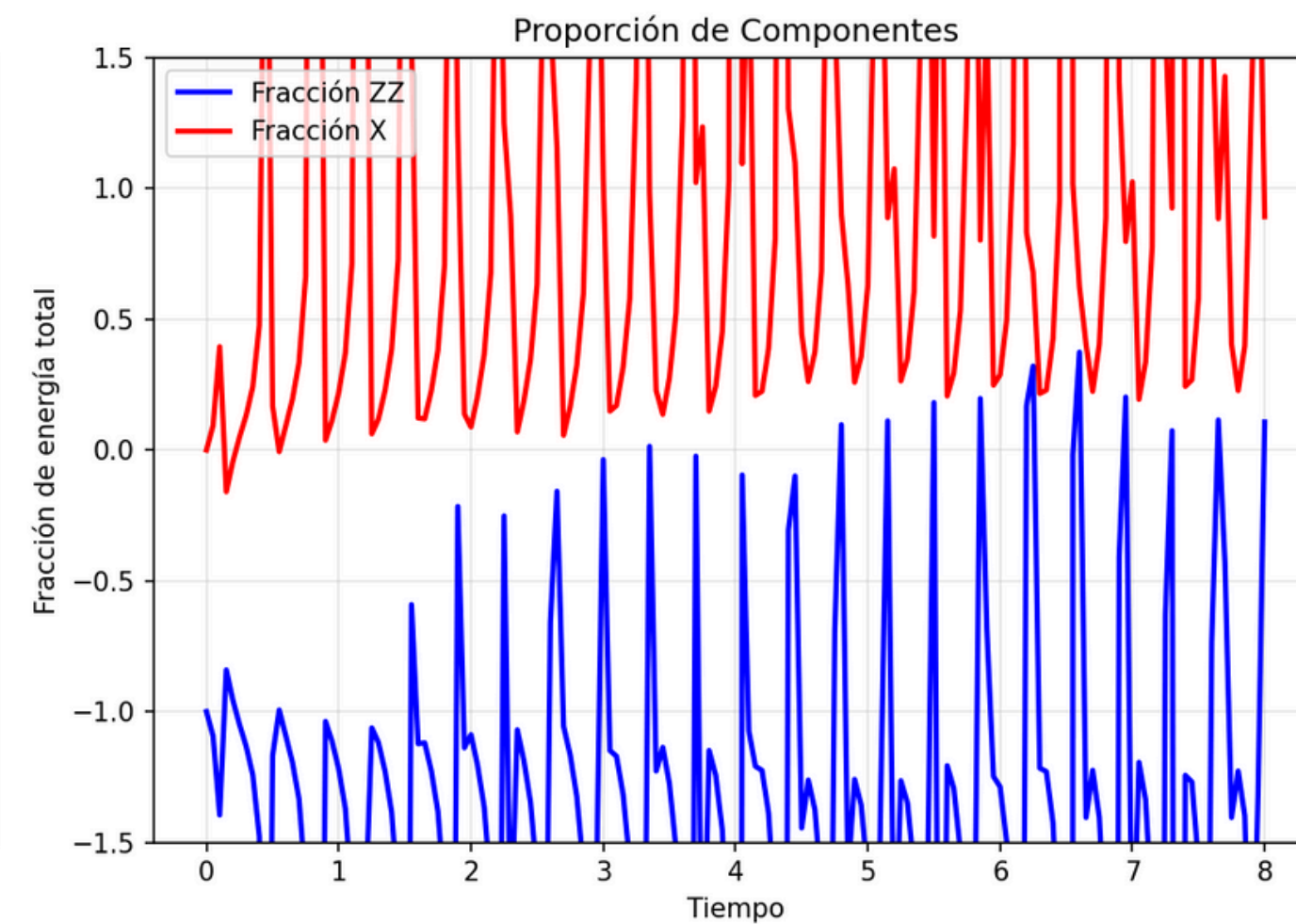
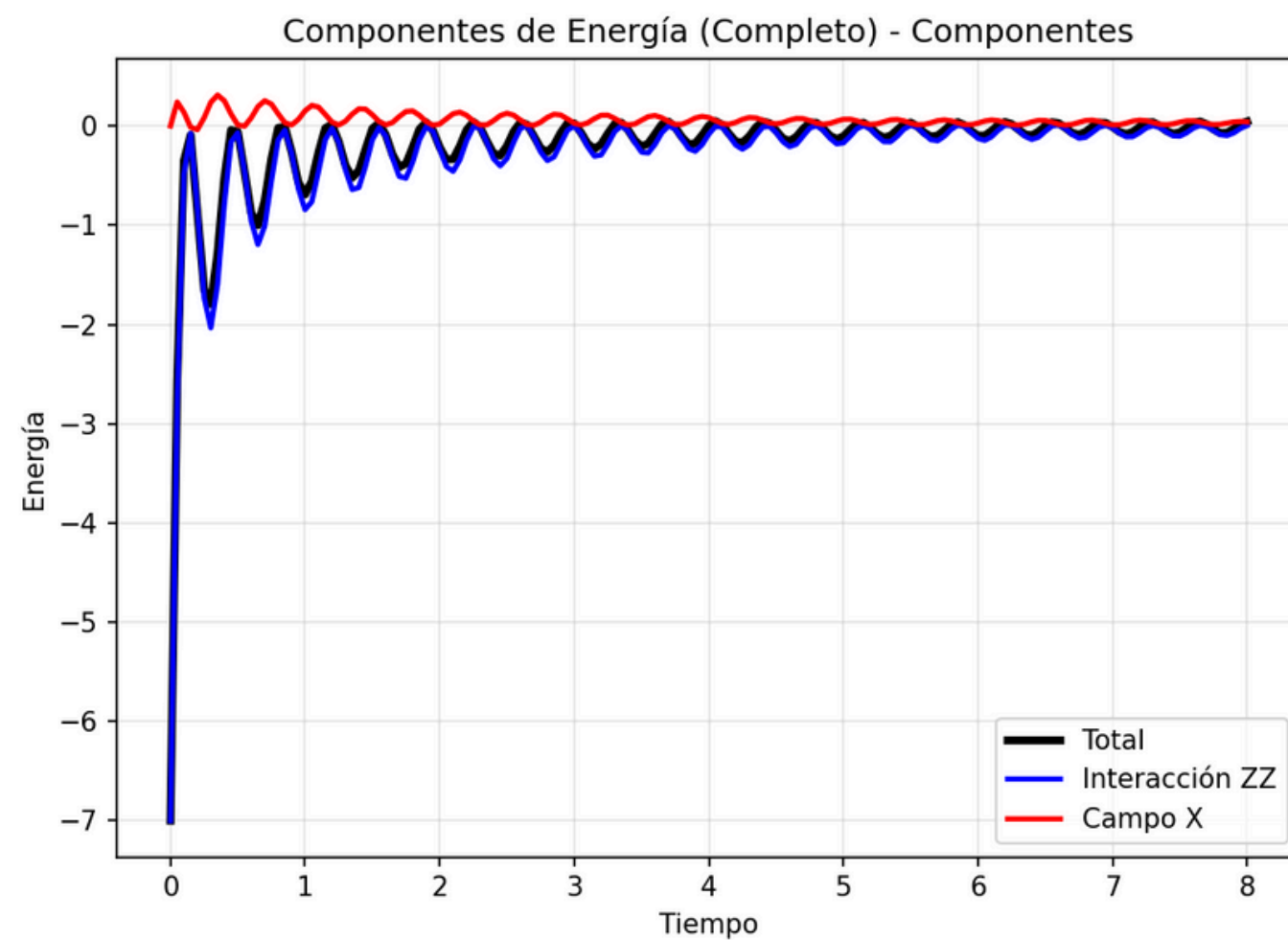
Error en magnetización

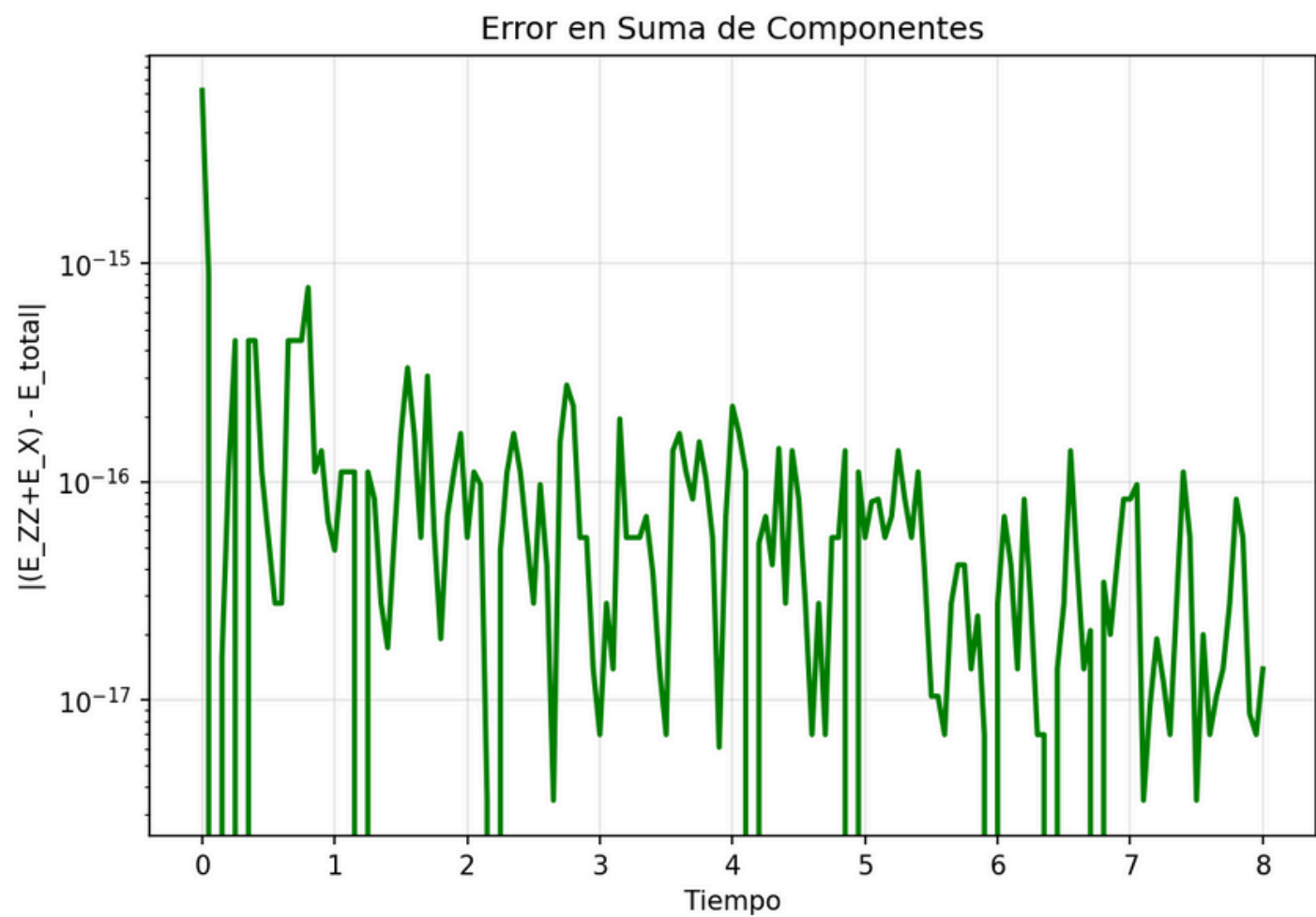
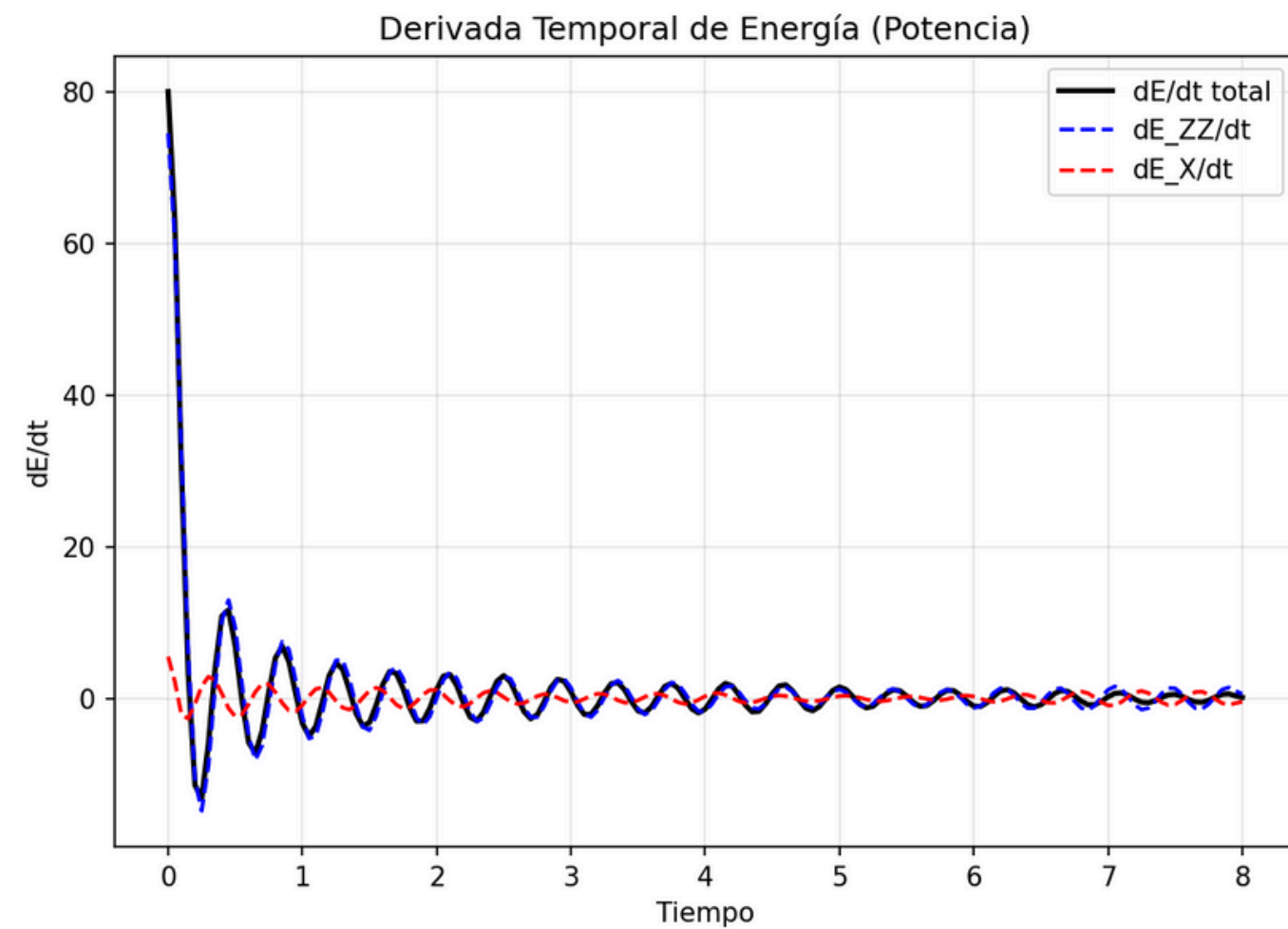
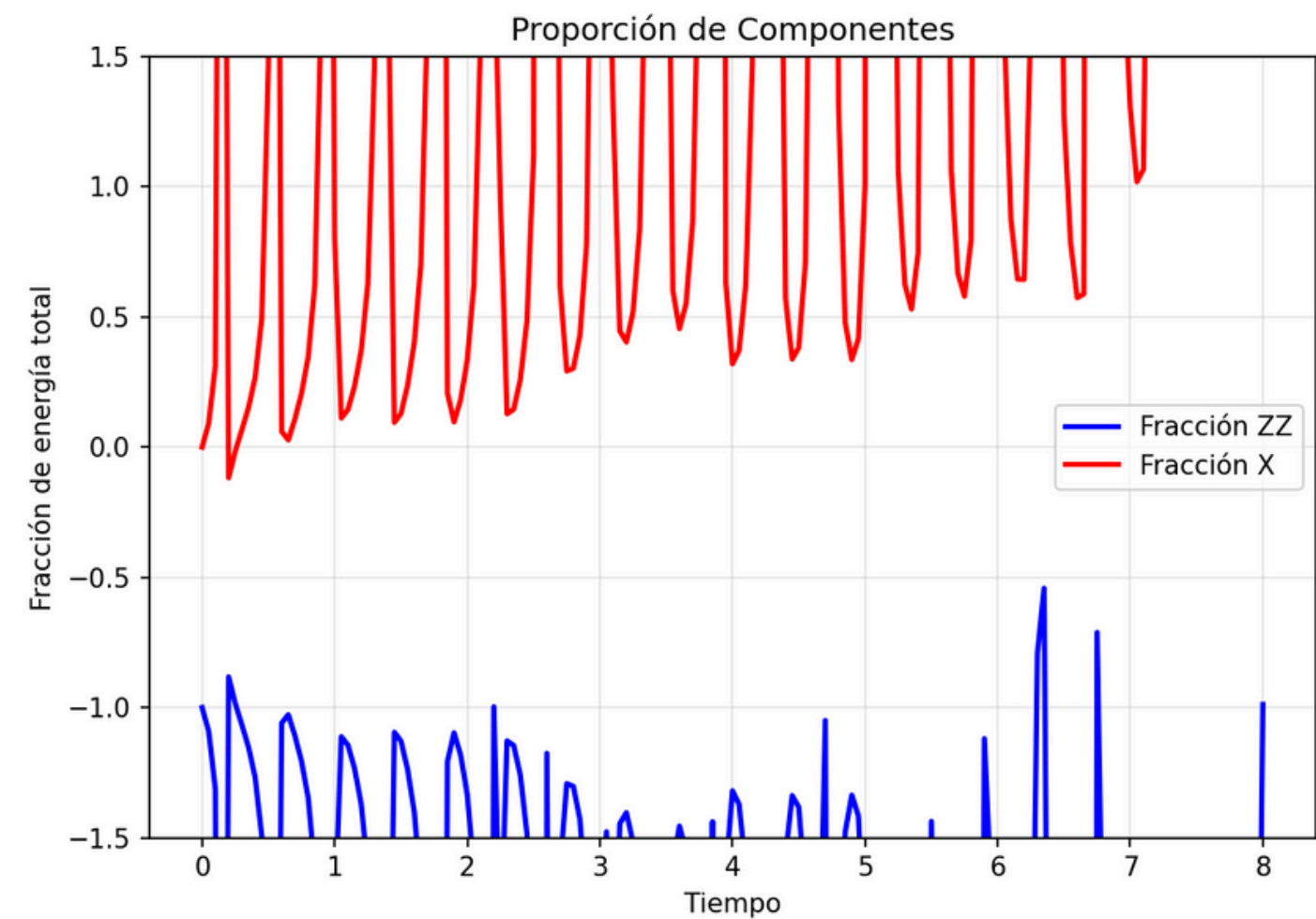
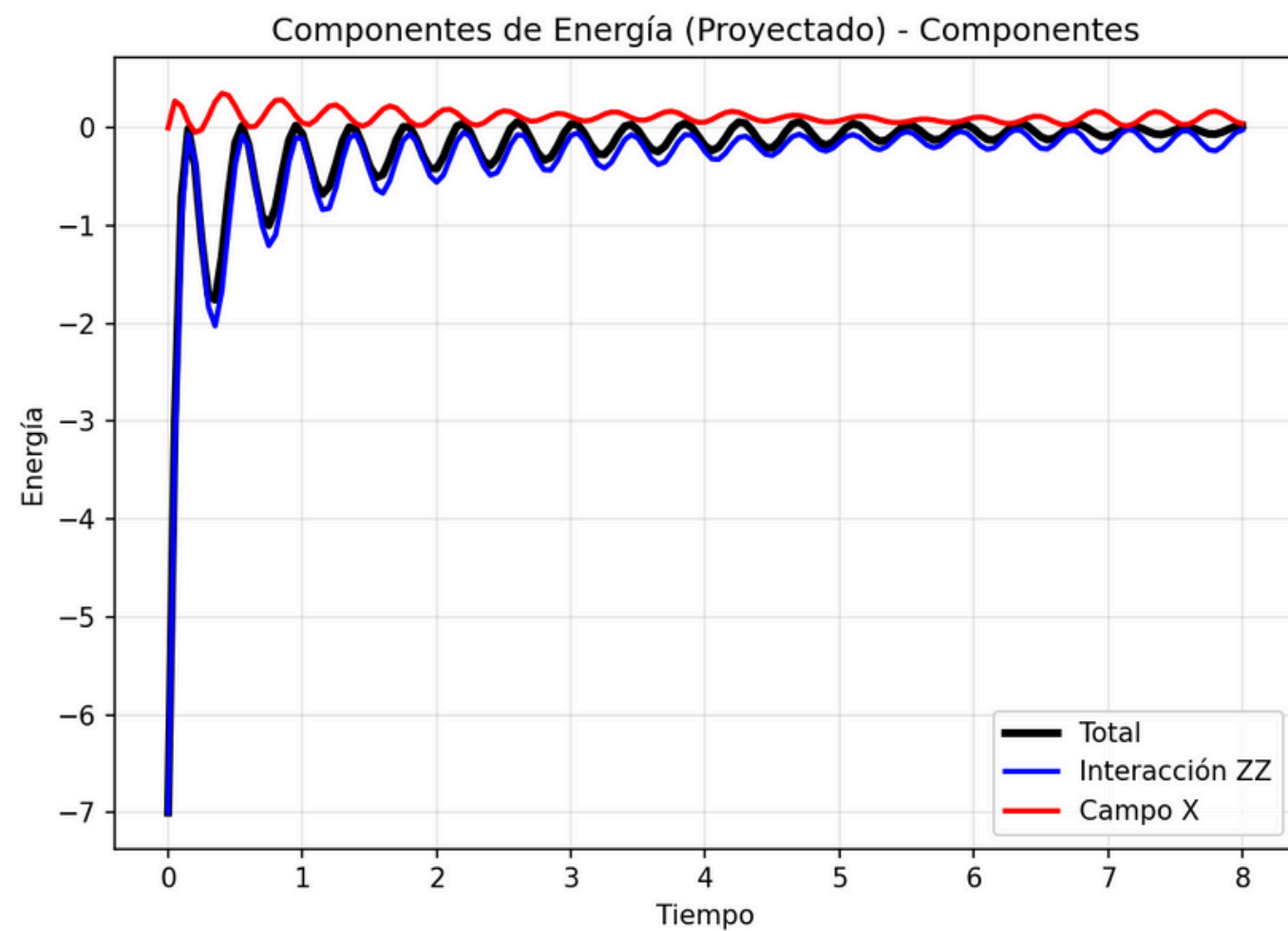


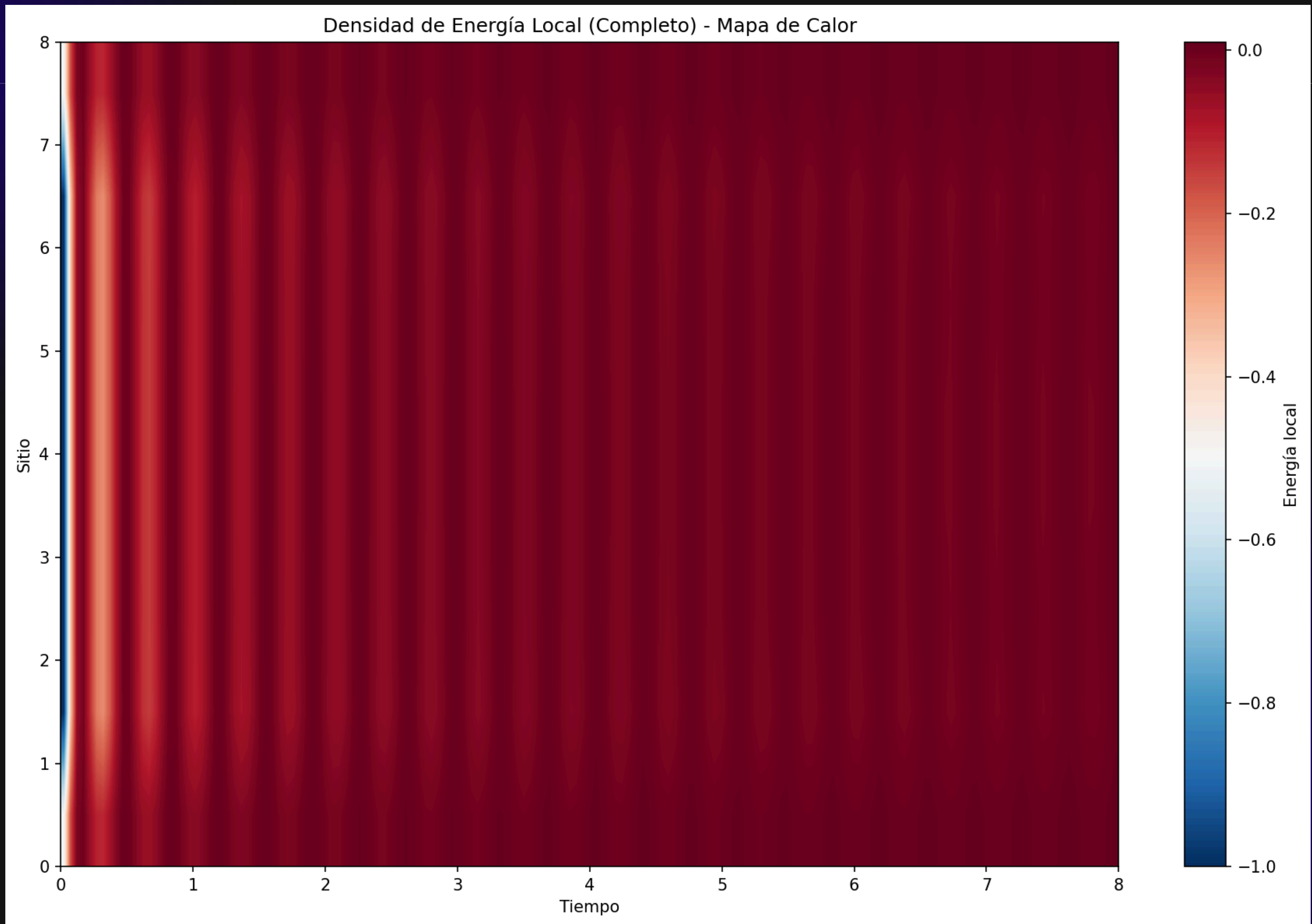
Fidelidad durante la evolución



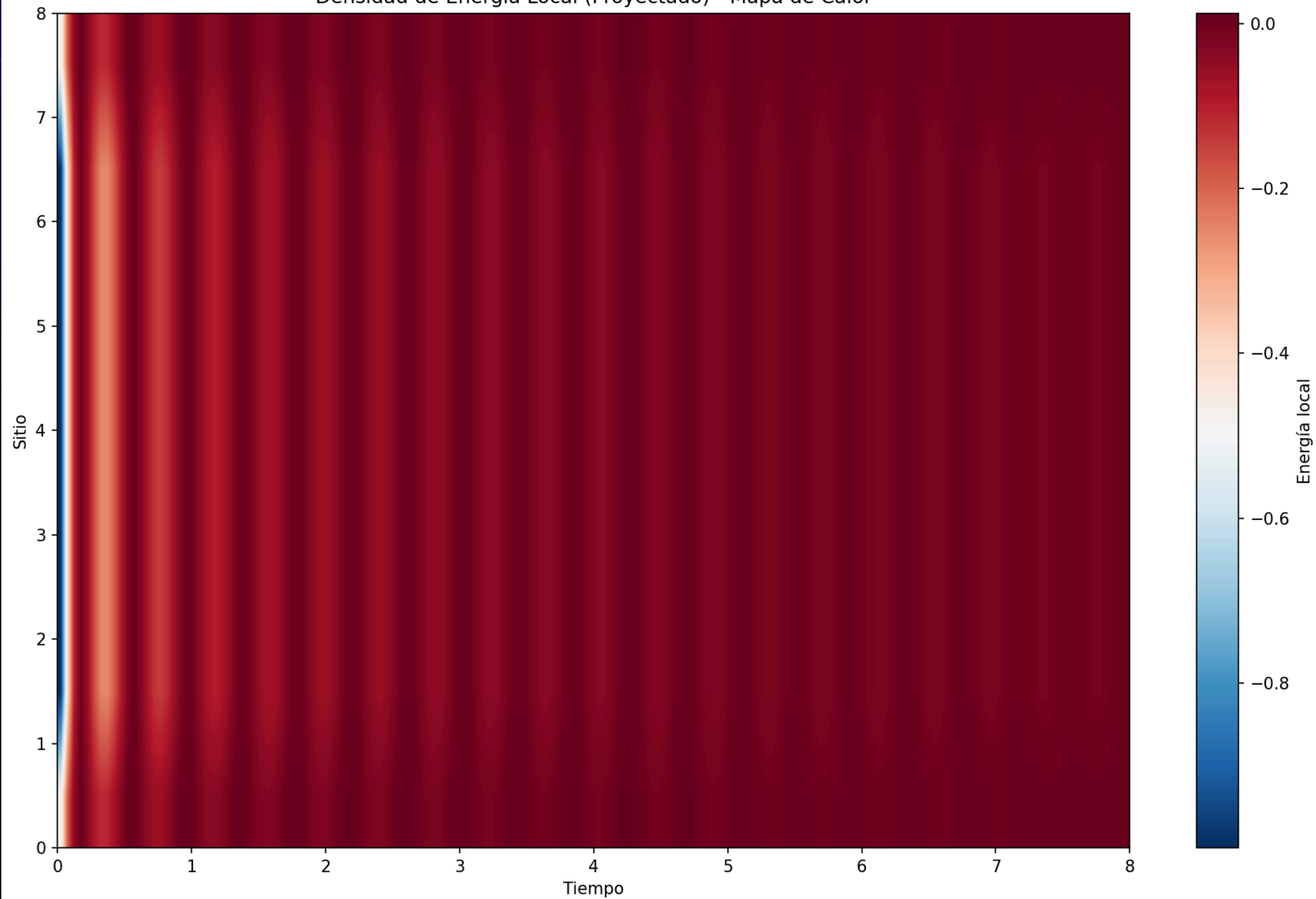




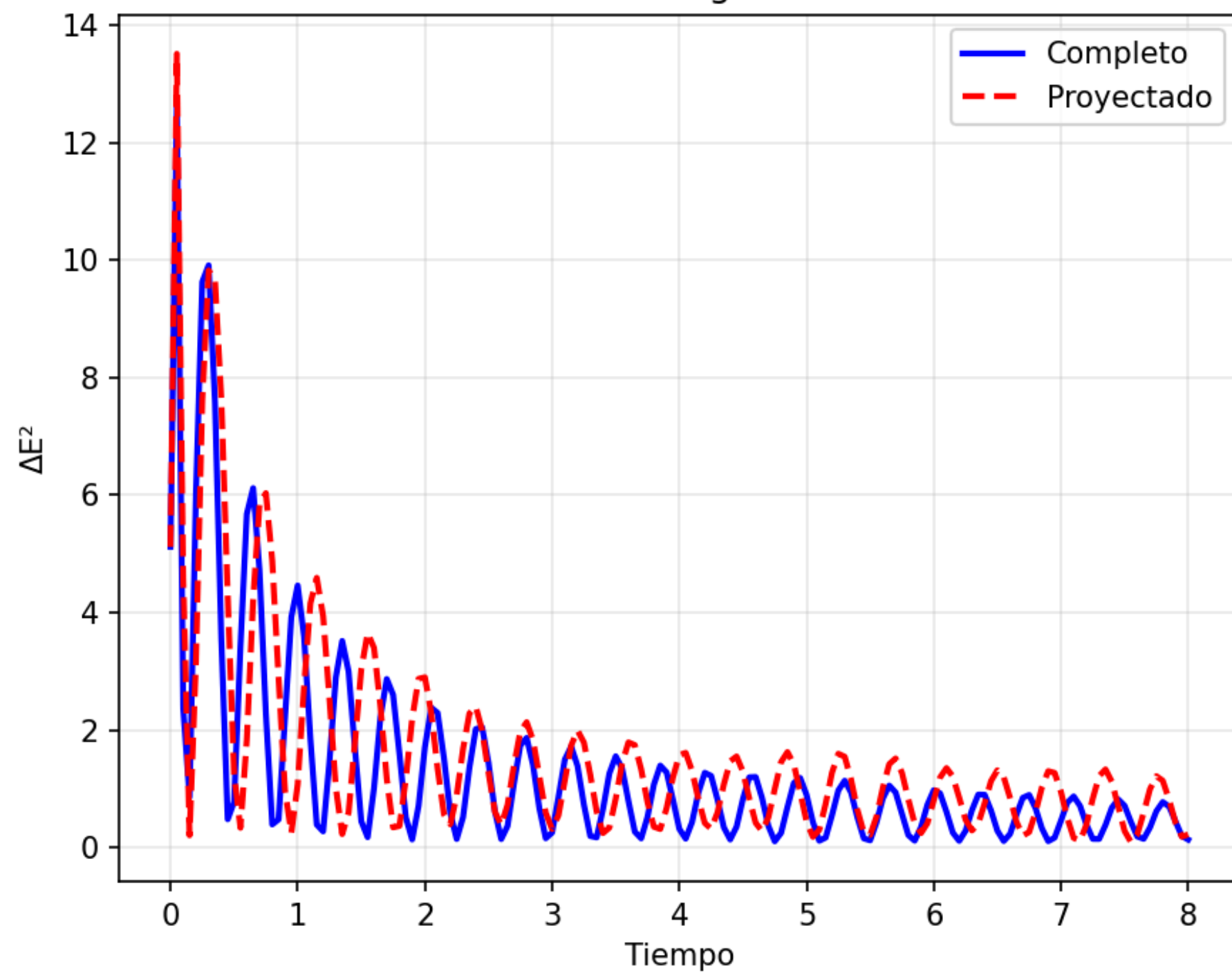




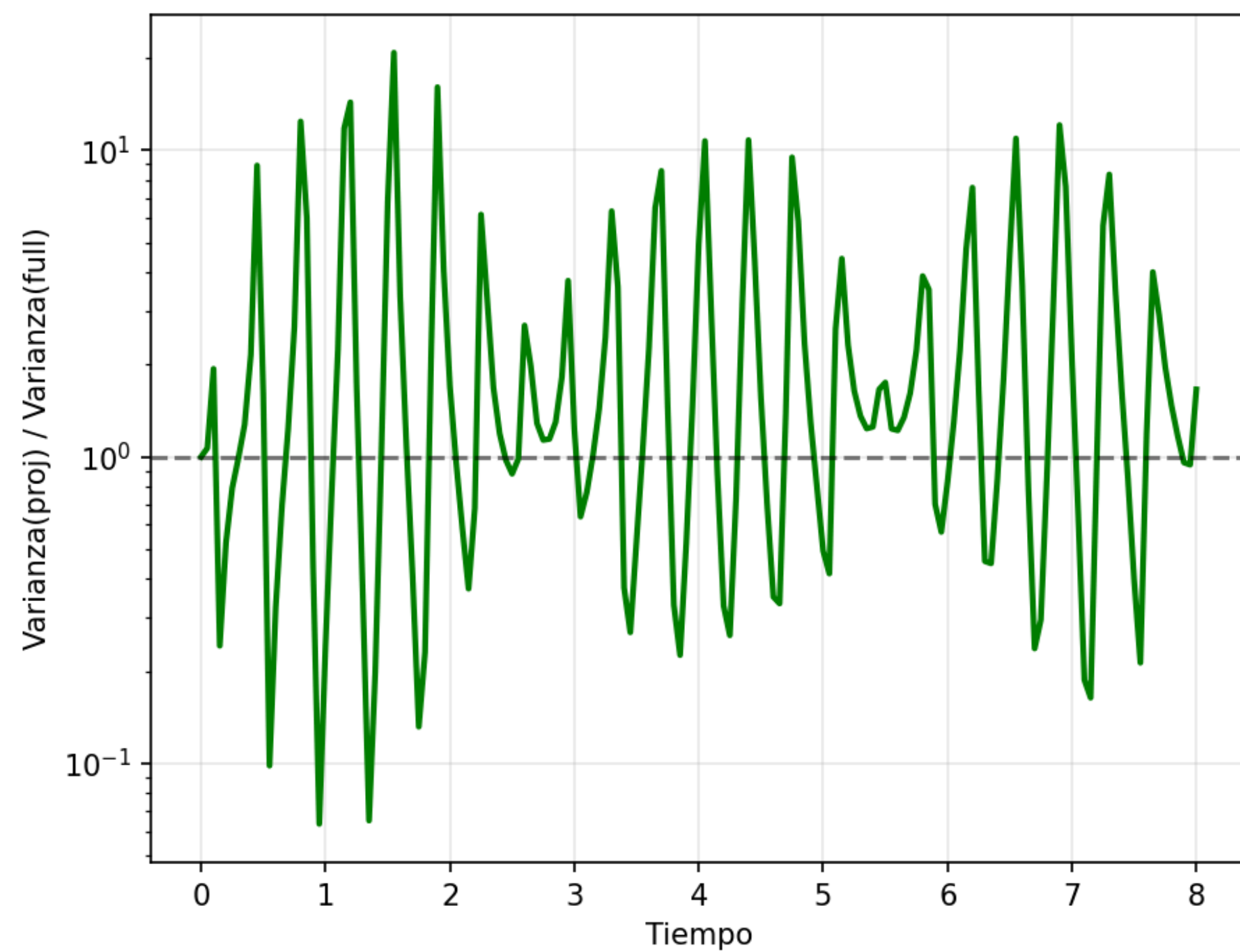
Densidad de Energía Local (Proyectado) - Mapa de Calor



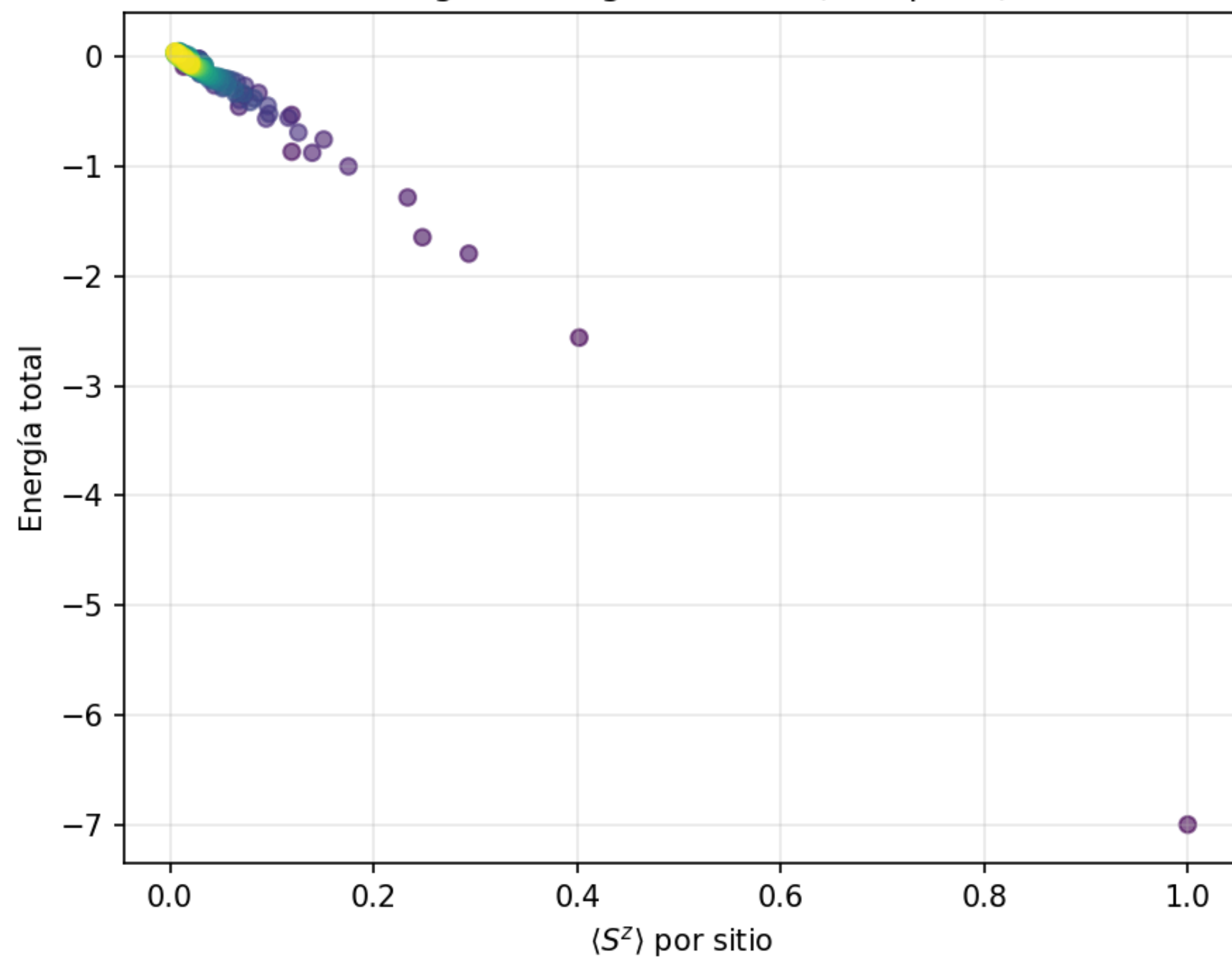
Varianza de Energía - Varianza



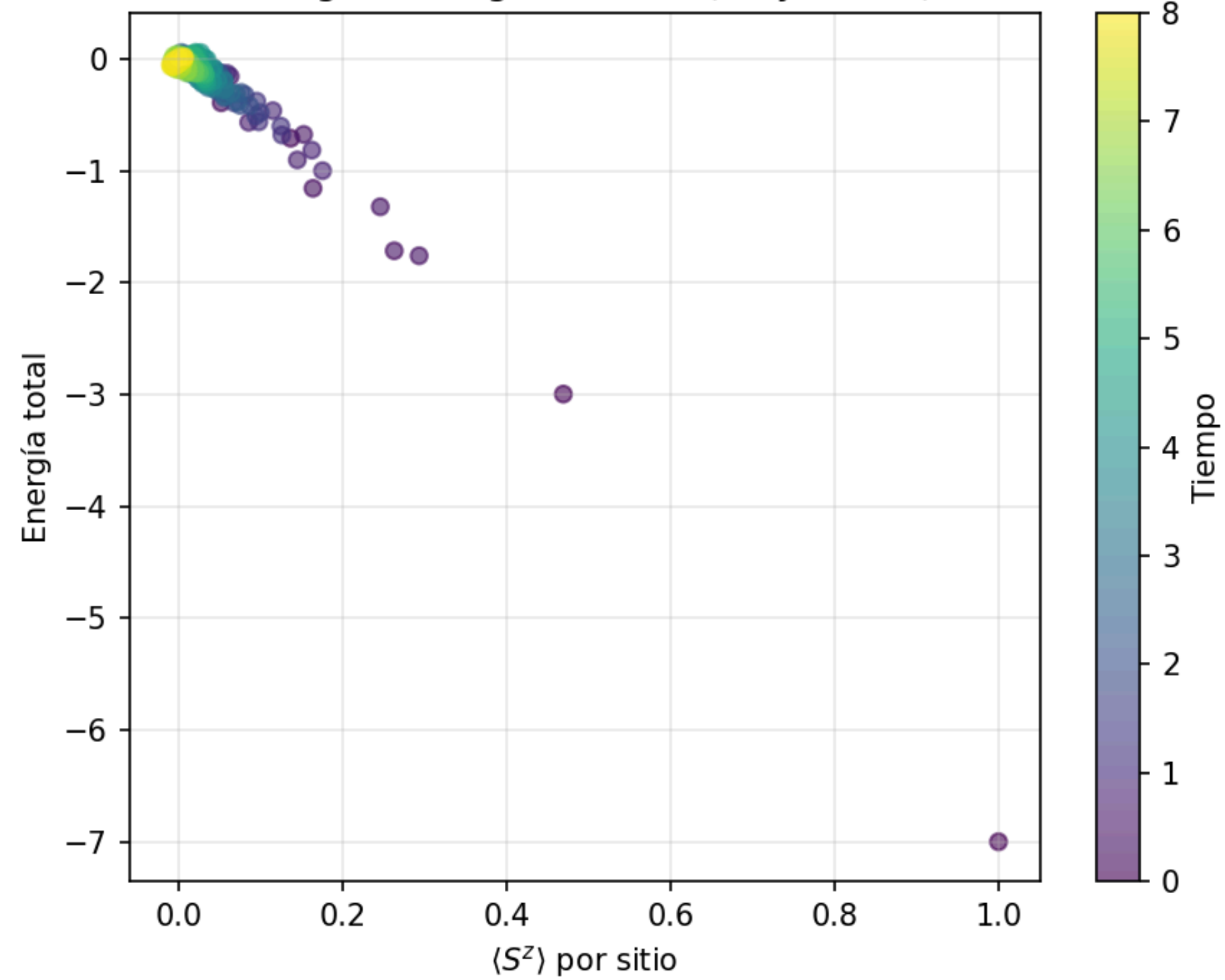
Ratio de Varianzas



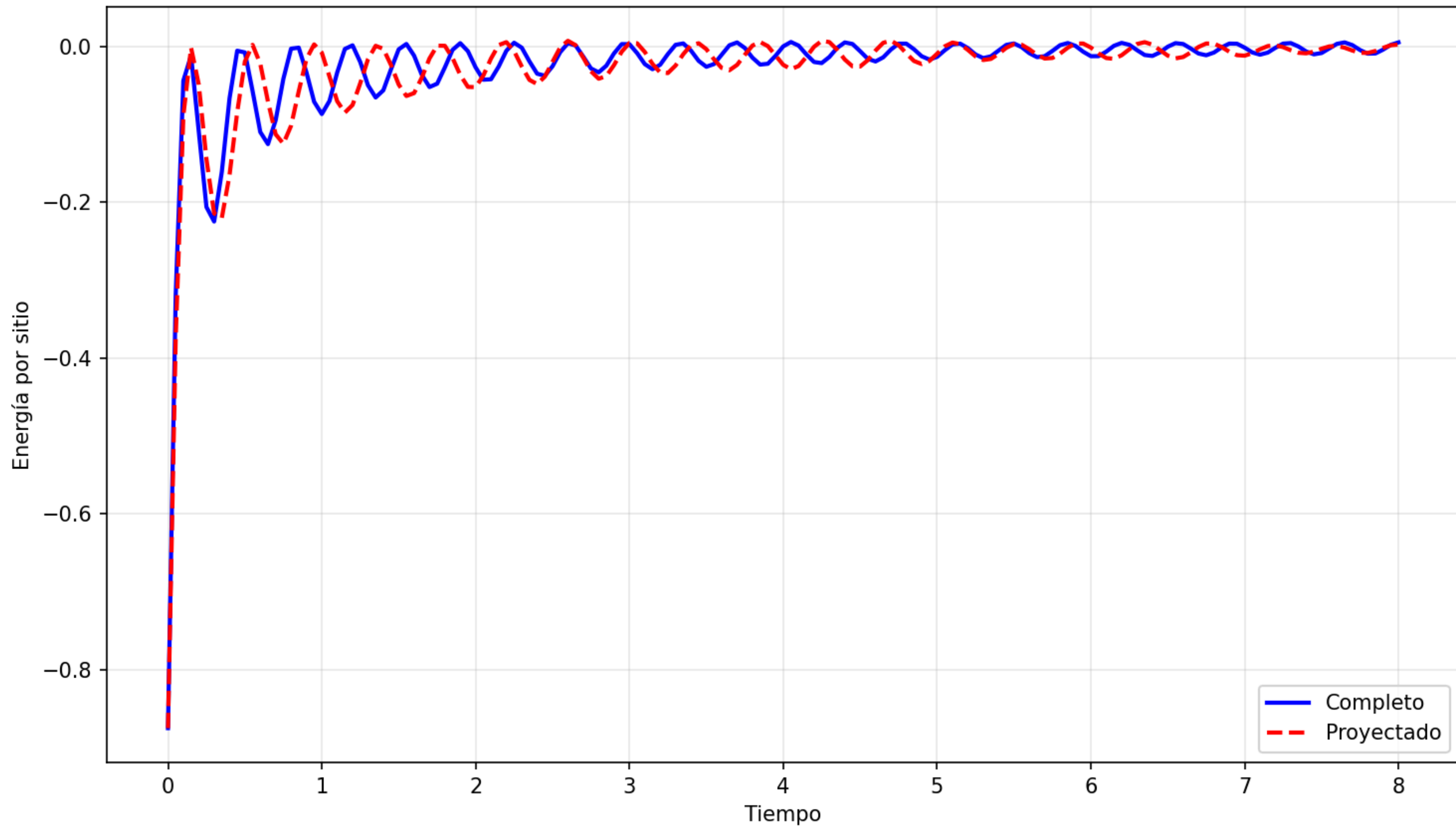
Energía vs Magnetización (Completo)

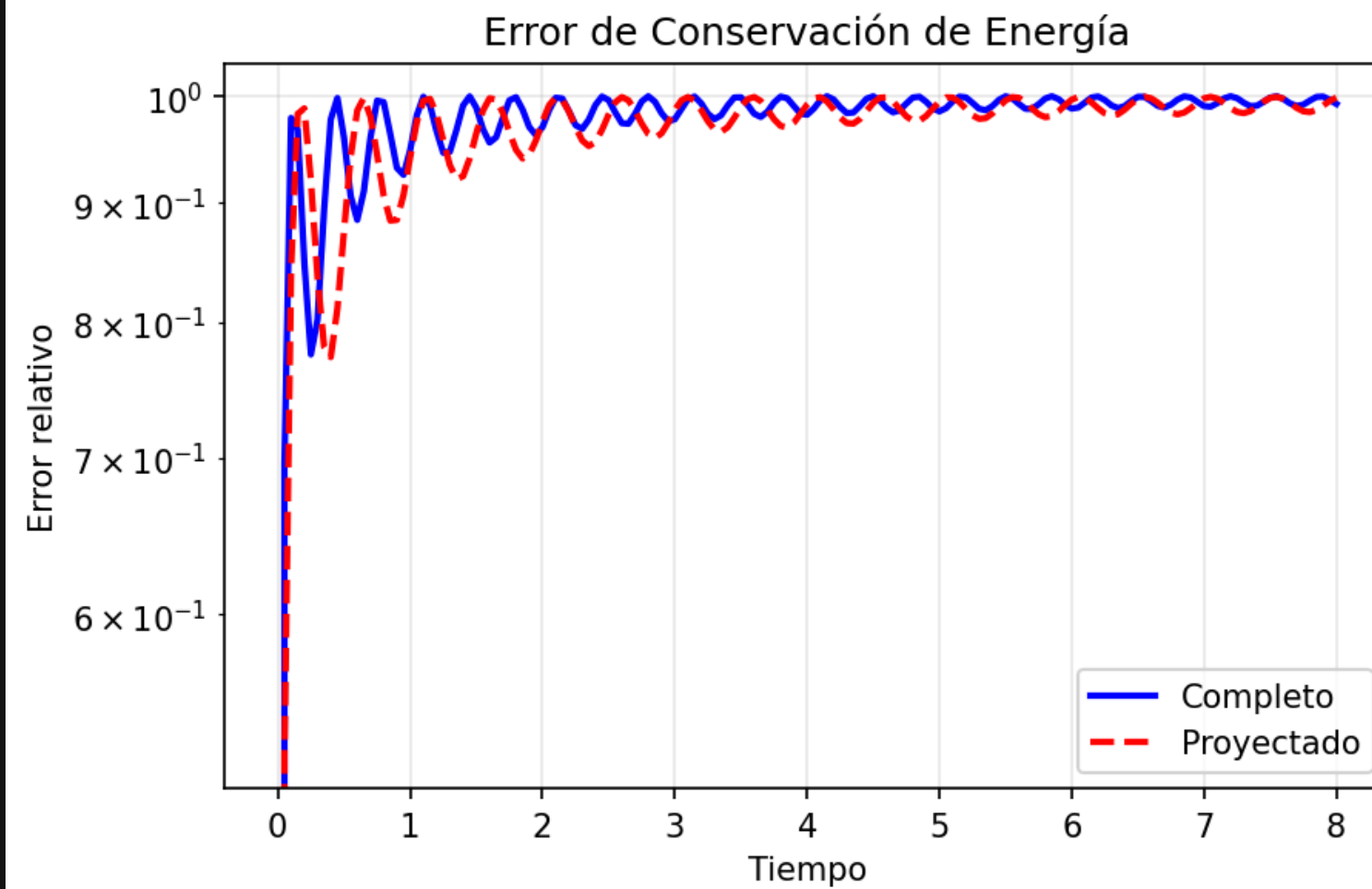
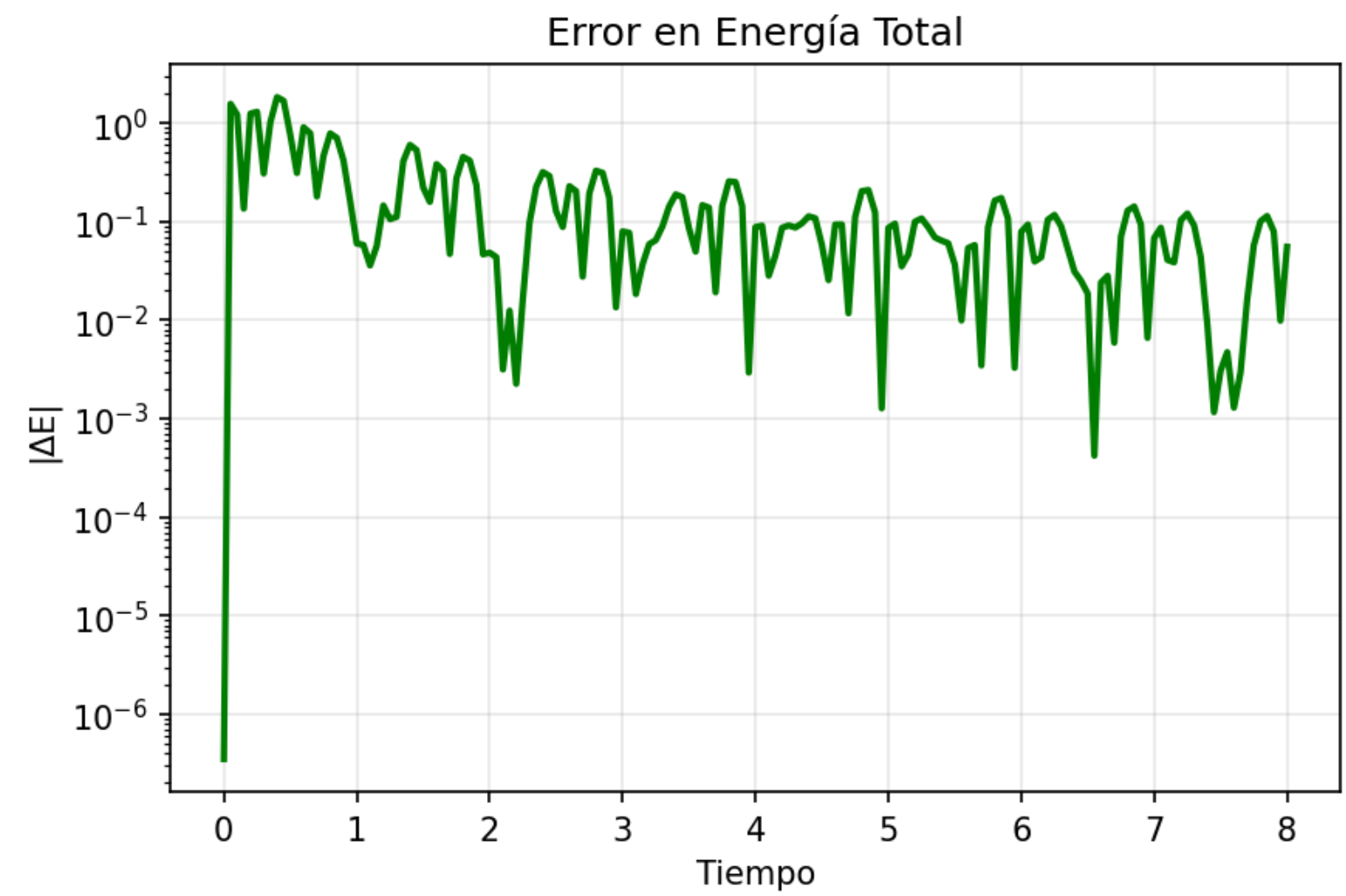
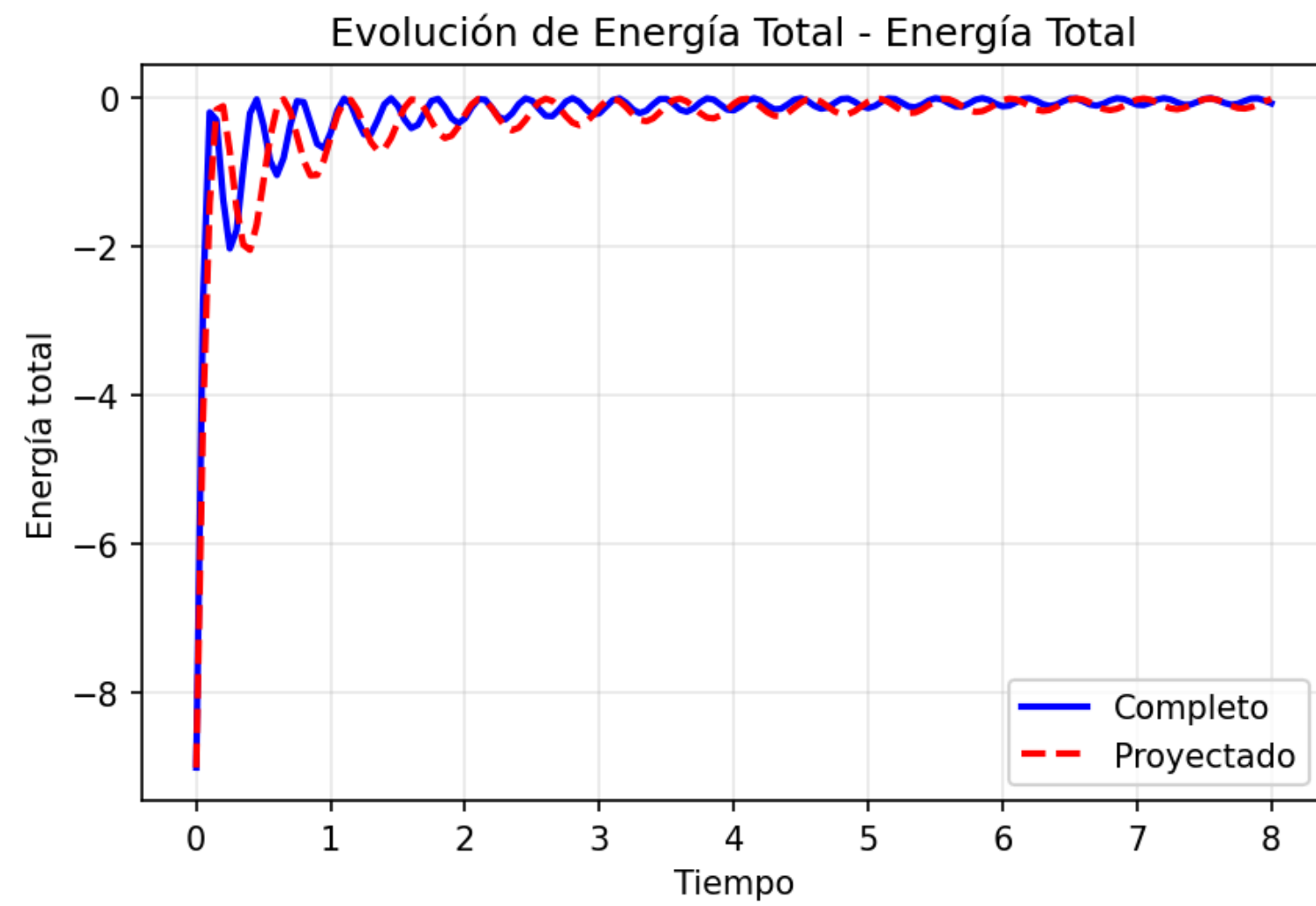


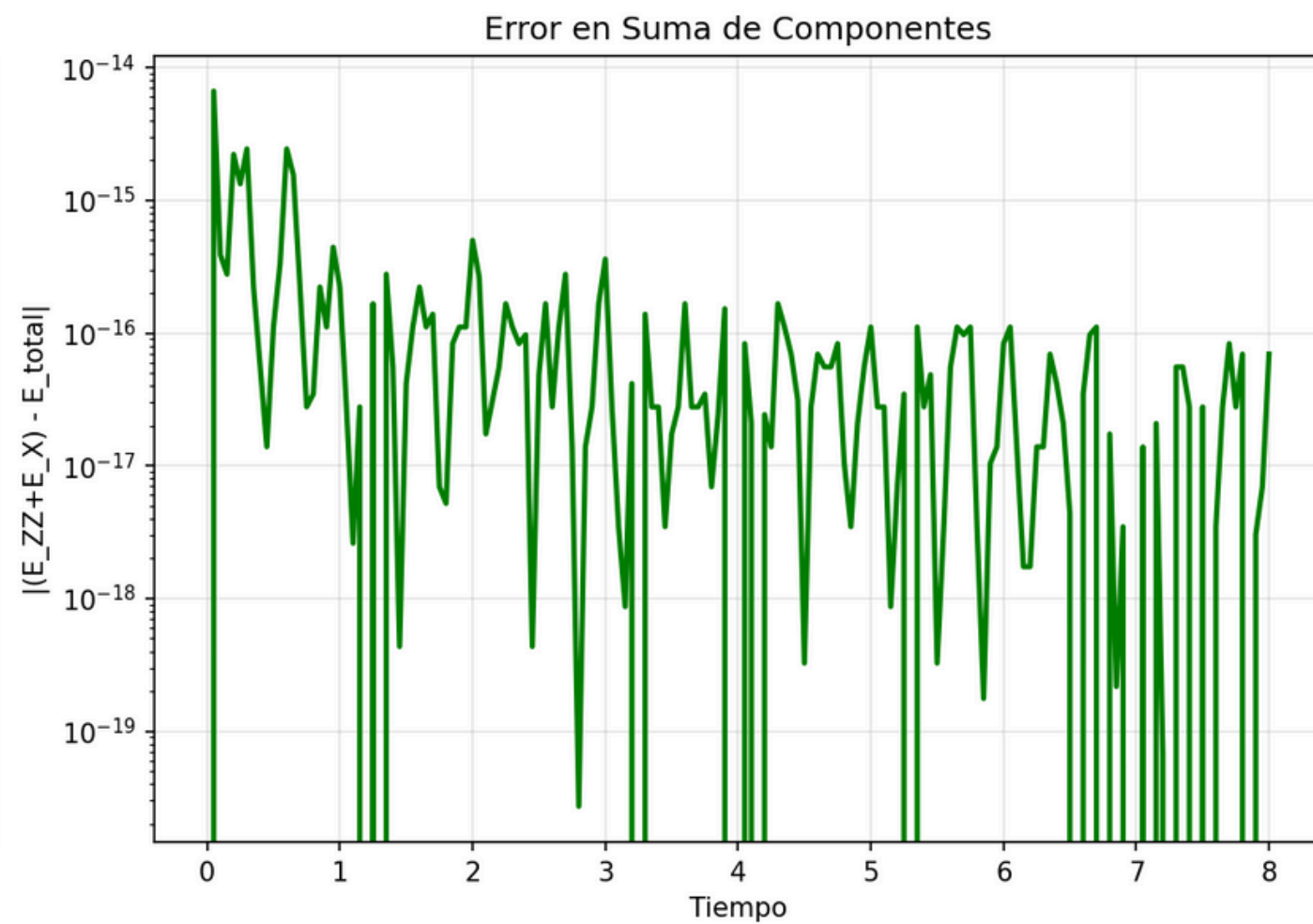
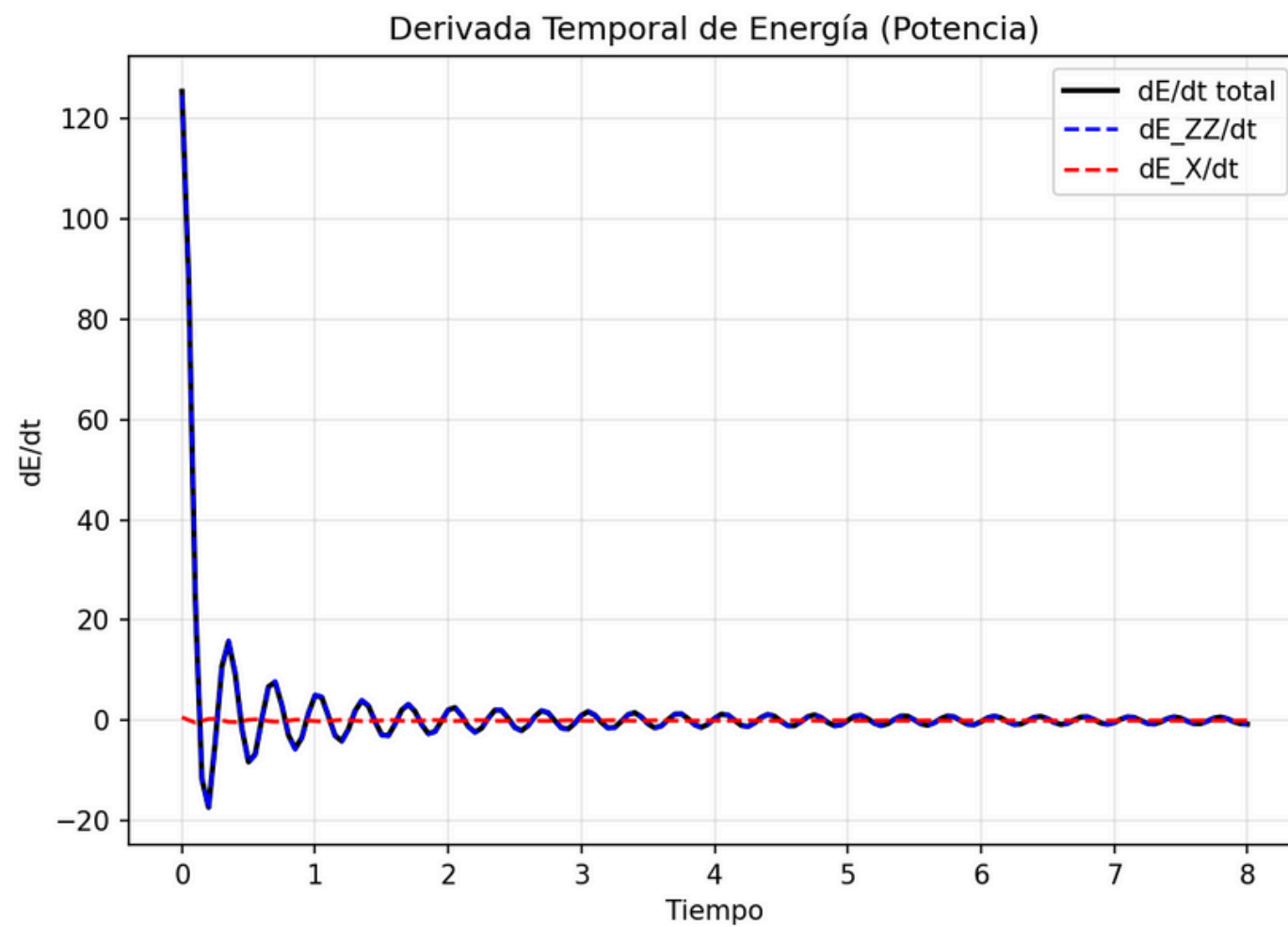
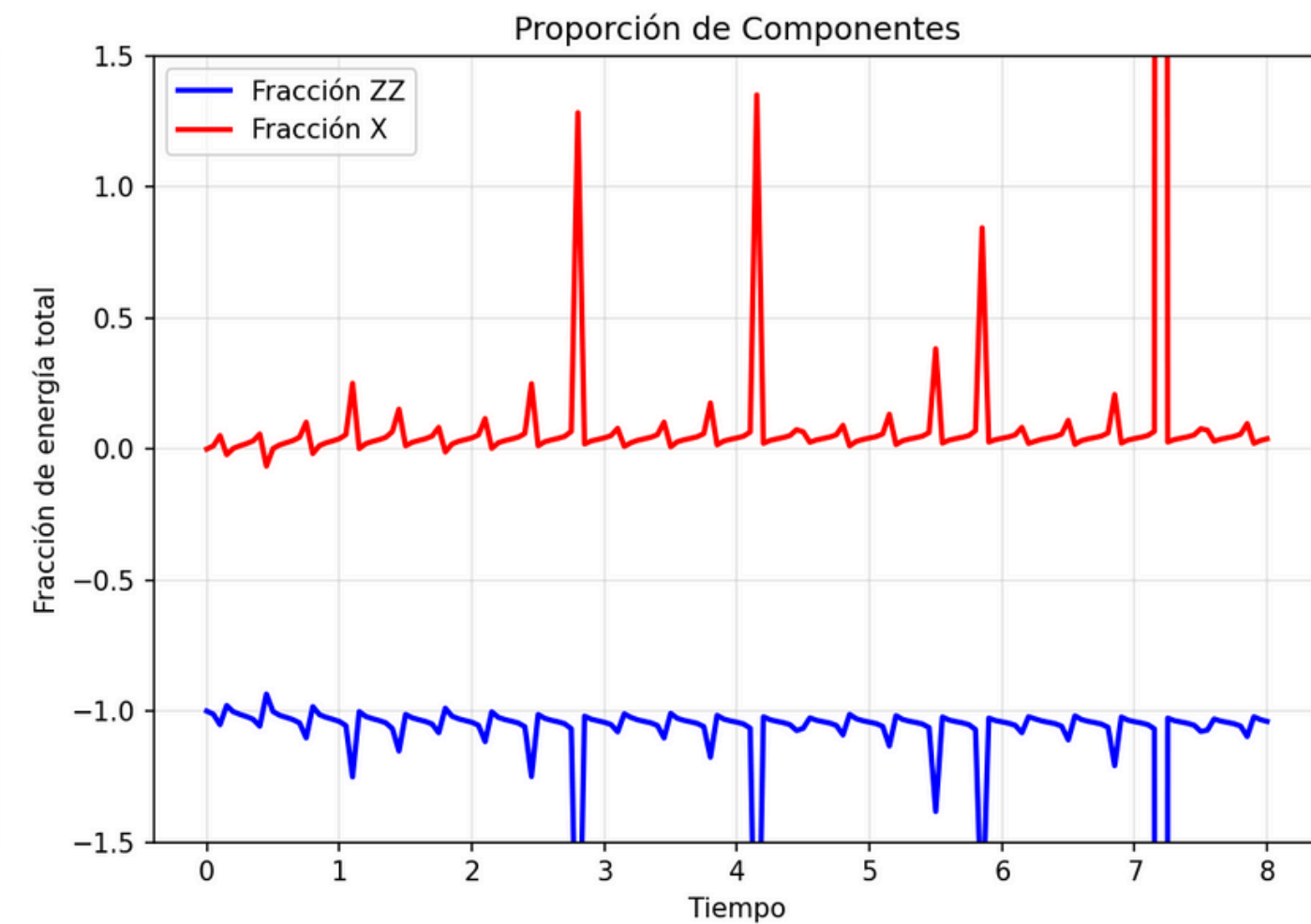
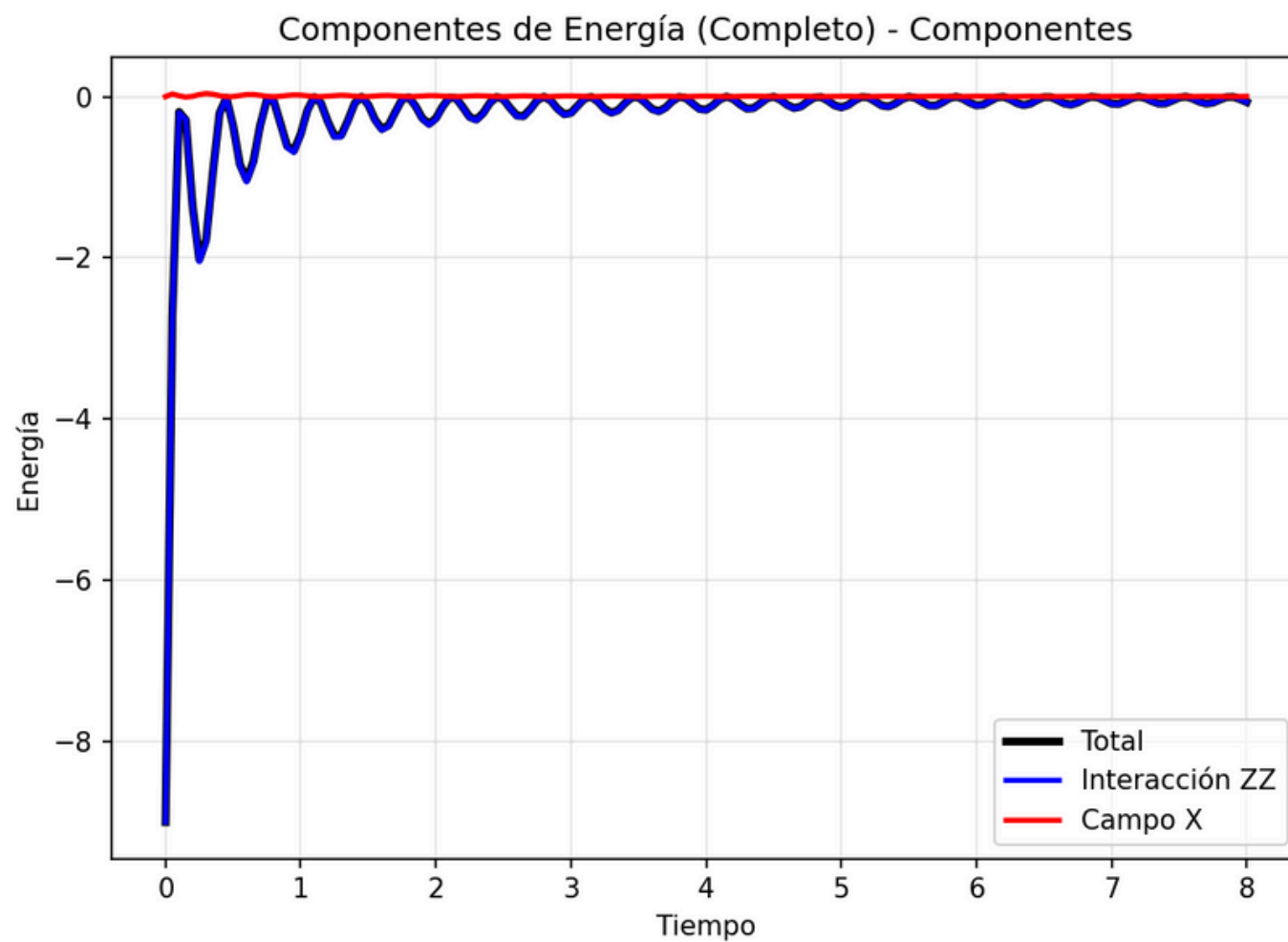
Energía vs Magnetización (Proyectado)

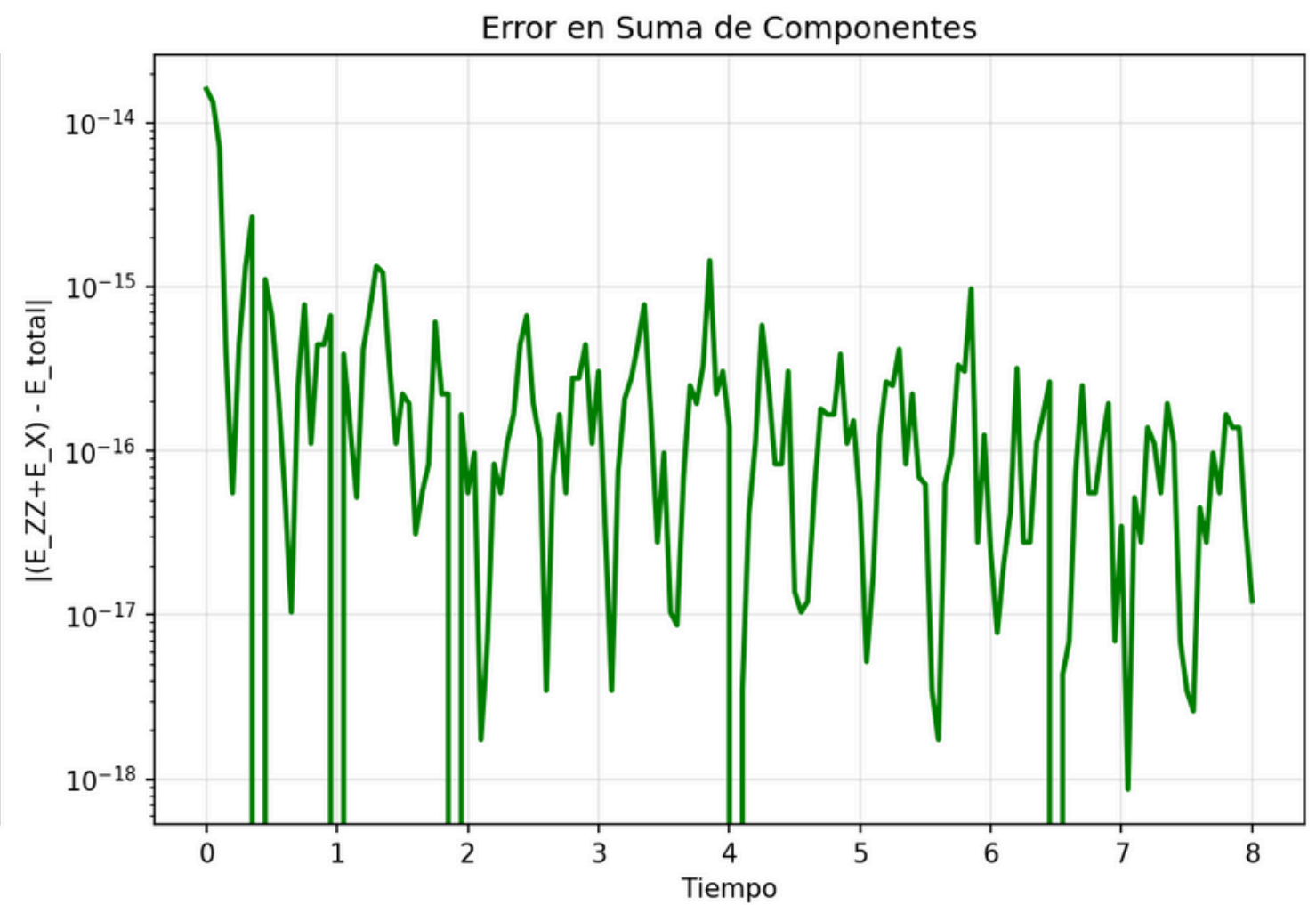
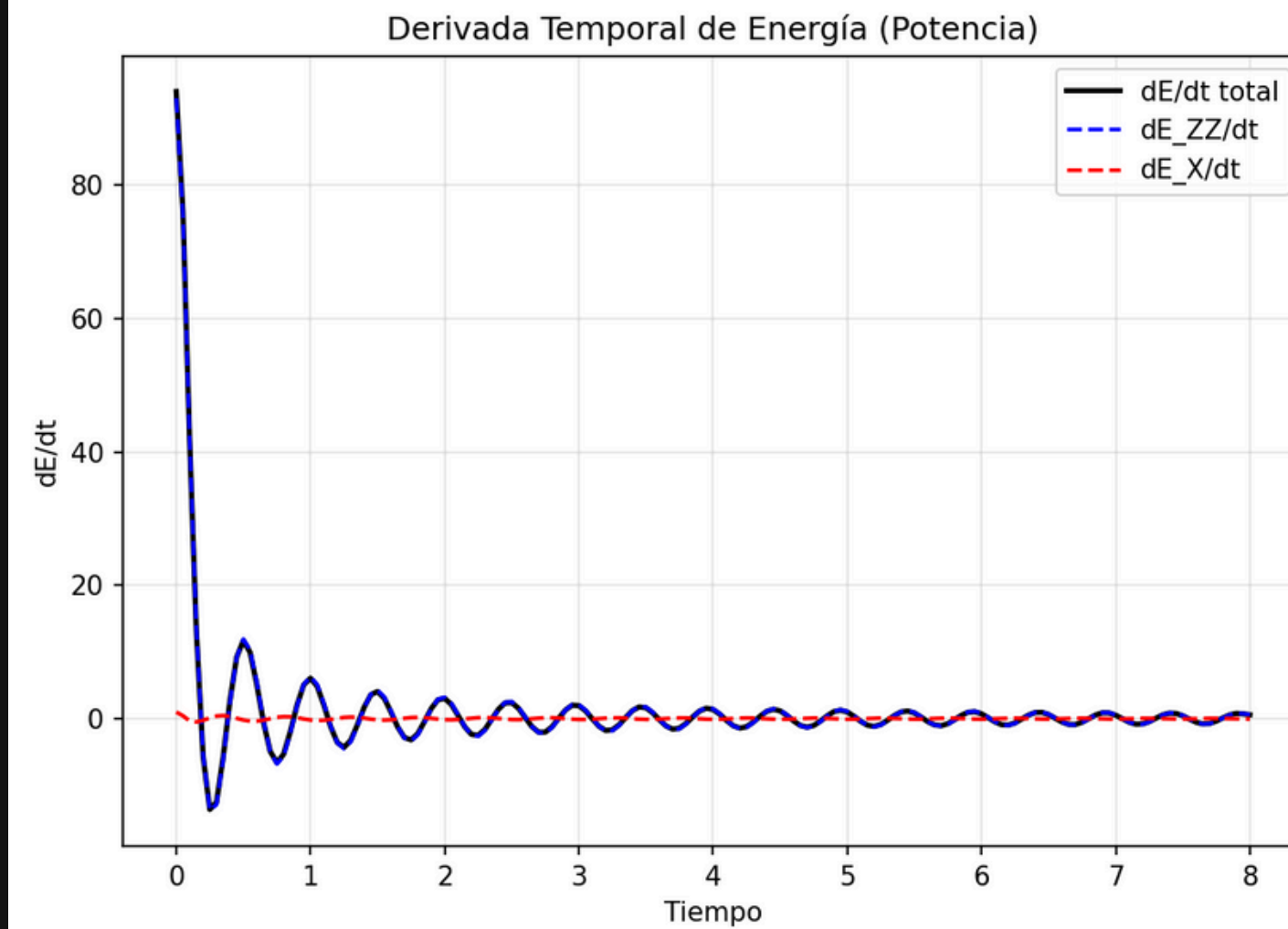
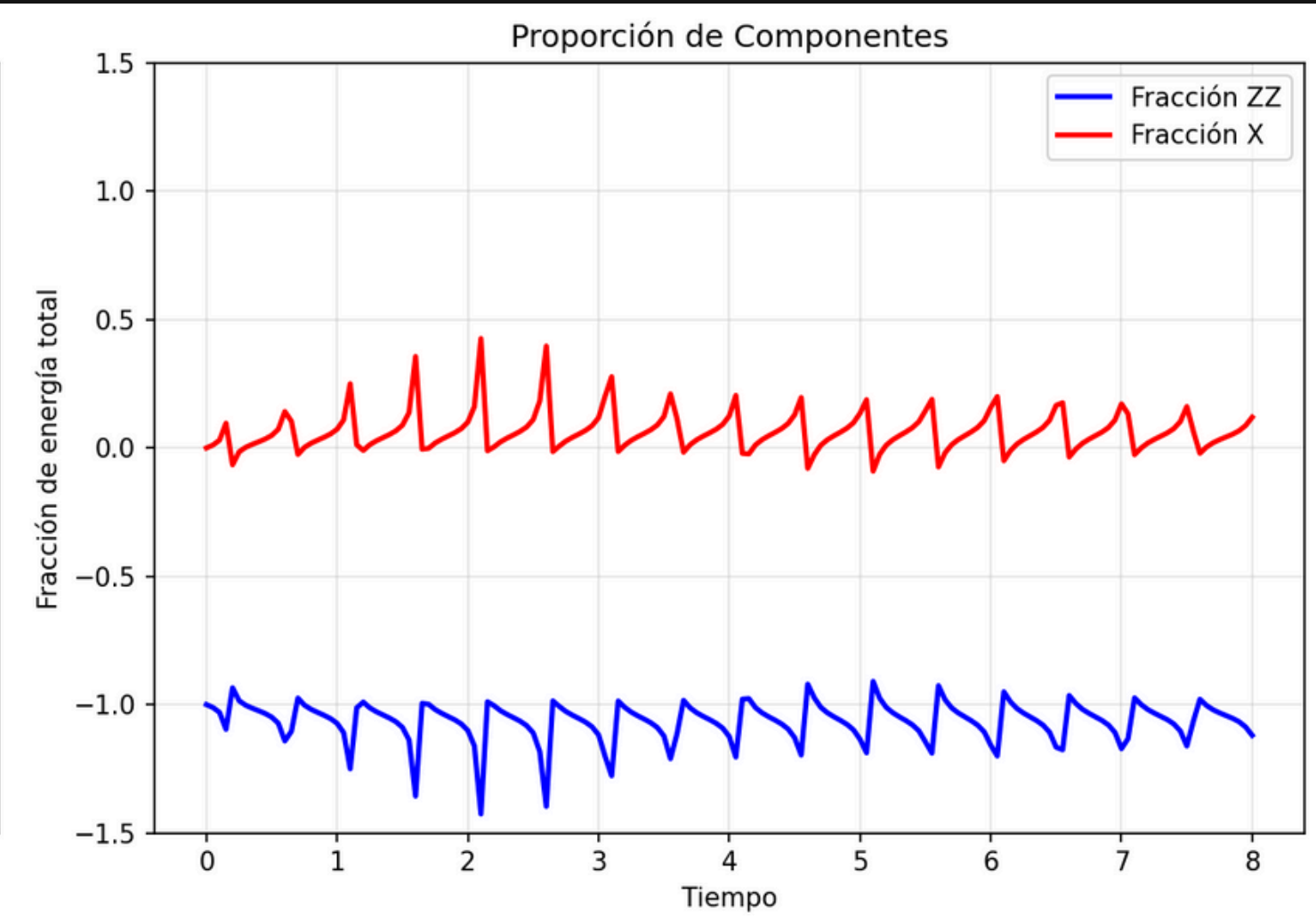
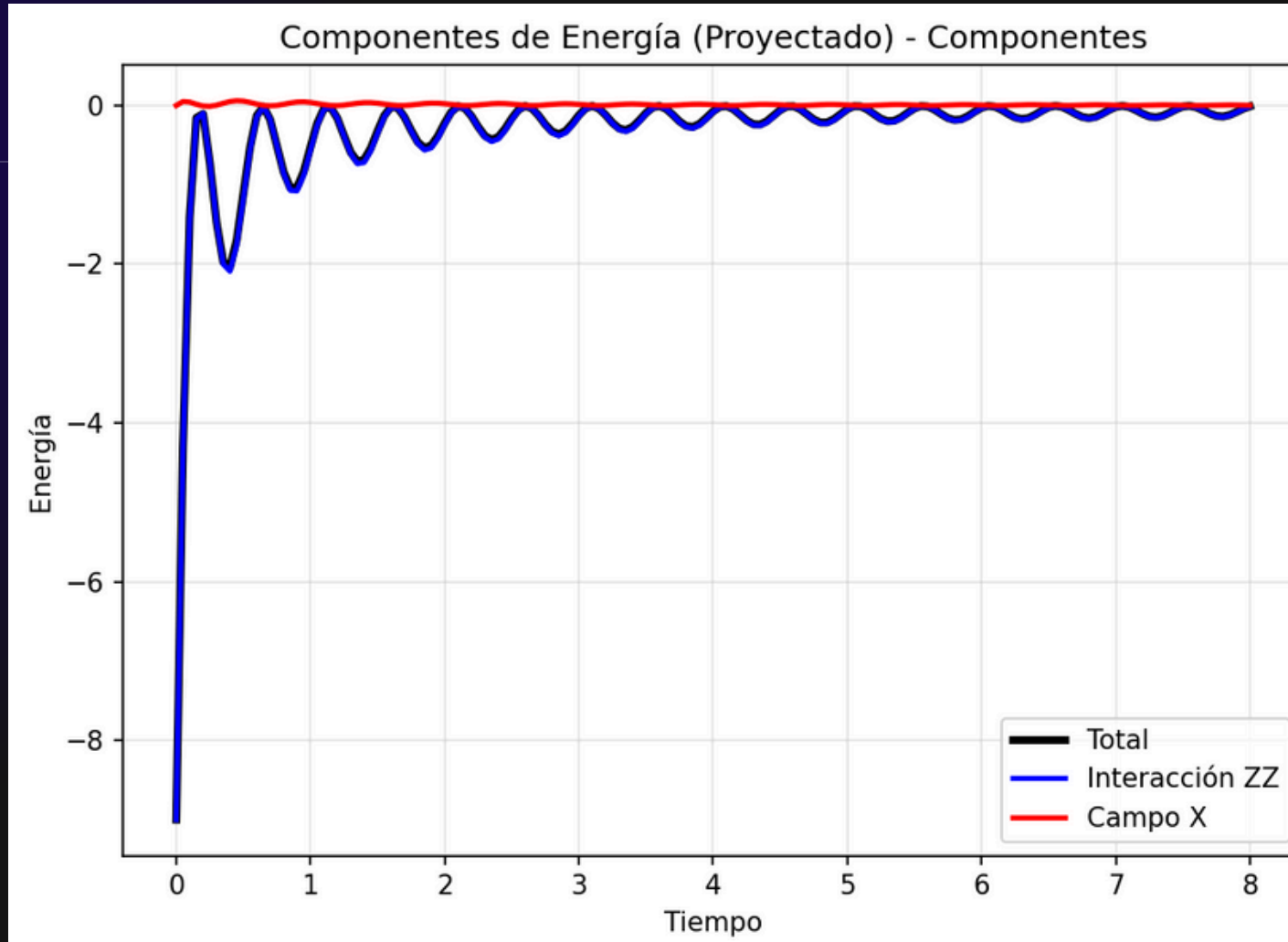


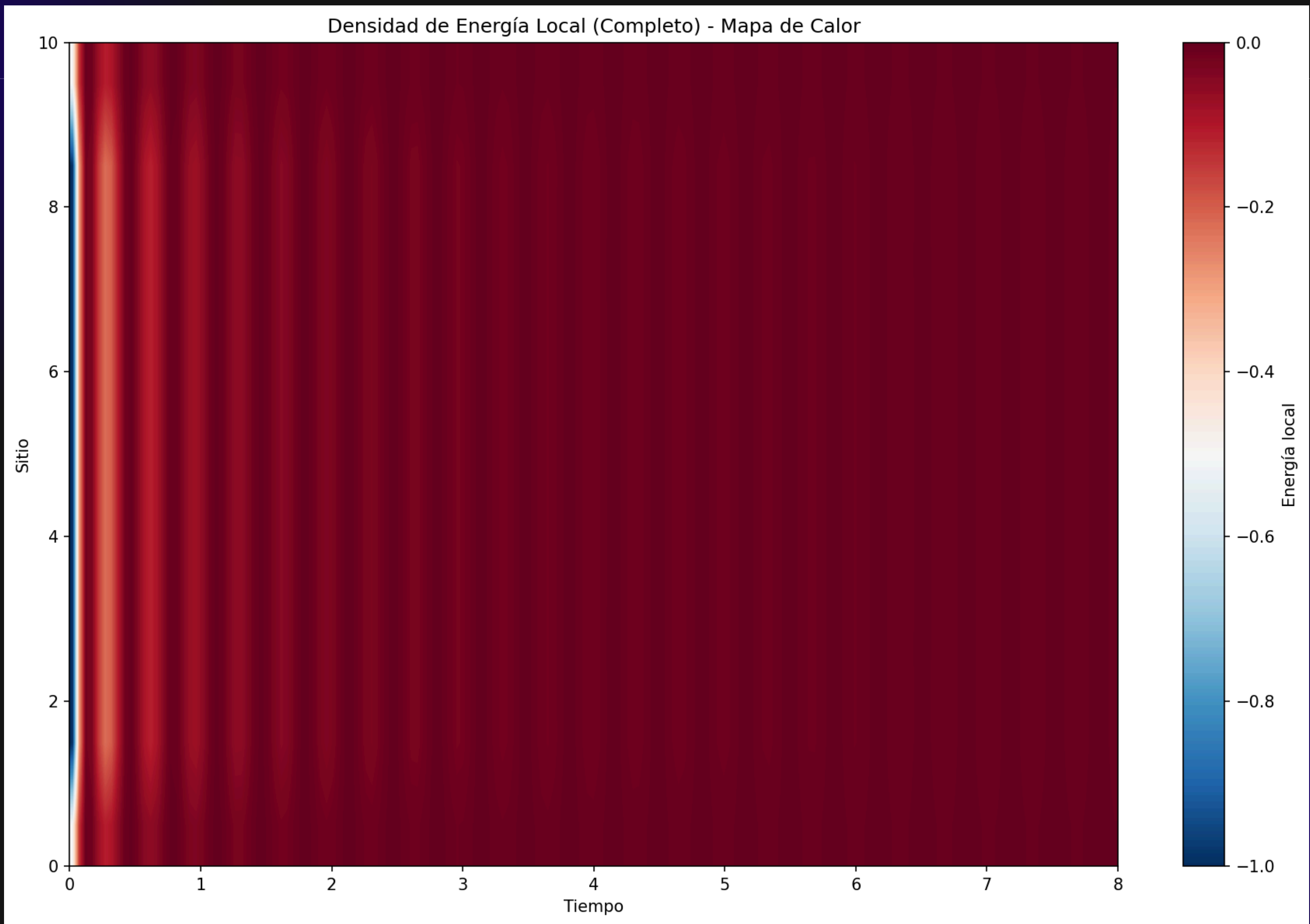
Energía por Sitio vs Tiempo

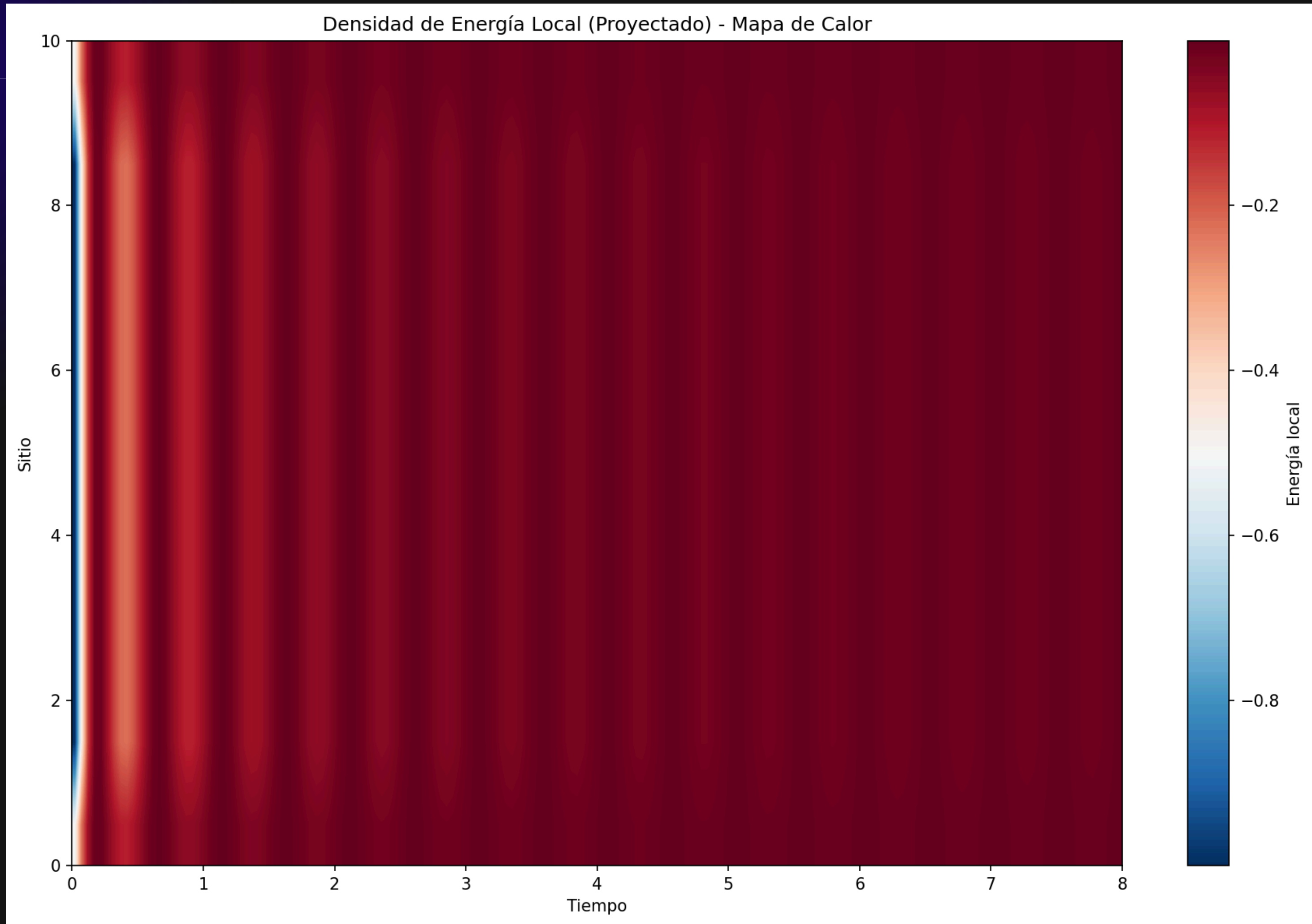




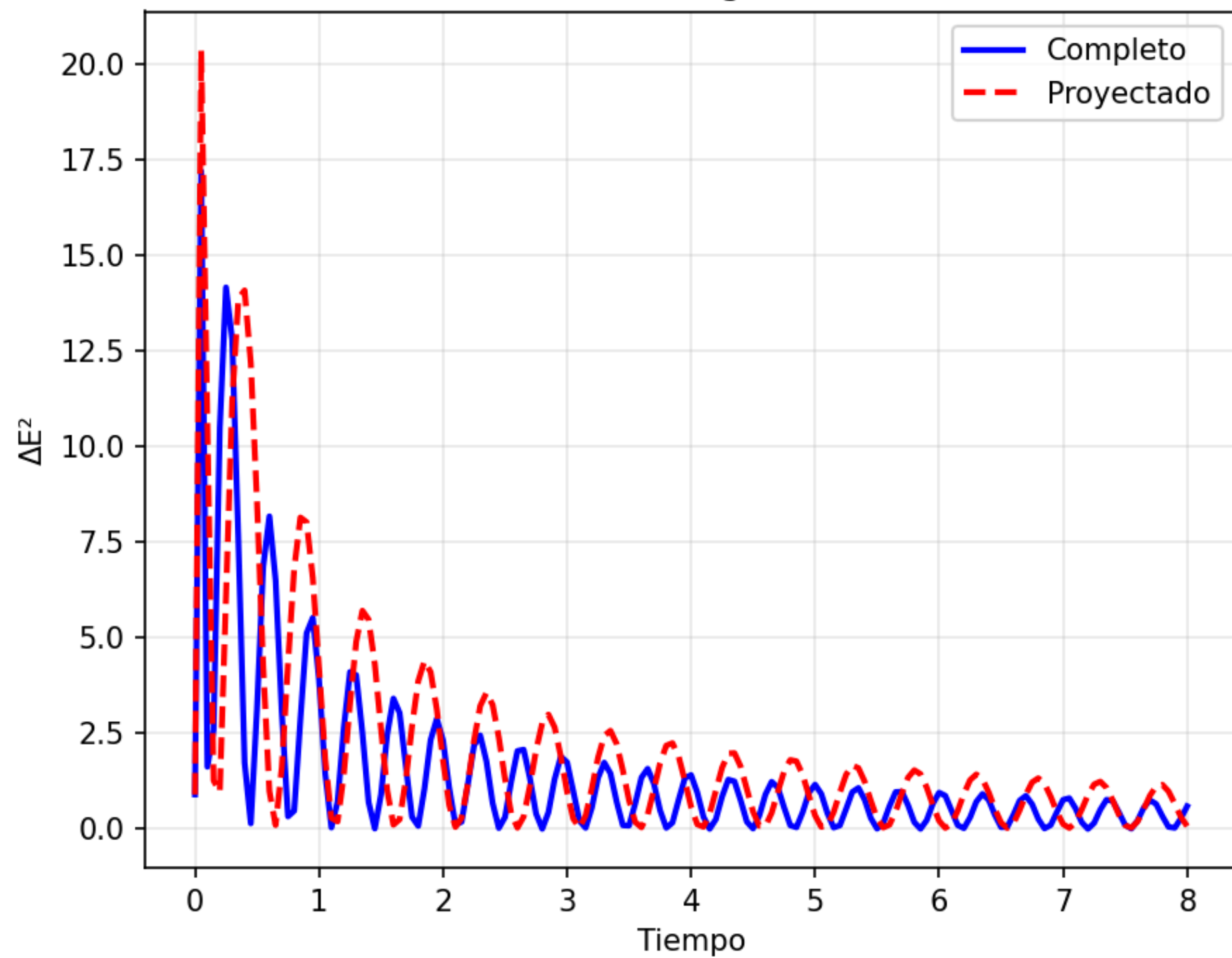




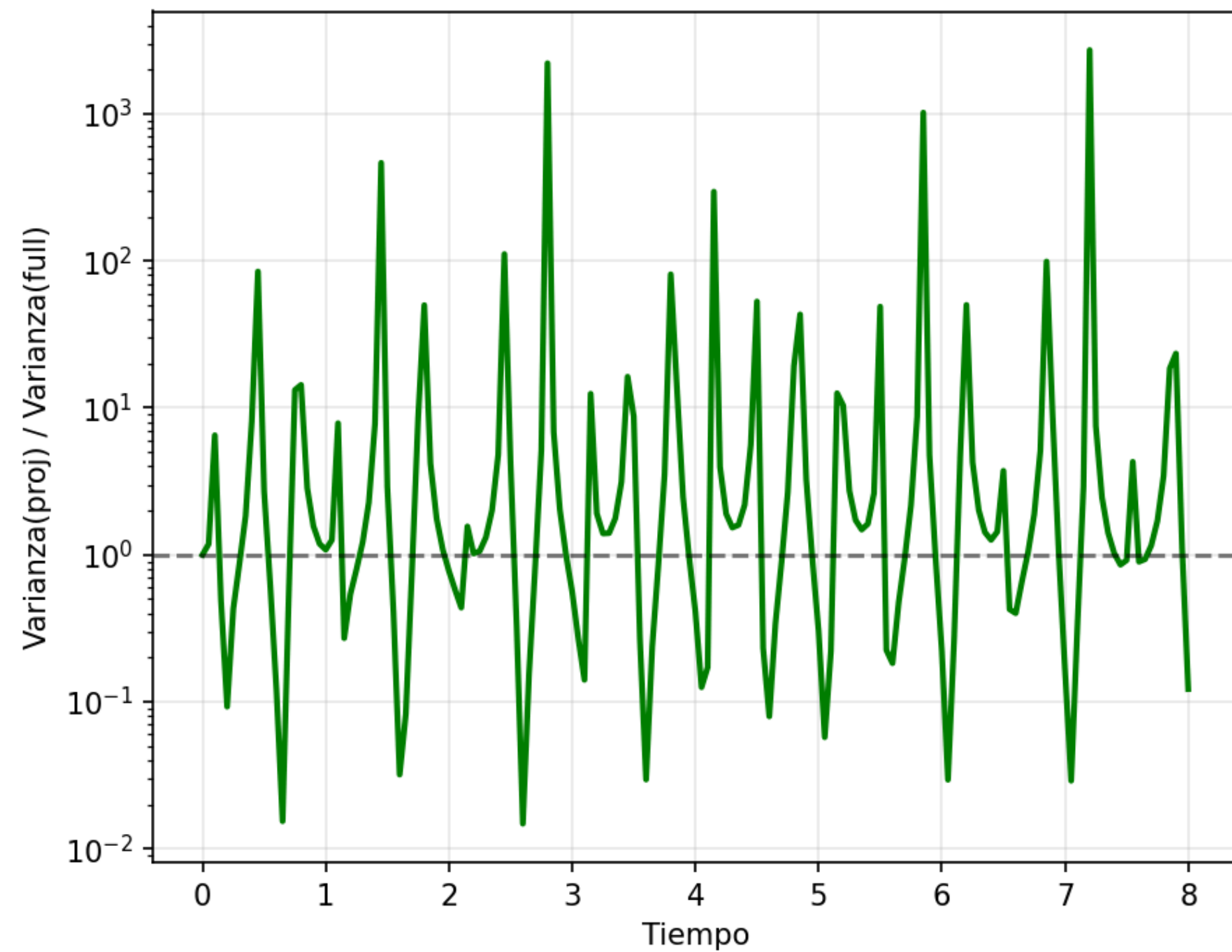




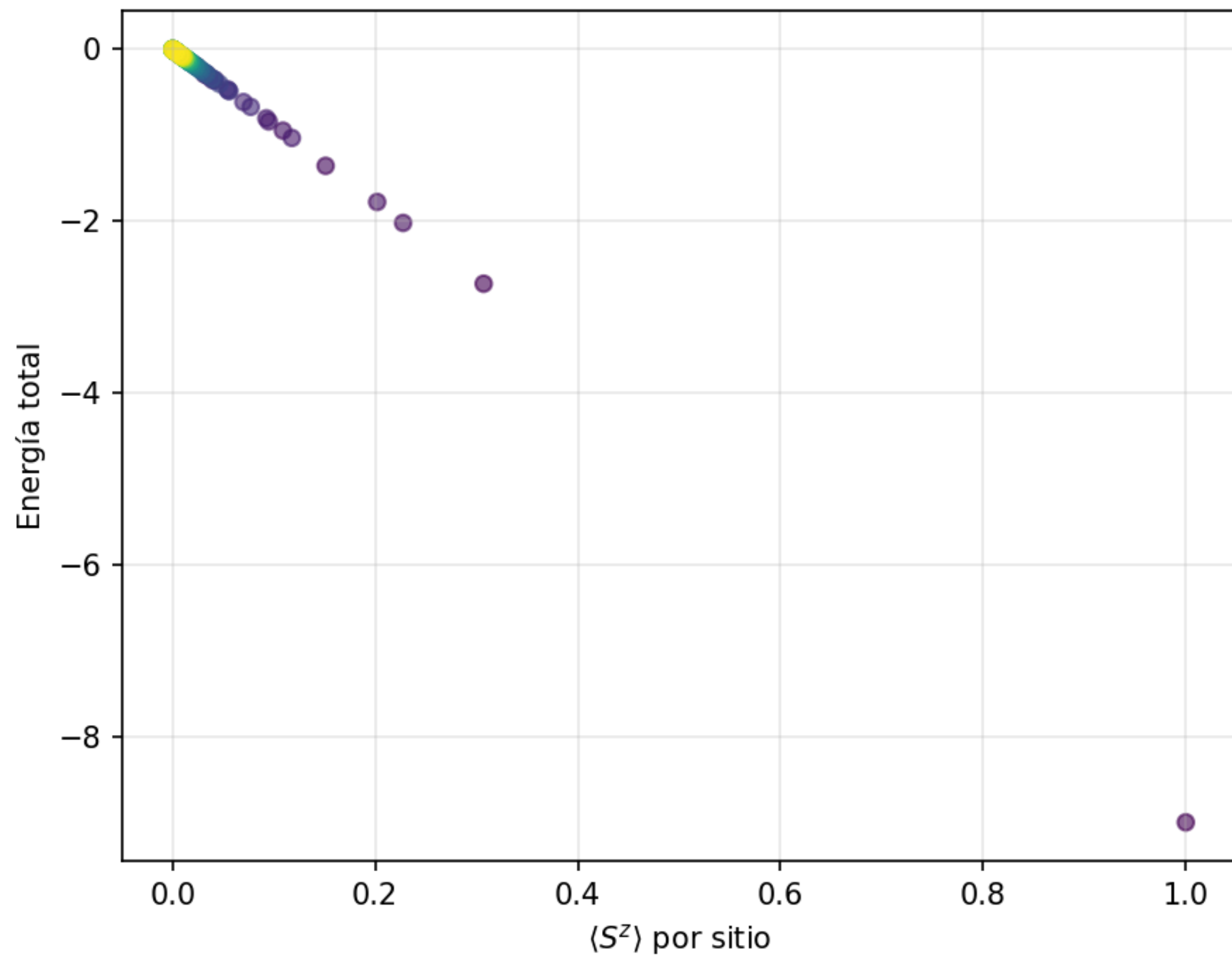
Varianza de Energía - Varianza



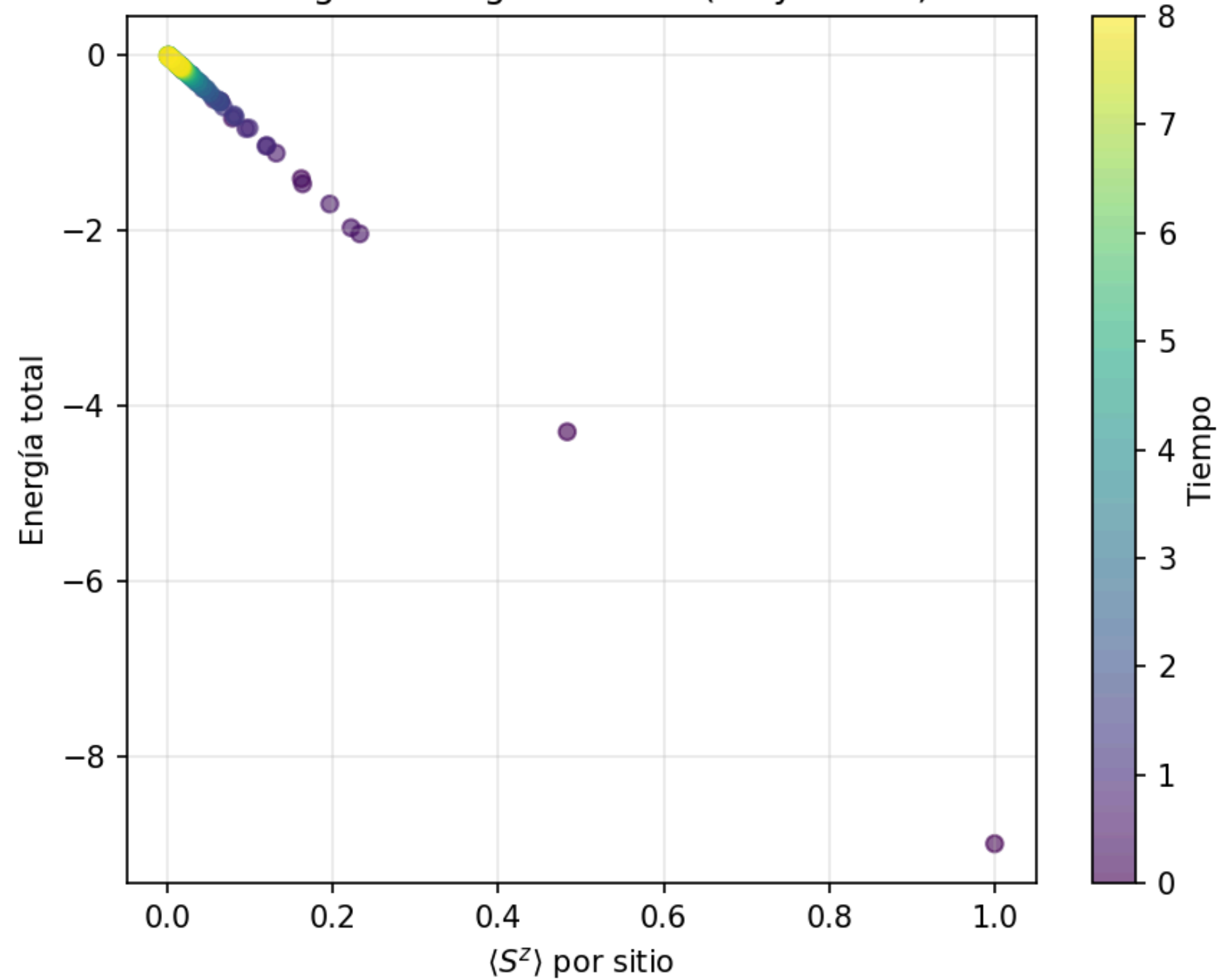
Ratio de Varianzas



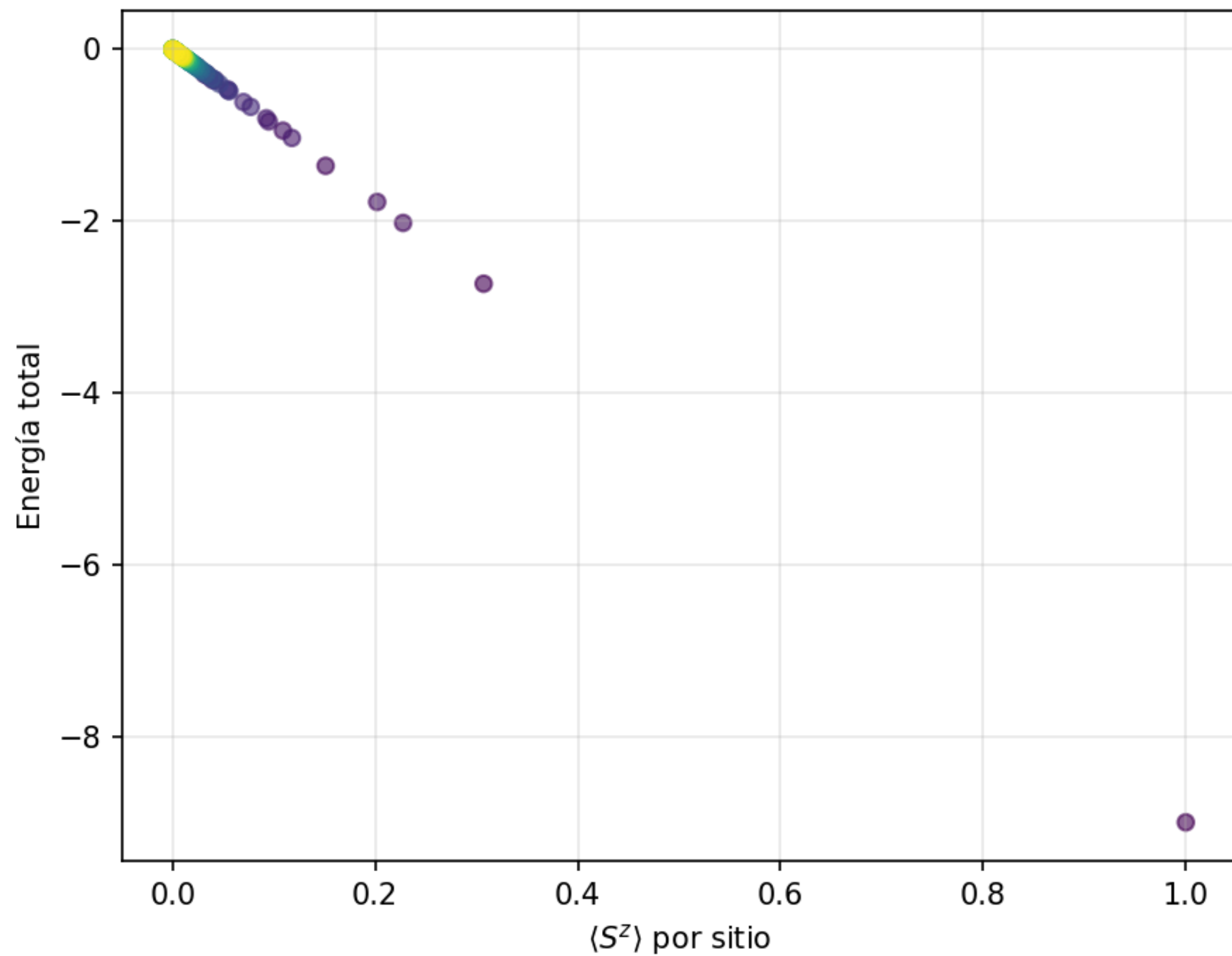
Energía vs Magnetización (Completo)



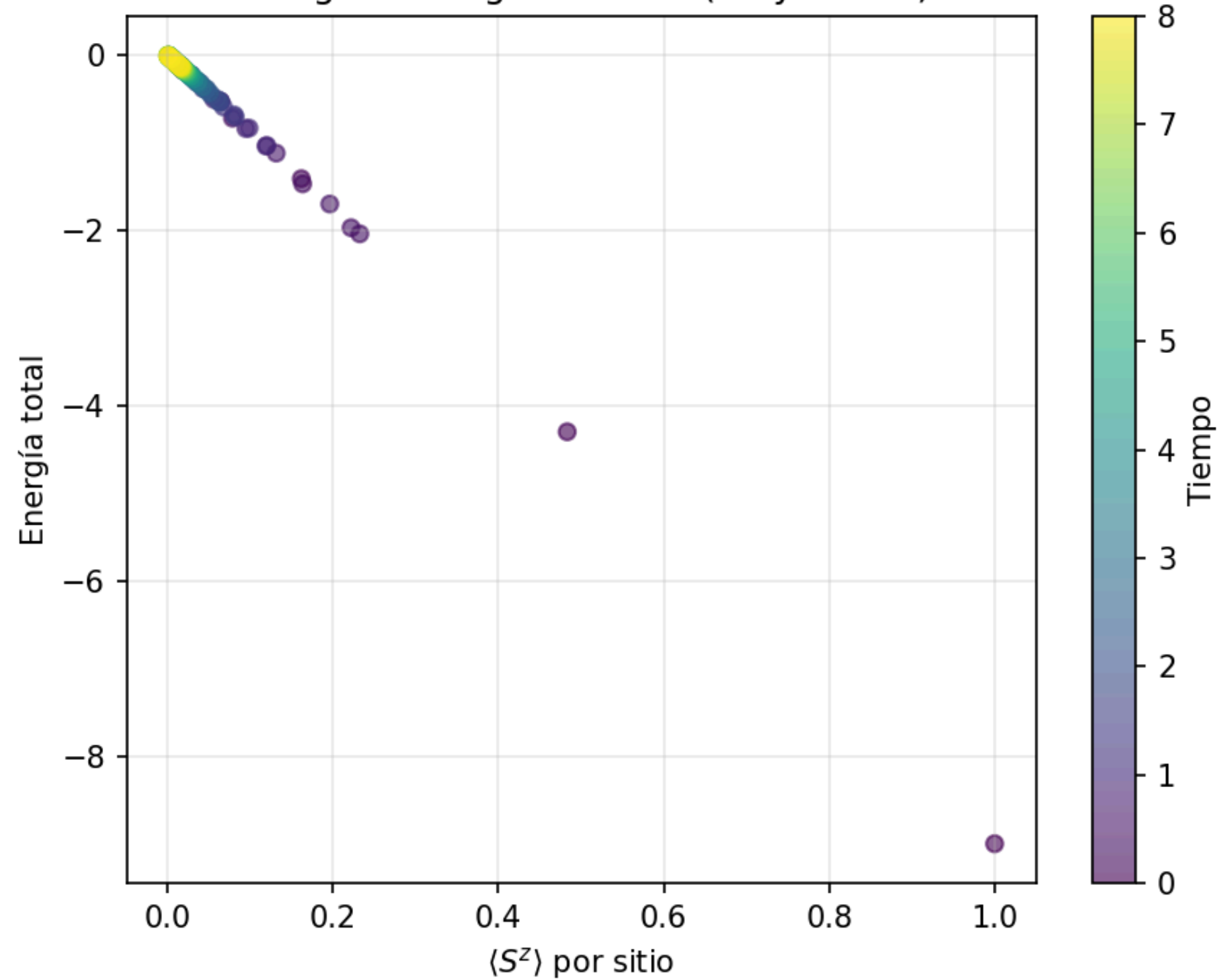
Energía vs Magnetización (Proyectado)



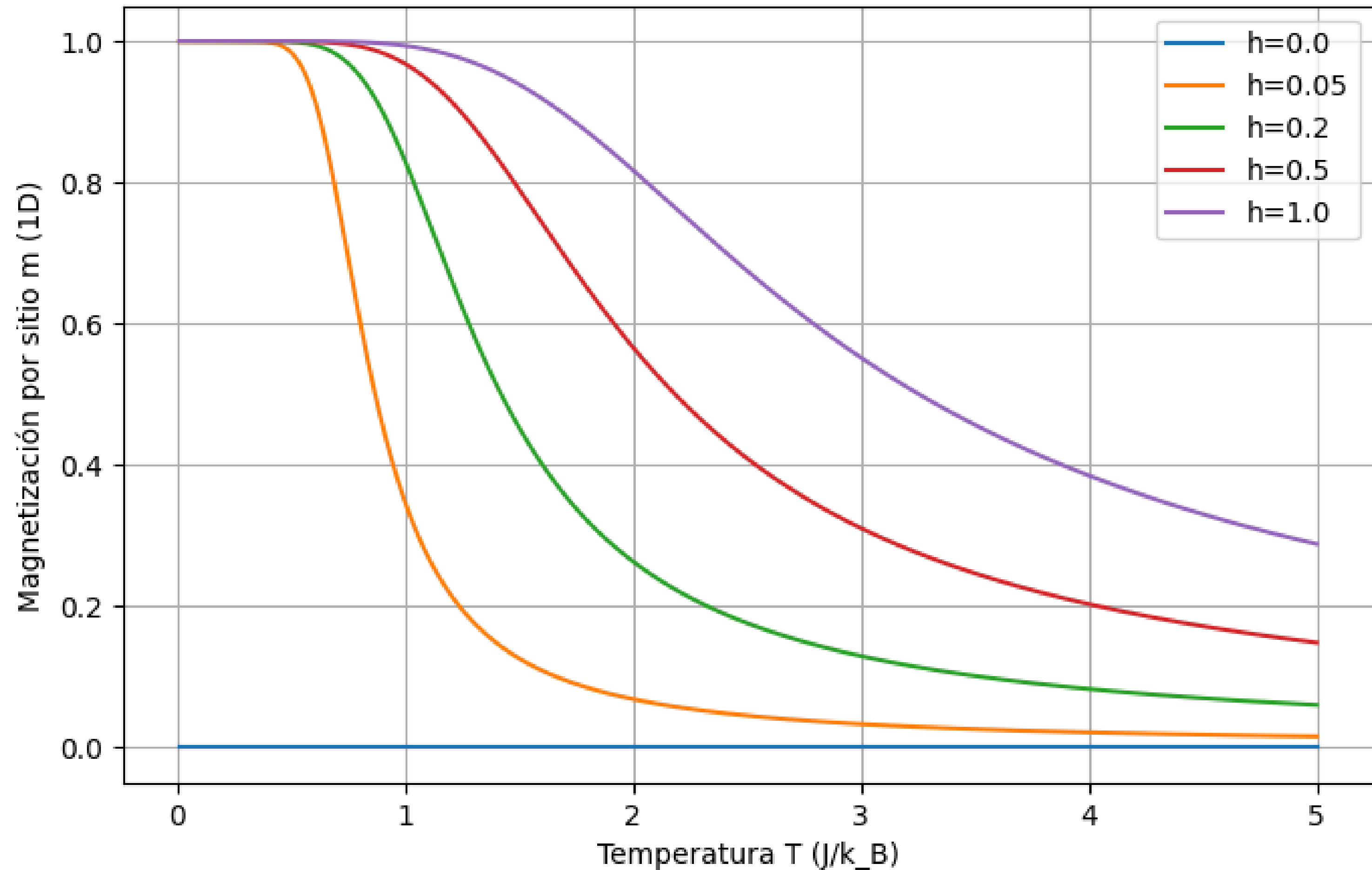
Energía vs Magnetización (Completo)



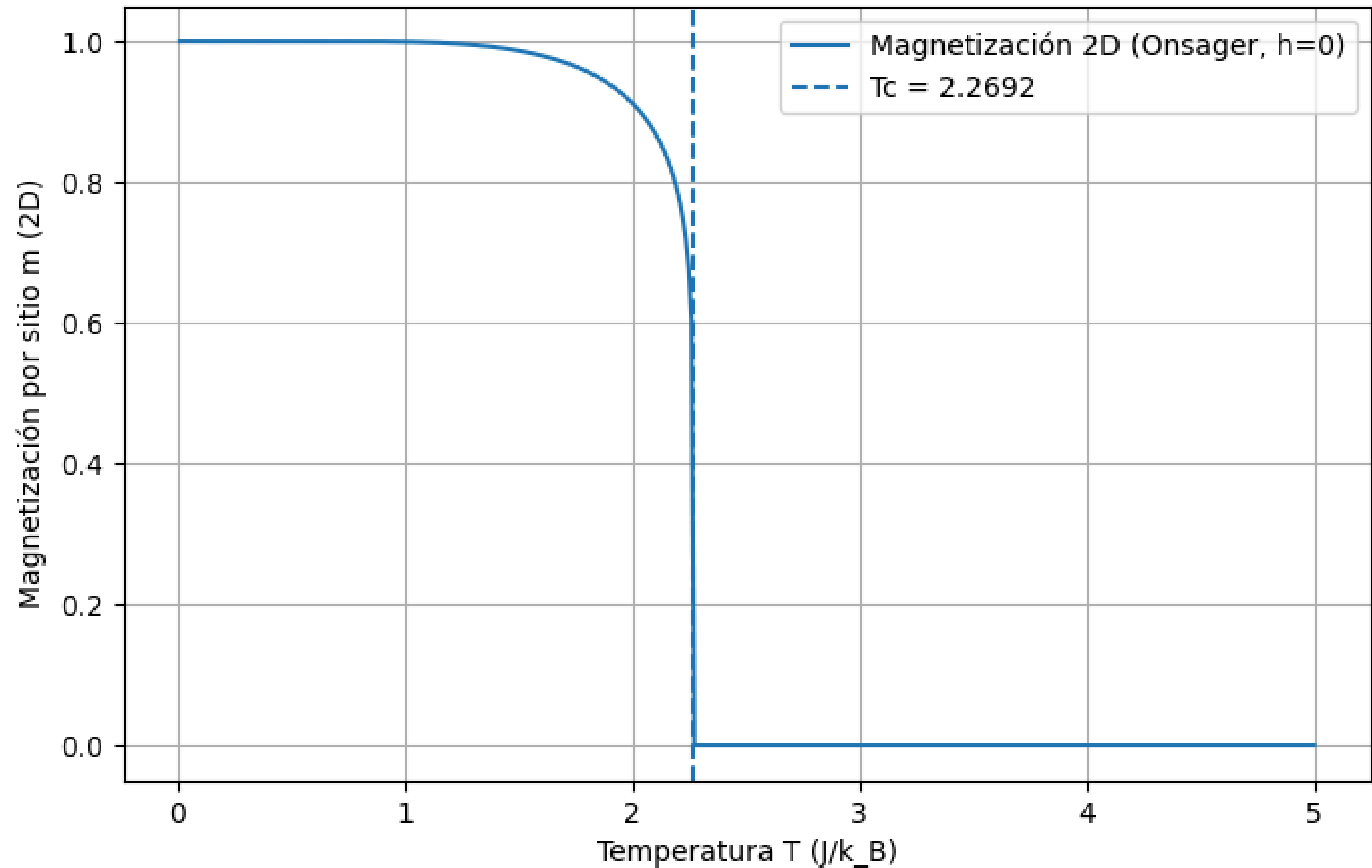
Energía vs Magnetización (Proyectado)



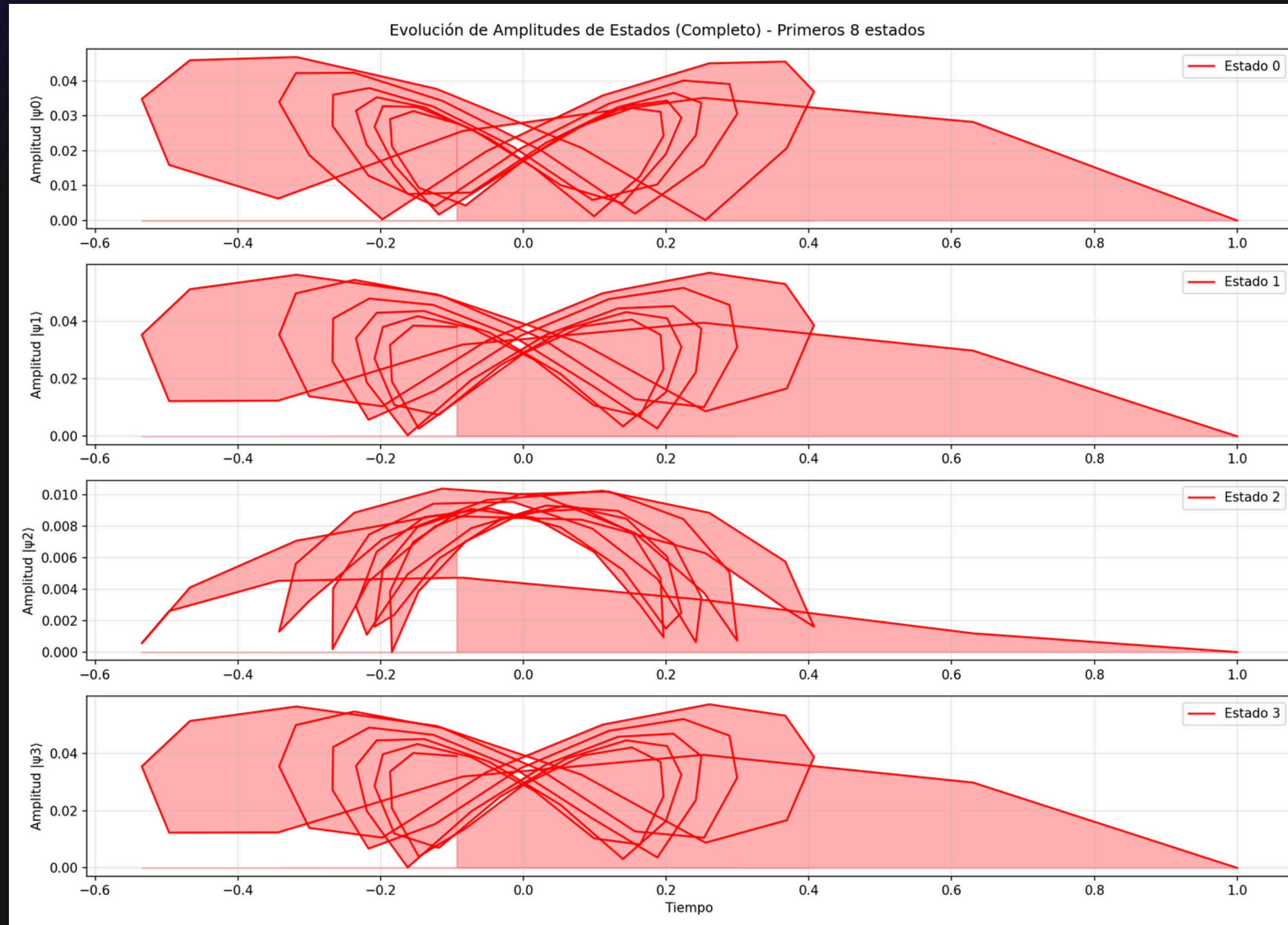
Ising 1D — magnetización exacta (transfer matrix)



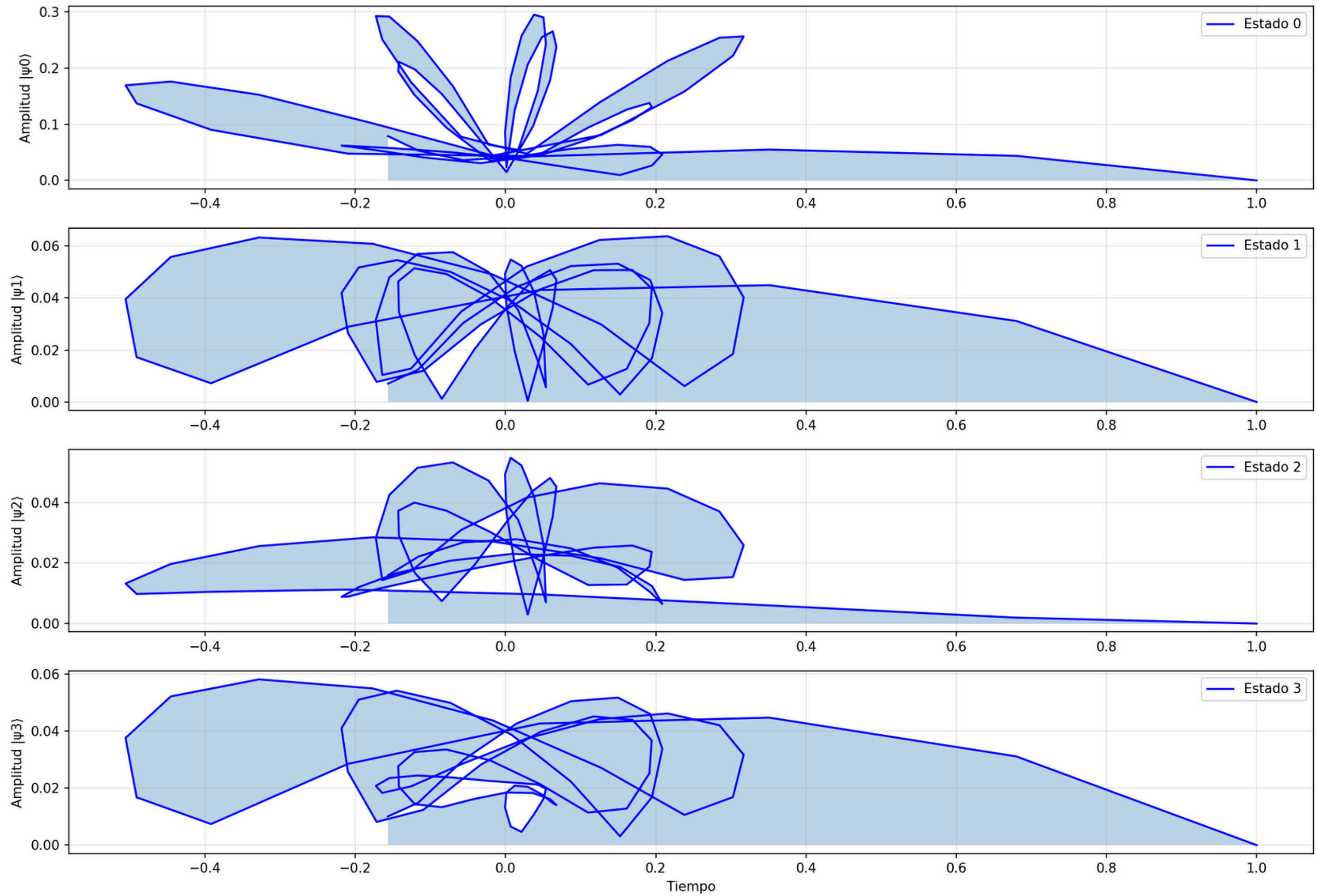
Ising 2D — Magnetización espontánea (Onsager)

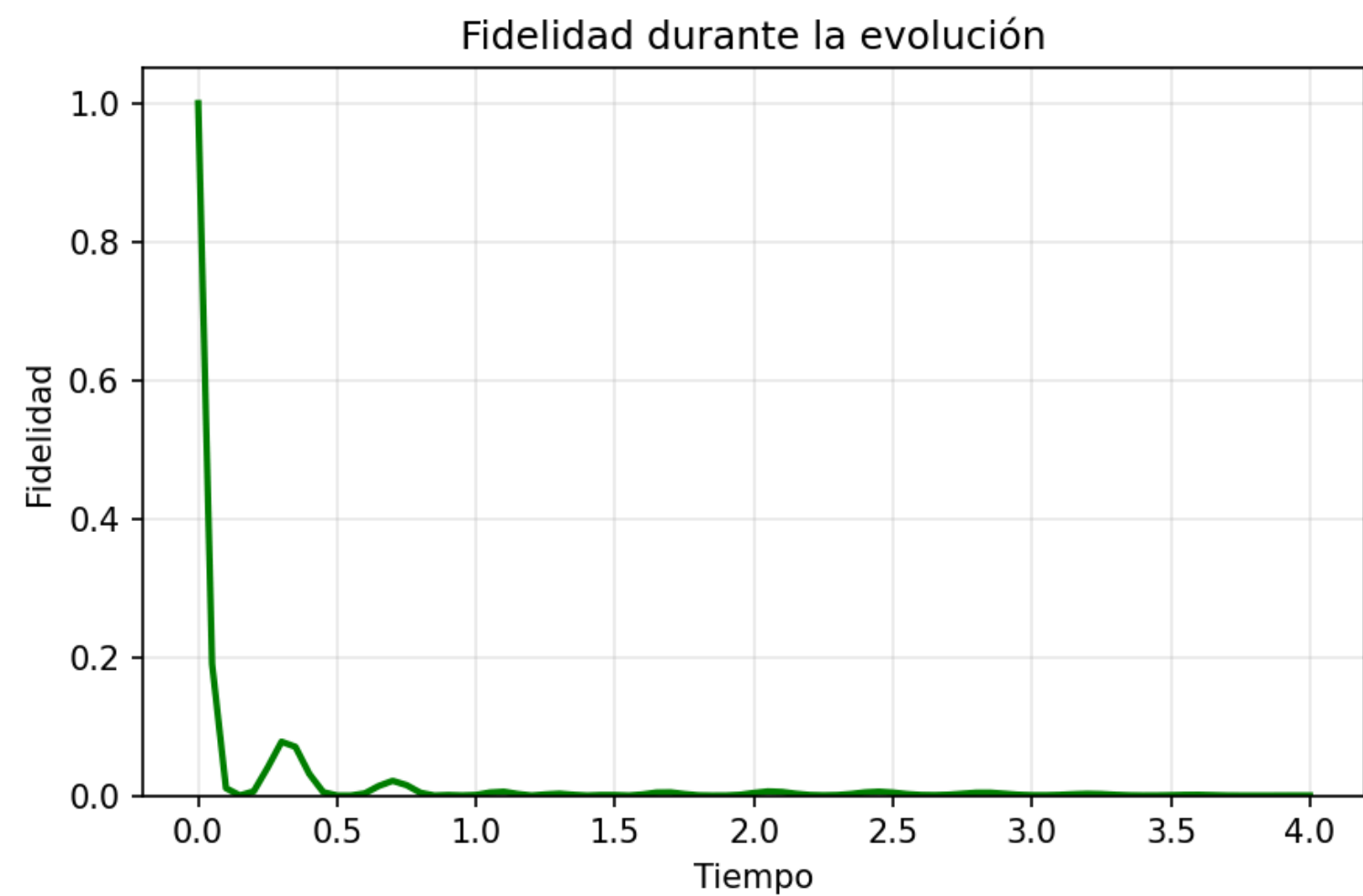
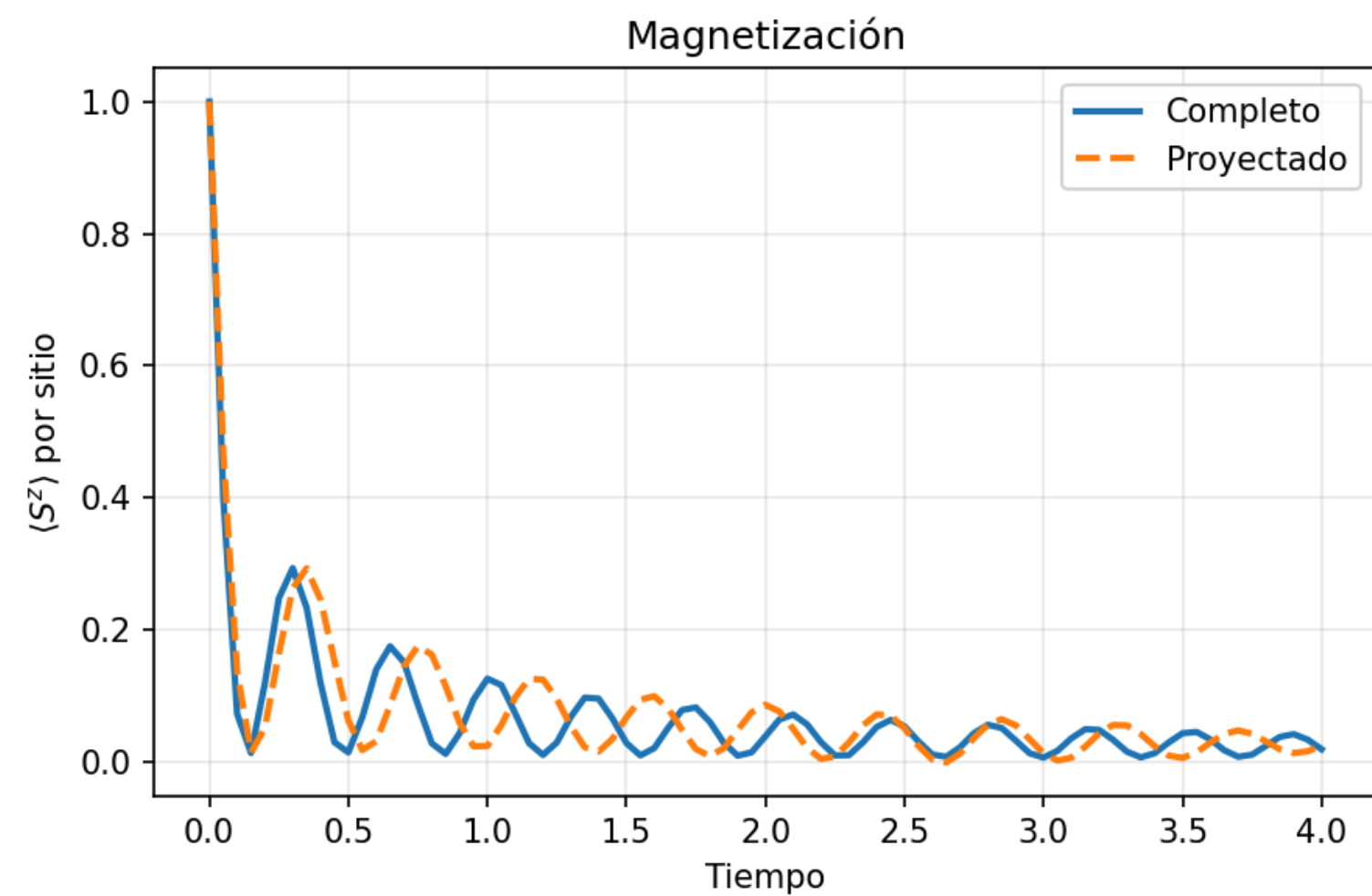
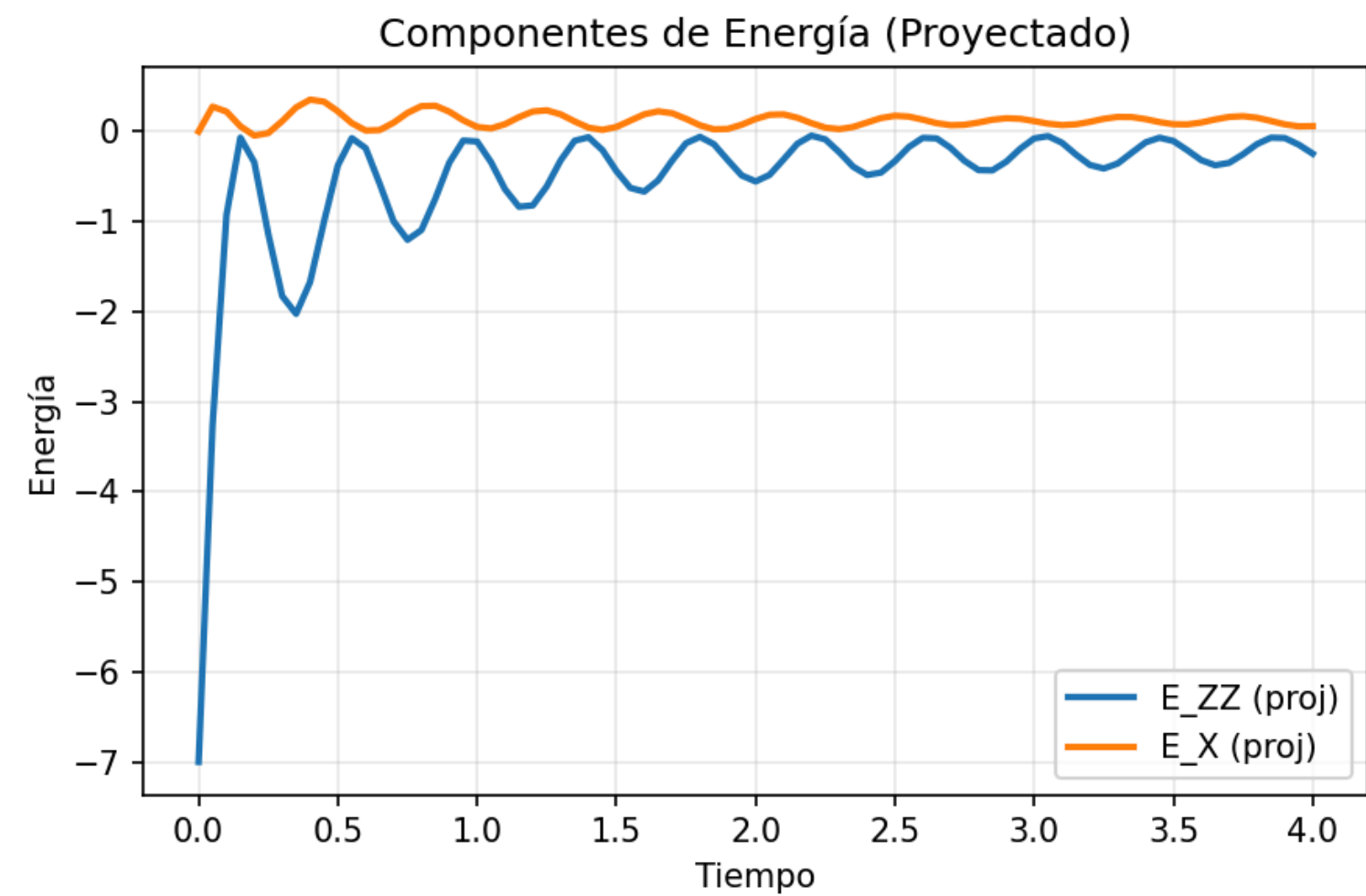
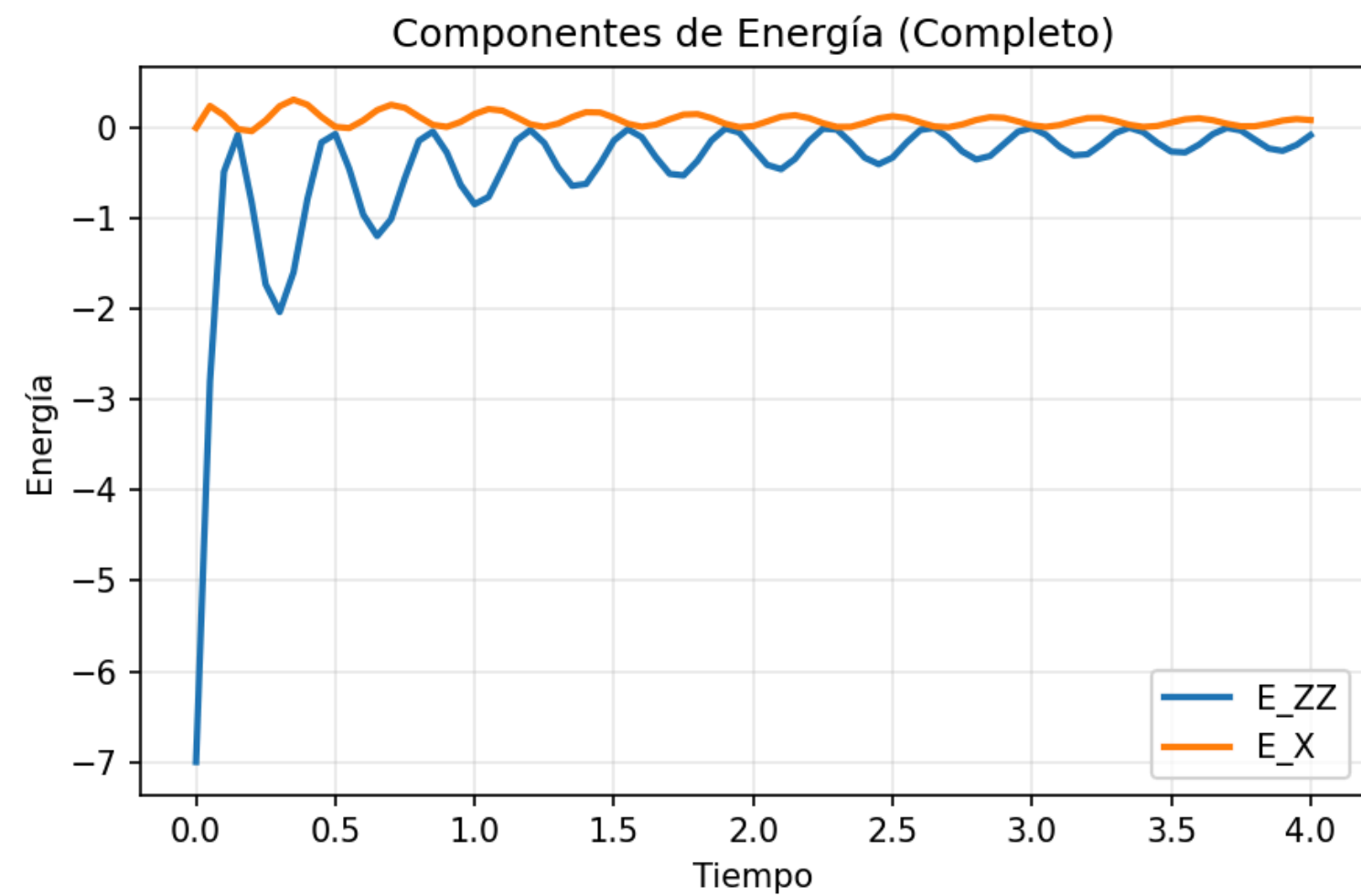


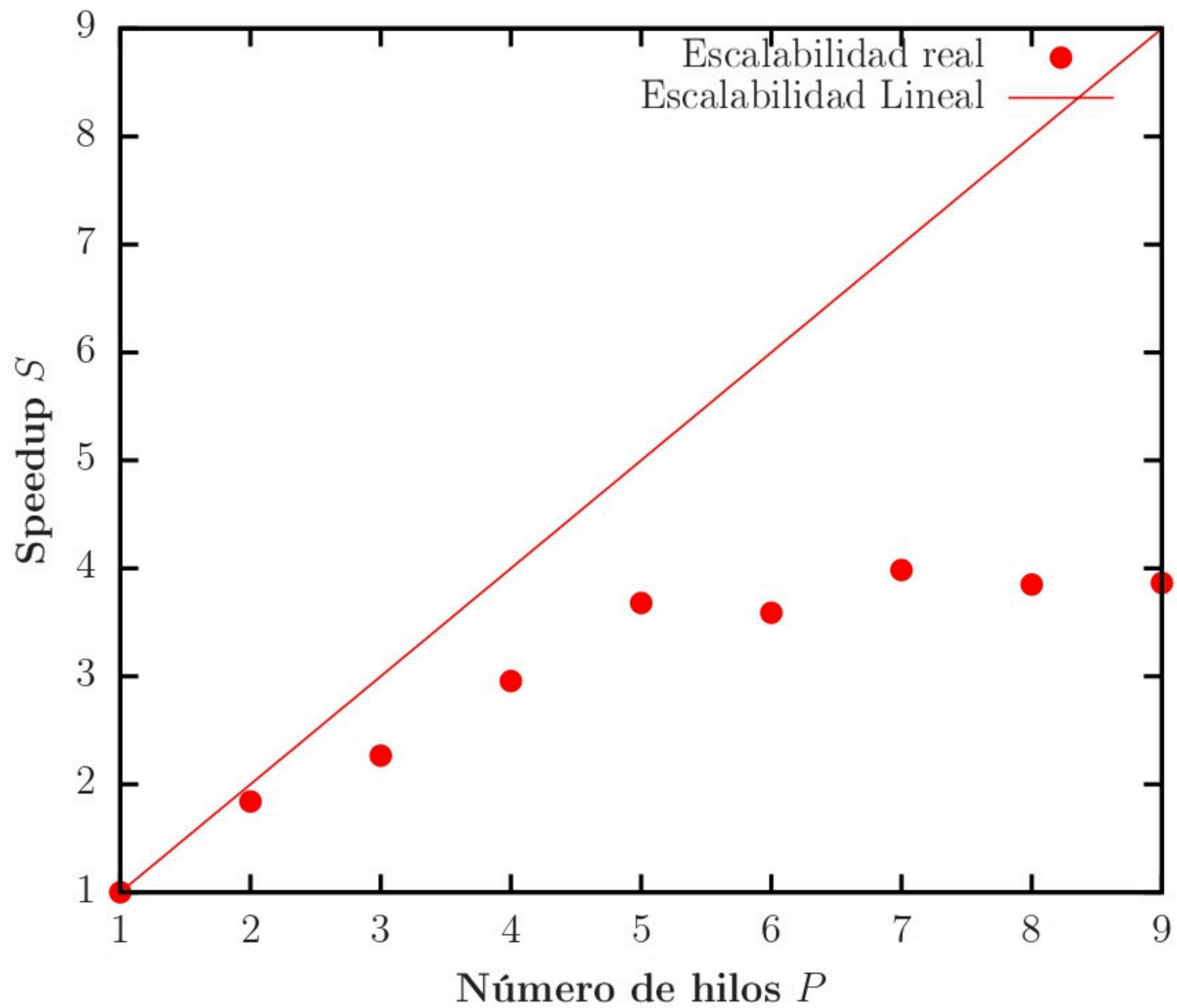
Resultados en C++



Evolución de Amplitudes de Estados (Proyectado)







Discusión

Qiang W, Hu G, Zhao P., 2008.

Transición de Fase en Redes Evolutivas de Mundo Local. Investigan la transición de fase del modelo de Ising en redes complejas evolutivas de "mundo local" (local-world evolving networks), donde la conectividad de los espines cambia dinámicamente. Analizan cómo la topología de la red afecta el comportamiento crítico y la temperatura de transición.

Dziarmaga J., 2005.

Dinámica de la Transición de Fase Cuántica (Solución Exacta). Proporciona una solución exacta para el modelo de Ising cuántico 1D cuando es impulsado (quenched) a través del punto crítico a una velocidad finita. El trabajo confirma el mecanismo de Kibble-Zurek, mostrando que la densidad de defectos (cuasipartículas) creados escala con la raíz cuadrada de la velocidad de transición.

Korniss G, White CJ, Rikvold PA, Novotny MA, 2000.

Transición de Fase Dinámica en Campo Oscilante. Investigan la transición de fase dinámica (DPT) en el modelo de Ising cinético bidimensional bajo la influencia de un campo magnético oscilante. Utilizando simulaciones de Monte Carlo y escalamiento de tamaño finito, demuestran que la DPT pertenece a la misma clase de universalidad que el modelo de Ising en equilibrio 2D.

Komura Y, Okabe Y., 2012.

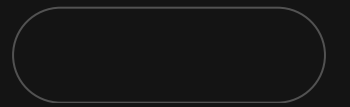
Algoritmo de Clúster basado en GPU para Simulación. Presentan una implementación altamente eficiente del algoritmo de Wolff (single-cluster) para simulaciones de Monte Carlo del modelo de Ising, optimizado para GPUs (CUDA). Logran aceleraciones significativas (varias veces más rápido que una CPU) en el cálculo de propiedades cerca del punto crítico para modelos 2D y 3D.

Yue W, Zhang T, Jing Z, et al., 2024.

Máquina Universal de Ising Escalable (Hardware). Describen una implementación de hardware de computación cuántica/analógica, denominada "Máquina de Ising universal", para resolver problemas de optimización combinatoria. Su innovación clave es un diseño de almacenamiento centrado en la interacción y computación en memoria (compute-in-memory) que permite soportar cualquier topología de gráfico Ising con gran eficiencia energética y de velocidad.

Wang Z, Feng L, Zhang W, Ding C., 2021.

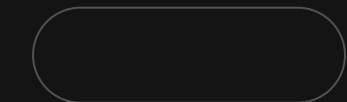
Transiciones de Fase en Ising 3D con Peso de Clúster. Proponen y simulan un modelo de Ising modificado con un "peso de clúster" en una red cúbica tridimensional. Mediante simulaciones de Monte Carlo, concluyen que este modelo modificado pertenece a una clase de universalidad diferente a la del modelo de Ising 3D tradicional.



Conclusiones

Referencias

1. Breuckmann NP, Placke B, Roy A. Critical properties of the Ising model in hyperbolic space. arXiv. 2019. <https://doi.org/10.48550/ARXIV.1909.12107>
2. Dziarmaga J. Dynamics of quantum phase transition: exact solution in quantum Ising model. arXiv. 2005. <https://doi.org/10.48550/ARXIV.COND-MAT/0509490>
3. Korniss G, White CJ, Rikvold PA, Novotny MA. Dynamic phase transition, universality, and finite-size scaling in the two-dimensional kinetic Ising model in an oscillating field. Physical Review E. 2000;63(1). <https://doi.org/10.1103/physreve.63.016120>
4. Komura Y, Okabe Y. GPU-based single-cluster algorithm for the simulation of the Ising model. Journal of Computational Physics. 2012;231(4):1209–1215. <https://doi.org/10.1016/j.jcp.2011.09.029>
5. Marra R, Miracle-Sole S. On the statistical mechanics of the gauge invariant Ising model. Communications in Mathematical Physics. 1979;67(3):233–240. <https://doi.org/10.1007/bf01238846>
6. Onsager L. Crystal Statistics. I. A Two-Dimensional Model with an Order-Disorder Transition. Physical Review. 1944;65(3–4):117–149. <https://doi.org/10.1103/physrev.65.117>
7. Pochinok AS, Molochkov AV, Chernodub MN. Ising Model on the Fibonacci Sphere (Version 1). arXiv. 2023. <https://doi.org/10.48550/ARXIV.2301.06849>
8. Qiang W, Hu G, Zhao P. Phase transition in the Ising model on local-world evolving networks. International Journal of Modern Physics C. 2008;19(11):1717–1726. <https://doi.org/10.1142/s0129183108013242>
9. Shekaari A, Jafari M. Theory and Simulation of the Ising Model (Version 1). arXiv. 2021. <https://doi.org/10.48550/ARXIV.2105.00841>
10. Swendsen RH. Phase Transitions and the Ising Model. In: An Introduction to Statistical Mechanics and Thermodynamics. Oxford University Press; 2019. p. 423–446. <https://doi.org/10.1093/oso/9780198853237.003.0031>
11. Wang Z, Feng L, Zhang W, Ding C. Phase transitions in a three-dimensional Ising model with cluster weight studied by Monte Carlo simulations. Physical Review E. 2021;104(4). <https://doi.org/10.1103/physreve.104.044132>
12. Yue W, Zhang T, Jing Z, Wu K, Yang Y, Yang Z, Wu Y, Bu W, Zheng K, Kang J, Lin Y, Tao Y, Yan B, Huang R, Yang Y. A scalable universal Ising machine based on interaction-centric storage and compute-in-memory. Nature Electronics. 2024;7(10):904–913. <https://doi.org/10.1038/s41928-024-01228-7>
13. Ziyang W, Chengxiang D, Wanzhou Z. Phase Transitions in Layered Ising Model with Cluster Weight by Monte Carlo Method. Journal of Shanxi University (Natural Science Edition). 2023.



¡Muchas gracias!

AneXos

```

#Se importan las librerías
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.linalg import eigh
from scipy.sparse.linalg import expm_multiply
from scipy import special
import matplotlib.pyplot as plt
import matplotlib.colors as colors
from matplotlib import cm
from sklearn.utils.extmath import randomized_svd
from tqdm import tqdm
import warnings
warnings.filterwarnings("ignore")

class SystemConfig:
    def __init__(self):
        # Parámetros físicos
        self.N = 10          # número de espines (sube/baja según tu RAM; 10 → dim 1024)
        self.J = 1.0         # acoplamiento ZZ
        self.h = 0.5         # campo transversal (Sx)
        self.periodic = False # condiciones periódicas en la cadena

        # Muestreo y proyección
        self.S = 200         # número de estados coherentes muestreados
        self.M_trunc = 60    # dimensión del subespacio proyectado (<= S)

        # Evolución temporal
        self.T = 8.0
        self.nt = 161
        self.t_list = np.linspace(0.0, self.T, self.nt)
        self.cheb_order = 120 # orden de la expansión de Chebyshev

        # Visualización / salida
        self.plot_dpi = 150
        self.colormap = 'viridis'
        self.save_figs = True

np.random.seed(42)
config = SystemConfig()
plt.rcParams["figure.dpi"] = config.plot_dpi

# se importa quimb si está disponible
try:
    import quimb
    from quimb import ham_ising
    HAVE_QUIMB = True
except Exception:
    HAVE_QUIMB = False

#Operadores y matrices de Pauli
sx = np.array([[0.0, 1.0], [1.0, 0.0]], dtype=complex)

```

```

HAVE_QUIMB = False

#Operadores y matrices de Pauli
sx = np.array([[0.0, 1.0], [1.0, 0.0]], dtype=complex)
sy = np.array([[0.0, -1j], [1j, 0.0]], dtype=complex)
sz = np.array([[1.0, 0.0], [0.0, -1.0]], dtype=complex)
id2 = np.eye(2, dtype=complex)

def kron_list_sparse(mats):
    out = mats[0]
    for A in mats[1:]:
        out = sp.kron(out, A, format='csr')
    return out

def kron_list_dense(mats): # Producto de kronecker para listas
    out = mats[0]
    for A in mats[1:]:
        out = np.kron(out, A)
    return out

def build_Sz_total_sparse(N): #operador Total de S^z
    dim = 2**N
    Sz_total = sp.csr_matrix((dim, dim), dtype=complex)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        Sz_total = Sz_total + kron_list_sparse(ops)
    return Sz_total

def build_spin_operators_dense(N):
    #Lista de operadores sz (densos) para correlaciones
    ops_list = []
    for i in range(N):
        ops = [np.eye(2, dtype=complex) for _ in range(N)]
        ops[i] = sz
        ops_list.append(kron_list_dense(ops))
    return ops_list

# Hamiltoniano de Ising
def ising_hamiltonian_sparse(N, J=1.0, h=0.5, periodic=False, use_quimb=True):
    #Construcción robusta del Hamiltoniano de Ising sparseándolo
    dim = 2**N
    if use_quimb and HAVE_QUIMB:
        try:
            Hq = ham_ising(N, j=J, bx=h, cyclic=periodic, sparse=True, dtype=complex)
            # quimb puede devolver sparse-like; intentar convertir a scipy csr
            try:
                H = sp.csr_matrix(Hq)
            except Exception:
                H = Hq
            return H
        except Exception:
            # fallback to quimb falling

```

```

except Exception:
    # fallback si quimb falla
    pass

H = sp.csr_matrix((dim, dim), dtype=complex)
n_bonds = N if periodic else (N - 1)
for i in range(n_bonds):
    j = (i + 1) % N
    ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
    ops[i] = sp.csr_matrix(sz)
    ops[j] = sp.csr_matrix(sz)
    H = H - J * kron_list_sparse(ops)
for i in range(N):
    ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
    ops[i] = sp.csr_matrix(sx)
    H = H - h * kron_list_sparse(ops)

return H

# Estados coherentes y proyecciones

def coherent_state(z):
    """Estado coherente spin-1/2 en  $CP^1$  dado  $z$  complejo."""
    norm = np.sqrt(1.0 + np.abs(z)**2)
    return np.array([1.0, z], dtype=complex) / norm

def build_product_coherent_vectors(N, z_list):
    # Construye matriz Data con estados producto  $|z\rangle^{\otimes N}$  por columnas
    S = len(z_list)
    dim = 2**N
    Data = np.zeros((dim, S), dtype=complex)
    for j, z in enumerate(z_list):
        single = coherent_state(z)
        psi = single
        for _ in range(1, N):
            psi = np.kron(psi, single)
        Data[:, j] = psi
    return Data

def sample_cp1_fibonacci(S):
    # Muestreo quasi-uniforme en la esfera (Fibonacci Lattice).
    indices = np.arange(0, S, dtype=float)
    phi = 2 * np.pi * indices * (1.0 / ((1.0 + np.sqrt(5.0)) / 2.0))
    theta = np.arccos(1.0 - 2.0 * (indices + 0.5) / S)
    z = np.tan(theta / 2.0) * np.exp(1j * phi)
    return z, theta, phi

def build_projection_from_data_matrix(Data, M):
    # Construye P usando randomized SVD (Data: dim x S).
    M_use = min(M, Data.shape[1])
    U, s, Vt = randomized_svd(Data, n_components=M_use, random_state=42)
    P = U[:, :M_use]
    return P, s[:M_use]

```

```

def build_projection_from_gram(Data, M):
    # Se construye P diagonalizando la matriz de Gram Data^{\dagger} Data
    G = Data.conj().T @ Data
    jitter = 1e-12 * np.trace(G) / max(1, G.shape[0])
    evals, evects = eig(G + jitter * np.eye(G.shape[0]))
    idx = np.argsort(evals)[::-1]
    evals = evals[idx]
    evects = evects[:, idx]
    M_eff = min(M, len(evals))
    vals = np.maximum(evals[:M_eff], 1e-16)
    V = evects[:, :M_eff]
    P = Data @ (V / np.sqrt(vals)[np.newaxis, :])
    Q, _ = np.linalg.qr(P)
    return Q, evals[:M_eff]

class ChebyshevPropagator:
    #Basado en altas energías dada la expansión en polinomios de Chebyshev.
    #Propagador basado en expansión por polinomios de Chebyshev para  $e^{-i H t}$ .
    #Se reescala H a  $H' \in [-1,1]$  y se aplica la recurrencia.

    def __init__(self, H, M=80):
        # Acepta H como scipy.sparse o array; convertir a csr
        self.H = H if sp.issparse(H) else sp.csr_matrix(H)
        self.M = int(M)
        self._prepare_rescaled_hamiltonian()

    def _prepare_rescaled_hamiltonian(self):
        # Estimar extremos espectrales con eigsh (k=2)
        k = 2
        try:
            emin_vals = spla.eigsh(self.H, k=k, which='SA', return_eigenvectors=False)
            emax_vals = spla.eigsh(self.H, k=k, which='LA', return_eigenvectors=False)
            emin = float(np.min(emin_vals))
            emax = float(np.max(emax_vals))
        except Exception:
            # Fallback a densificación solo si dim < \sim 3000
            H_arr = self.H.toarray()
            evals = np.linalg.eigvalsh(H_arr)
            emin, emax = float(evals.min()), float(evals.max())

        if emax <= emin:
            # evita división sobre cero; por lo que lo desplaza ligeramente de manera infinitesimal
            emax = emin + 1e-16

        self.emin = emin
        self.emax = emax
        self.a = (emax - emin) / 2.0
        self.b = (emax + emin) / 2.0

        I = sp.eye(self.H.shape[0], format='csr')
        self.H_rescaled = (self.H - self.b * I) * (1.0 / self.a)

    def evolve(self, psi0, t):
        """Evoluciona psi0 hasta tiempo t y devolvemos psi(t)."""
        M = self.M
        # coeffs usando Bessel
        coeff = np.empty(M + 1, dtype=complex)

```

```

def evolve(self, psi0, t):
    """Evoluciona psi0 hasta tiempo y devolvemos psi(t)."""
    M = self.M
    # coeffs usando Bessel
    coeff = np.empty(M + 1, dtype=complex)
    z = self.a * t
    pref = np.exp(-1j * self.b * t)
    coeff[0] = pref * special.jv(0, z)
    for n in range(1, M + 1):
        coeff[n] = 2.0 * pref * special.jv(n, z)

    # Recurrencia de Chebyshev accionando sobre cada vector
    T0 = psi0
    T1 = self.H_rescaled.dot(psi0)
    psi_t = coeff[0] * T0 + coeff[1] * T1

    for n in range(2, M + 1):
        Tn = 2.0 * self.H_rescaled.dot(T1) - T0
        psi_t = psi_t + coeff[n] * Tn
        T0, T1 = T1, Tn

    return psi_t

# -----
# Función de evolución (proyectada vs completa)
# -----
def project_and_evolve_chebyshev(H, P, psi0, t_list, cheb_order=80):
    """
    Proyecta H y psi0, evoluciona con Chebyshev en subespacio reducido y
    compara con evolución completa por Chebyshev.
    Devuelve: states_proj_lifted (nt x dim), states_full (nt x dim), H_eff, psi0_proj
    """
    # H puede ser sparse; HP = H @ P
    HP = H.dot(P) if sp.issparse(H) else np.asarray(H) @ P
    H_eff = P.conj().T @ HP

    # Estado inicial proyectado
    psi0_proj = P.conj().T @ psi0
    nrm = np.linalg.norm(psi0_proj)
    if nrm > 0:
        psi0_proj = psi0_proj / nrm

    # Preparar propagadores
    prop_eff = ChebyshevPropagator(sp.csr_matrix(H_eff), M=cheb_order)
    prop_full = ChebyshevPropagator(H if sp.issparse(H) else sp.csr_matrix(H), M=cheb_order)

    states_proj_sub = []
    states_full = []

    for t in tqdm(t_list, desc="Evolución Chebyshev"):
        psi_proj_t = prop_eff.evolve(psi0_proj, t)
        states_proj_sub.append(psi_proj_t.copy())
        psi_full_t = prop_full.evolve(psi0, t)
        states_full.append(psi_full_t.copy())

    states_proj_sub = np.array(states_proj_sub) # shape (nt, M_eff)
    states_full = np.array(states_full) # shape (nt, dim)

```

```

states_full = np.array(states_full) # shape (nt, dim)

# Levantar subespacio al espacio completo
states_proj_lifted = np.array([P @ v for v in states_proj_sub])

return states_proj_lifted, states_full, H_eff, psi0_proj

def compute_expectation_series(states, operator):
    """Calcula <op>(t) para una lista de estados (states: nt x dim)."""
    if sp.issparse(operator):
        return np.array([np.vdot(s, operator.dot(s)).real for s in states])
    else:
        return np.array([np.vdot(s, operator @ s).real for s in states])

def plot_spectra(H, H_eff, outname="spectra.png"):
    """Dibuja histogramas espectrales comparando completo y efectivo."""
    try:
        k = min(300, H.shape[0]-2)
        vals_full = spla.eigsh(H, k=k, return_eigenvalues=False, which='SA')
    except Exception:
        vals_full = np.linalg.eigvalsh(H.toarray() if sp.issparse(H) else np.asarray(H))
        vals_eff = eigh(H_eff)[0]

    plt.figure(figsize=(10, 4))
    plt.hist(vals_full, bins=80, alpha=0.6, label='Completo', density=True)
    plt.hist(vals_eff, bins=40, alpha=0.8, label='Efectivo', density=True)
    plt.xlabel("Energía")
    plt.ylabel("Densidad")
    plt.legend()
    plt.title("Densidad espectral: completo vs proyectado")
    if config.save_figs:
        plt.savefig(outname, bbox_inches='tight', dpi=config.plot_dpi)
    plt.show()

def plot_wavefunction_amplitude(states, t_list, max_basis_states=80, title="Evolución de la Función de Onda", outname="w
    """Imágenes de amplitud y fase para los primeros basis states (o componentes)."""
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
    n_basis = min(states.shape[1], max_basis_states)
    amplitud = np.abs(states[:, :n_basis])
    im1 = ax1.imshow(amplitud.T, aspect='auto', origin='lower',
                    extent=[t_list[0], t_list[-1], 0, n_basis],
                    cmap=config.colormap, interpolation='bilinear')
    ax1.set_ylabel("Estado base")
    ax1.set_title(f"{title} - Amplitud")
    plt.colorbar(im1, ax=ax1, label='|ψ|')

    phase = np.angle(states[:, :n_basis])
    im2 = ax2.imshow(phase.T, aspect='auto', origin='lower',
                    extent=[t_list[0], t_list[-1], 0, n_basis],
                    cmap='twilight', interpolation='bilinear')
    ax2.set_xlabel("Tiempo")
    ax2.set_ylabel("Estado base")
    ax2.set_title(f"{title} - Fase")
    plt.colorbar(im2, ax=ax2, label='Fase (rad)')
    plt.tight_layout()

```

```

ax2.set_title(f"{title} - Fase")
plt.colorbar(im2, ax=ax2, label='Fase (rad)')
plt.tight_layout()
if config.save_figs:
    plt.savefig(outname, bbox_inches='tight', dpi=config.plot_dpi)
plt.show()

def plot_probability_density(states, t_list, selected_states=[0, 1, 2, 3], outname="prob_density.png"):
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    axes = axes.flatten()
    for i, idx in enumerate(selected_states):
        if idx < states.shape[1]:
            prob = np.abs(states[:, idx])**2
            axes[i].plot(t_list, prob, linewidth=2)
            axes[i].set_xlabel("Tiempo")
            axes[i].set_ylabel(f" $|\psi_{\{idx\}}|^2$ ")
            axes[i].set_title(f"Probabilidad Estado {idx}")
            axes[i].grid(True, alpha=0.3)
    plt.tight_layout()
    if config.save_figs:
        plt.savefig(outname, bbox_inches='tight', dpi=config.plot_dpi)
    plt.show()

def plot_error_analysis(t_list, exp_full, exp_proj, fidelity=None, outname="error_analysis.png"):
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    axes = axes.flatten()

    error_sz = np.abs(exp_full - exp_proj)
    axes[0].plot(t_list, error_sz, 'r-', linewidth=2)
    axes[0].set_xlabel("Tiempo")
    axes[0].set_ylabel(" $|\Delta(S^z)|$ ")
    axes[0].set_title("Error en magnetización")
    axes[0].grid(True, alpha=0.3)
    axes[0].set_yscale("log")

    rel_error = error_sz / (np.abs(exp_full) + 1e-12)
    axes[1].plot(t_list, rel_error, 'g-', linewidth=2)
    axes[1].set_xlabel("Tiempo")
    axes[1].set_ylabel("Error relativo")
    axes[1].set_title("Error relativo en magnetización")
    axes[1].grid(True, alpha=0.3)
    axes[1].set_yscale("log")

    if fidelity is not None:
        axes[2].plot(t_list, fidelity, 'b-', linewidth=2)
        axes[2].set_xlabel("Tiempo")
        axes[2].set_ylabel("Fidelidad")
        axes[2].set_title("Fidelidad")
        axes[2].grid(True, alpha=0.3)
        axes[2].set_ylim(0, 1.05)

    cumulative_error = np.cumsum(error_sz) * (t_list[1] - t_list[0])
    axes[3].plot(t_list, cumulative_error, 'm-', linewidth=2)
    axes[3].set_xlabel("Tiempo")
    axes[3].set_ylabel("Error acumulado")
    axes[3].set_title("Error acumulado")
    axes[3].grid(True, alpha=0.3)

```

```

axes[3].set_title("Error acumulado")
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
if config.save_figs:
    plt.savefig(outname, bbox_inches='tight', dpi=config.plot_dpi)
plt.show()

def plot_comparison(t_list, exp_proj, exp_full=None, outname="comparison.png"):
    plt.figure(figsize=(10, 6))
    plt.plot(t_list, exp_proj, label="Proyectado", linewidth=2)
    if exp_full is not None:
        plt.plot(t_list, exp_full, label="Completo", linestyle="--", linewidth=2)
    plt.xlabel("Tiempo")
    plt.ylabel(r" $\langle S^z \rangle$  por sitio")
    plt.legend()
    plt.grid(True, alpha=0.3)
    if config.save_figs:
        plt.savefig(outname, bbox_inches='tight', dpi=config.plot_dpi)
    plt.show()

# -----
# Main flow
# -----
def main():
    print("=== INICIANDO: Modelo de Ising - Chebyshev (proyectado vs completo) ===")
    N = config.N
    dim = 2**N
    print(f"N = {N} → dim = {dim}")

    # Construir Hamiltoniano
    print("Construyendo Hamiltoniano...")
    H = ising_hamiltonian_sparse(N, J=config.J, h=config.h, periodic=config.periodic, use_quimb=True)

    # Estado inicial:  $|00\dots0\rangle$ 
    psi0 = np.zeros(dim, dtype=complex)
    psi0[0] = 1.0

    # Operador Sz total (sparse) y operadores para correlaciones (densos)
    Sz_total = build_Sz_total_sparse(N)
    spin_ops = build_spin_operators_dense(N)

    # Muestreo coherente y Data matrix
    print("Muestreando estados coherentes (Fibonacci) y construyendo Data matrix...")
    z_list, theta, phi = sample_cp1_fibonacci(config.S)
    Data = build_product_coherent_vectors(N, z_list)
    norms = np.linalg.norm(Data, axis=0)
    norms[norms == 0] = 1.0
    Data = Data / norms[np.newaxis, :]

    # Construir proyección
    print("Construyendo proyección (SVD/Gram)...")
    if dim > config.S:
        P, evals = build_projection_from_gram(Data, config.M_trunc)
    else:
        P, svals = build_projection_from_data_matrix(Data, config.M_trunc)
    M = P.shape[1]

```

```

M = P.shape[1]
print(f"Dimensión reducida M = {M}")

# Evolución con Chebyshev
print("Evolución (Chebyshev) - proyectado y completo...")
states_proj_lifted, states_full, H_eff, psi0_proj = project_and_evolve_chebyshev(
    H, P, psi0, config.t_list, cheb_order=config.cheb_order
)

# Cálculo de expectativas (por sitio)
print("Calculando observables (S^z) por sitio...")
exp_proj = compute_expectation_series(states_proj_lifted, Sz_total) / M
exp_full = compute_expectation_series(states_full, Sz_total) / M

# Fidelidad entre estados (completo vs proyectado)
fidelity = np.abs(np.sum(states_full.conj() * states_proj_lifted, axis=1))**2

# Espectros
print("Generando espectros...")
plot_spectra(H, H_eff, outname="spectra.png")

# Wavefunction amplitude + phase (proyectado)
print("Generando plot de función de onda (proyectado)...")
plot_wavefunction_amplitude(states_proj_lifted, config.t_list, max_basis_states=80, title="Función de onda proyectada")
plot_probability_density(states_proj_lifted, config.t_list, selected_states=[0,1,2,3], outname="prob_density_project")

# Comparación magnetización
print("Comparando magnetización proyectada vs completa...")
plot_comparison(config.t_list, exp_proj, exp_full, outname="comparison_magnetization.png")

# Error y fidelidad
print("Analizando error y fidelidad...")
plot_error_analysis(config.t_list, exp_full, exp_proj, fidelity=fidelity, outname="error_and_fidelity.png")

# Guardar algunas métricas
avg_fidelity = float(np.mean(fidelity))
max_error = float(np.max(np.abs(exp_full - exp_proj)))
print("\n=== MÉTRICAS ===")
print(f"Fidelidad promedio: {avg_fidelity:.6e}")
print(f"Error máximo en (S^z) por sitio: {max_error:.6e}")
print(f"Dimensión reducida: {M}/{dim} ({100.0 * M / dim:.4f}%)")

print("\n=== PROCESO COMPLETADO ===")

if __name__ == "__main__":
    main()

```

```

import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
from scipy.linalg import eigh, expm
from scipy import special
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.colors as colors
from matplotlib import cm
from sklearn.utils.extmath import randomized_svd
from tqdm import tqdm
import time
import warnings
warnings.filterwarnings("ignore")

class SystemConfig:
    def __init__(self):
        # Parámetros físicos (Ising)
        self.N = 10 # número de espines
        self.J = 1.0 # acoplamiento ZZ
        self.h = 0.8 # campo transversal (Sx)
        self.periodic = False # condiciones periódicas

        # Muestreo y proyección
        self.S = 500 # número de estados coherentes
        self.M_trunc = 50 # dimensión del subespacio proyectado

        # Evolución temporal (Chebyshev)
        self.T = 8.0 # tiempo final
        self.nt = 161 # número de puntos temporales
        self.t_list = np.linspace(0.0, self.T, self.nt)
        self.cheb_order = 120 # orden de Chebyshev

        # Flujo de Ricci
        self.BETA_MAX = 5.0 # máximo parámetro de escala
        self.n_beta = 15 # número de puntos en beta

        # Visualización
        self.plot_dpi = 150
        self.colormap = 'viridis'
        self.save_figs = True
        self.plot_ricci_flow = True
        self.plot_comparison = True

np.random.seed(42)
config = SystemConfig()
plt.rcParams["figure.dpi"] = config.plot_dpi

# Operadores de Pauli
sx = np.array([[0.0, 1.0], [1.0, 0.0]], dtype=complex)
sy = np.array([[0.0, -1j], [1j, 0.0]], dtype=complex)
sz = np.array([[1.0, 0.0], [0.0, -1.0]], dtype=complex)
id2 = np.eye(2, dtype=complex)

```

```

def kron_list_sparse(mats):
    out = mats[0]
    for A in mats[1:]:
        out = sp.kron(out, A, format='csr')
    return out

def kron_list_dense(mats):
    out = mats[0]
    for A in mats[1:]:
        out = np.kron(out, A)
    return out

def build_Sz_total_sparse(N):
    dim = 2**N
    Sz_total = sp.csr_matrix((dim, dim), dtype=complex)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        Sz_total = Sz_total + kron_list_sparse(ops)
    return Sz_total

def build_spin_operators_dense(N):
    ops_list = []
    for i in range(N):
        ops = [np.eye(2, dtype=complex) for _ in range(N)]
        ops[i] = sz
        ops_list.append(kron_list_dense(ops))
    return ops_list

def ising_hamiltonian_sparse(N, J=1.0, h=0.5, periodic=False, use_quimb=True):
    dim = 2**N

    # Intentar usar quimb si está disponible
    try:
        import quimb
        from quimb import ham_ising
        Hq = ham_ising(N, j=J, bx=h, cyclic=periodic, sparse=True, dtype=complex)
        try:
            H = sp.csr_matrix(Hq)
        except Exception:
            H = Hq
        return H
    except Exception:
        pass

    # Construcción manual
    H = sp.csr_matrix((dim, dim), dtype=complex)
    n_bonds = N if periodic else (N - 1)

    for i in range(n_bonds):
        j = (i + 1) % N
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        ops[j] = sp.csr_matrix(sz)
        H = H - J * kron_list_sparse(ops)

    for i in range(N):

```

```

        for i in range(N):
            ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
            ops[i] = sp.csr_matrix(sx)
            H = H - h * kron_list_sparse(ops)

    return H

def coherent_state(z):
    norm = np.sqrt(1.0 + np.abs(z)**2)
    return np.array([1.0, z], dtype=complex) / norm

def sample_cp1_stratified(count, method='fibonacci'):
    if method == 'fibonacci':
        indices = np.arange(0, count, dtype=float)
        phi = 2 * np.pi * indices * (1.0 / ((1.0 + np.sqrt(5.0)) / 2.0))
        theta = np.arccos(1.0 - 2.0 * (indices + 0.5) / count)
    elif method == 'haar':
        u = np.random.uniform(-1.0, 1.0, size=count)
        phi = np.random.uniform(0.0, 2*np.pi, size=count)
        theta = np.arccos(u)
    elif method == 'grid':
        n_theta = int(np.sqrt(count))
        n_phi = int(count / n_theta)
        theta = np.arccos(1.0 - 2.0 * (np.arange(n_theta) + 0.5) / n_theta)
        phi = 2 * np.pi * np.arange(n_phi) / n_phi
        theta, phi = np.meshgrid(theta, phi)
        theta, phi = theta.flatten(), phi.flatten()

    z = np.tan(theta/2.0) * np.exp(1j*phi)
    return z[:count], theta[:count], phi[:count]

def build_product_coherent_vectors(N, z_list, heterogeneous=False):
    S = len(z_list)
    dim = 2**N
    Data = np.zeros((dim, S), dtype=complex)

    if heterogeneous:
        # Diferente z para cada espín en cada muestra
        for j in range(S):
            # Para cada muestra, genera N valores de z
            if j == 0:
                # Primera muestra, todos iguales tq
                z_samples = np.array([z_list[j]] * N)
            else:
                # Muestras posteriores: aleatorias
                _, _, phi_all = sample_cp1_stratified(N * S, method='haar')
                z_samples = np.tan(np.random.uniform(0, np.pi/2, N)/2.0) * np.exp(1j*phi_all[j*N:(j+1)*N])

            # Construir producto tensor
            psi = coherent_state(z_samples[0])
            for i in range(1, N):
                single = coherent_state(z_samples[i])
                psi = np.kron(psi, single)
            Data[:, j] = psi
    else:
        # Mismo z para todos los espines en cada muestra
        for j, z in enumerate(z_list):
            single = coherent_state(z)

```

```

        for j, z in enumerate(z_list):
            single = coherent_state(z)
            psi = single
            for _ in range(1, N):
                psi = np.kron(psi, single)
            Data[:, j] = psi

# Normalizar cada columna
norms = np.linalg.norm(Data, axis=0)
norms[norms == 0] = 1.0
Data = Data / norms[np.newaxis, :]

return Data

class RicciFlowOptimizer:

    def __init__(self, config):
        self.config = config

    def get_thermal_weights(self, H, Data, beta):
        if beta < 1e-16:
            return np.ones(Data.shape[1], dtype=float) / Data.shape[1]

# Operador de densidad térmica
H_dense = H.todense() if sp.issparse(H) else H
P_op = expm(-beta * H_dense)

weights = np.zeros(Data.shape[1], dtype=float)
for k in range(Data.shape[1]):
    Zk = Data[:, k]
    weights[k] = np.real(Zk.conj().T @ (P_op @ Zk))

# Suavizado y normalización
weights = np.maximum(weights, 1e-12)
weights /= np.sum(weights)

return weights

def build_weighted_projection(self, Data, M, weights, regularization=1e-8):
    sqrt_weights = np.sqrt(weights)
    D_tilde = Data * sqrt_weights[np.newaxis, :]

# Matriz Gram con regularización
G_tilde = D_tilde.conj().T @ D_tilde
G_tilde += regularization * np.eye(G_tilde.shape[0])

# Descomposición espectral
evals, evecs = eigh(G_tilde)
idx = np.argsort(evals)[::-1]
evals, evecs = evals[idx], evecs[:, idx]

# Selección adaptativa de dimensión
M_eff = min(M, len(evals))

# Construcción de la base proyectada
P = np.zeros((Data.shape[0], M_eff), dtype=complex)
for m in range(M_eff):
    v = evecs[:, m]
    lam = evals[m]

```

```

        lam = evals[m]
        P[:, m] = (D_tilde @ v) / np.sqrt(lam)

return P, evals, M_eff

def calculate_fidelity(self, P, psi0):
    psi0_proj = P.conj().T @ psi0
    psi0_reconstructed = P @ psi0_proj
    overlap = np.vdot(psi0_reconstructed, psi0)
    return np.abs(overlap)**2

def optimize_projection(self, H, psi0, Data, beta_list=None):
    if beta_list is None:
        beta_list = np.linspace(0.01, self.config.BETA_MAX, self.config.n_beta)

    fidelities = []
    projections = []
    weights_list = []

    print("Optimizando proyección con flujo de Ricci...")
    for beta in tqdm(beta_list, desc="β"):
# 1. Calcular pesos térmicos
weights = self.get_thermal_weights(H, Data, beta)
weights_list.append(weights)

# 2. Construir proyección ponderada
P, evals, M_eff = self.build_weighted_projection(Data, self.config.M_trunc, weights)
projections.append(P)

# 3. Calcular fidelidad
fid = self.calculate_fidelity(P, psi0)
fidelities.append(fid)

# Encontrar mejor proyección
best_idx = np.argmax(fidelities)
best_beta = beta_list[best_idx]
best_P = projections[best_idx]
best_fidelity = fidelities[best_idx]

# Construir Hamiltoniano efectivo
HP = H.dot(best_P) if sp.issparse(H) else H @ best_P
H_eff = best_P.conj().T @ HP

results = {
    'beta_list': beta_list,
    'fidelities': np.array(fidelities),
    'weights_list': weights_list,
    'best_beta': best_beta,
    'best_fidelity': best_fidelity,
    'best_P': best_P,
    'best_H_eff': H_eff,
    'best_idx': best_idx
}

return results

class ChebyshevPropagator:

    def __init__(self, H, H_0):

```

```

return results

class ChebyshevPropagator:

    def __init__(self, H, M=80):
        self.H = H if sp.issparse(H) else sp.csr_matrix(H)
        self.M = int(M)
        self._prepare_rescaled_hamiltonian()

    def _prepare_rescaled_hamiltonian(self):
        k = 2
        try:
            emin_vals = spla.eigsh(self.H, k=k, which='SA', return_eigenvectors=False)
            emax_vals = spla.eigsh(self.H, k=k, which='LA', return_eigenvectors=False)
            emin = float(np.min(emin_vals))
            emax = float(np.max(emax_vals))
        except Exception:
            H_arr = self.H.toarray()
            evals = np.linalg.eigvalsh(H_arr)
            emin, emax = float(evals.min()), float(evals.max())

        if emax <= emin:
            emax = emin + 1e-10

        self.emin = emin
        self.emax = emax
        self.a = (emax - emin) / 2.0
        self.b = (emax + emin) / 2.0

        I = sp.eye(self.H.shape[0], format='csr')
        self.H_rescaled = (self.H - self.b * I) * (1.0 / self.a)

    def evolve(self, psi0, t):
        M = self.M
        coeff = np.empty(M + 1, dtype=complex)
        z = self.a * t
        pref = np.exp(-1j * self.b * t)
        coeff[0] = pref * special.jv(0, z)

        for n in range(1, M + 1):
            coeff[n] = 2.0 * pref * special.jv(n, z)

        T0 = psi0
        T1 = self.H_rescaled.dot(psi0)
        psi_t = coeff[0] * T0 + coeff[1] * T1

        for n in range(2, M + 1):
            Tn = 2.0 * self.H_rescaled.dot(T1) - T0
            psi_t = psi_t + coeff[n] * Tn
            T0, T1 = T1, Tn

        return psi_t

    def project_and_evolve_chebyshev(H, P, psi0, t_list, cheb_order=80):
        HP = H.dot(P) if sp.issparse(H) else H @ P
        H_eff = P.conj().T @ HP

        # Estado inicial proyectado
        psi0_proj = P.conj().T @ psi0

```

```

        # Estado inicial proyectado
        psi0_proj = P.conj().T @ psi0
        nrm = np.linalg.norm(psi0_proj)
        if nrm > 0:
            psi0_proj = psi0_proj / nrm

        # Preparar propagadores
        prop_eff = ChebyshevPropagator(sp.csr_matrix(H_eff), M=cheb_order)
        prop_full = ChebyshevPropagator(H if sp.issparse(H) else sp.csr_matrix(H), M=cheb_order)

        states_proj_sub = []
        states_full = []

        for t in tqdm(t_list, desc="Evolución temporal"):
            psi_proj_t = prop_eff.evolve(psi0_proj, t)
            states_proj_sub.append(psi_proj_t.copy())
            psi_full_t = prop_full.evolve(psi0, t)
            states_full.append(psi_full_t.copy())

        states_proj_sub = np.array(states_proj_sub)
        states_full = np.array(states_full)

        # Levantar al espacio completo
        states_proj_lifted = np.array([P @ v for v in states_proj_sub])

        return states_proj_lifted, states_full, H_eff, psi0_proj

    def compute_expectation_series(states, operator):
        """Calcula <op>(t) para una lista de estados"""
        if sp.issparse(operator):
            return np.array([np.vdot(s, operator.dot(s)).real for s in states])
        else:
            return np.array([np.vdot(s, operator @ s).real for s in states])

# =====
# VISUALIZACIÓN COMBINADA
# =====

    def plot_ricci_flow_results(results):
        """Grafica resultados del flujo de Ricci"""
        beta_list = results['beta_list']
        fidelities = results['fidelities']

        fig, axes = plt.subplots(1, 2, figsize=(12, 5))

        # Fidelidad vs beta
        axes[0].plot(beta_list, fidelities, 'o-', linewidth=2.5, color='#8b5cf6')
        axes[0].axvline(results['best_beta'], color='r', linestyle='--', alpha=0.7,
                        label=f'Óptimo:  $\beta = \{results["best\_beta"]:.3f\}$ ')
        axes[0].set_xlabel(r'Parámetro de escala  $\beta$ ', fontsize=12)
        axes[0].set_ylabel('Fidelidad  $\mathcal{F}$ ', fontsize=12)
        axes[0].set_title('Flujo de Ricci: Fidelidad vs  $\beta$ ', fontsize=14)
        axes[0].legend()
        axes[0].grid(True, alpha=0.3)

        # Distribución de pesos para el mejor beta
        best_weights = results['weights_list'][results['best_idx']]
        axes[1].hist(best_weights, bins=30, alpha=0.7, color='#10b981', edgecolor='black')

```

```

# Distribución de pesos para el mejor beta
best_weights = results['weights_list'][results['best_idx']]
axes[1].hist(best_weights, bins=30, alpha=0.7, color='#10b981', edgecolor='black')
axes[1].set_xlabel('Peso térmico', fontsize=12)
axes[1].set_ylabel('Frecuencia', fontsize=12)
axes[1].set_title(f'Distribución de pesos ( $\beta$ ={results["best_beta"]:.3f})', fontsize=14)
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
if config.save_figs:
    plt.savefig('ricci_flow_optimization.png', dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

def plot_projection_analysis(P, H_eff, beta_optimal):
    """Análisis de la proyección óptima"""
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))

    # Matriz de proyección
    im1 = axes[0].imshow(np.abs(P), aspect='auto', cmap='hot')
    axes[0].set_title(f'Matriz de Proyección ( $\beta$ ={beta_optimal:.2f})')
    axes[0].set_xlabel('Dimensión Reducida')
    axes[0].set_ylabel('Dimensión Completa')
    plt.colorbar(im1, ax=axes[0])

    # Hamiltoniano efectivo (parte real)
    im2 = axes[1].imshow(H_eff.real, aspect='auto', cmap='RdBu_r')
    axes[1].set_title('Hamiltoniano Efectivo (Real)')
    axes[1].set_xlabel('Base Reducida')
    axes[1].set_ylabel('Base Reducida')
    plt.colorbar(im2, ax=axes[1])

    # Hamiltoniano efectivo (parte imaginaria)
    im3 = axes[2].imshow(H_eff.imag, aspect='auto', cmap='RdBu_r')
    axes[2].set_title('Hamiltoniano Efectivo (Imag)')
    axes[2].set_xlabel('Base Reducida')
    axes[2].set_ylabel('Base Reducida')
    plt.colorbar(im3, ax=axes[2])

    plt.tight_layout()
    if config.save_figs:
        plt.savefig('optimal_projection_analysis.png', dpi=config.plot_dpi, bbox_inches='tight')
    plt.show()

def plot_wavefunction_amplitude(states, t_list, max_basis_states=80, title="Evolución"):
    """Imágenes de amplitud y fase"""
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8))
    n_basis = min(states.shape[1], max_basis_states)

    amplitud = np.abs(states[:, :n_basis])
    im1 = ax1.imshow(amplitud.T, aspect='auto', origin='lower',
                    extent=[t_list[0], t_list[-1], 0, n_basis],
                    cmap=config.colormap, interpolation='bilinear')
    ax1.set_ylabel("Estado base")
    ax1.set_title(f"{title} - Amplitud")
    plt.colorbar(im1, ax=ax1, label='| $\psi$ |')

    phase = np.angle(states[:, :n_basis])

```

```

phase = np.angle(states[:, :n_basis])
im2 = ax2.imshow(phase.T, aspect='auto', origin='lower',
                extent=[t_list[0], t_list[-1], 0, n_basis],
                cmap='twilight', interpolation='bilinear')
ax2.set_xlabel("Tiempo")
ax2.set_ylabel("Estado base")
ax2.set_title(f"{title} - Fase")
plt.colorbar(im2, ax=ax2, label='Fase (rad)')

plt.tight_layout()
if config.save_figs:
    plt.savefig(f'{title.lower().replace(" ", "_")}_wavefunction.png',
                dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

def plot_comparison(t_list, exp_proj, exp_full=None, title="Comparación"):
    """Compara magnetización proyectada vs completa"""
    plt.figure(figsize=(10, 6))
    plt.plot(t_list, exp_proj, label="Proyectado (Ricci)", linewidth=2)
    if exp_full is not None:
        plt.plot(t_list, exp_full, label="Completo", linestyle="--", linewidth=2)
    plt.xlabel("Tiempo")
    plt.ylabel(r" $\langle S^z \rangle$  por sitio")
    plt.title(title)
    plt.legend()
    plt.grid(True, alpha=0.3)
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}.png',
                    dpi=config.plot_dpi, bbox_inches='tight')
    plt.show()

def main_integrated():

    # 1. Inicialización
    N = config.N
    dim = 2**N
    print(f"N = {N}   dim = {dim}")

    # 2. Construir Hamiltoniano
    print("Construyendo Hamiltoniano...")
    H = ising_hamiltonian_sparse(N, J=config.J, h=config.h,
                                periodic=config.periodic, use_quimb=True)

    # 3. Estado inicial
    psi0 = np.zeros(dim, dtype=complex)
    psi0[0] = 1.0 # |00...0>

    # 4. Muestreo de estados coherentes
    print("Muestreando estados coherentes...")
    z_list, theta, phi = sample_cpl_stratified(config.S, method='fibonacci')
    Data = build_product_coherent_vectors(N, z_list, heterogeneous=False)

    # 5. Optimización con Flujo de Ricci
    print("\n=== OPTIMIZACIÓN CON FLUJO DE RICCI ===")
    ricci_optimizer = RicciFlowOptimizer(config)

```

```

ricci_optimizer = RicciFlowOptimizer(config)

beta_list = np.linspace(0.01, config.BETA_MAX, config.n_beta)
ricci_results = ricci_optimizer.optimize_projection(H, psi0, Data, beta_list)

print(f"\nMejor  $\beta$  encontrado: {ricci_results['best_beta']:.4f}")
print(f"Fidelidad óptima: {ricci_results['best_fidelity']:.6f}")
print(f"Dimensión efectiva: {ricci_results['best_P'].shape[1]}/{dim} "
      f"({100.0*ricci_results['best_P'].shape[1]/dim:.2f}%)")

# 6. Visualizar resultados del flujo de Ricci
if config.plot_ricci_flow:
    plot_ricci_flow_results(ricci_results)
    plot_projection_analysis(ricci_results['best_P'],
                           ricci_results['best_H_eff'],
                           ricci_results['best_beta'])

# 7. Evolución temporal con la proyección óptima
print("\n=== EVOLUCIÓN TEMPORAL ===")
P_optimal = ricci_results['best_P']

print("Evolucionando con Chebyshev...")
states_proj, states_full, H_eff, psi0_proj = project_and_evolve_chebyshev(
    H, P_optimal, psi0, config.t_list, cheb_order=config.cheb_order
)

# 8. Observables
Sz_total = build_Sz_total_sparse(N)
exp_proj = compute_expectation_series(states_proj, Sz_total) / N
exp_full = compute_expectation_series(states_full, Sz_total) / N

# 9. Fidelidad entre estados
fidelity = np.abs(np.sum(states_full.conj() * states_proj, axis=1))**2

# 10. Visualizaciones de evolución
print("\n=== VISUALIZACIONES ===")

# Función de onda proyectada
plot_wavefunction_amplitude(states_proj, config.t_list,
                             title="Evolución Proyectada (Ricci)")

# Comparación de magnetización
if config.plot_comparison:
    plot_comparison(config.t_list, exp_proj, exp_full,
                    title="Magnetización: Ricci vs Completo")

# Análisis de error
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

error_sz = np.abs(exp_full - exp_proj)
ax1.plot(config.t_list, error_sz, 'r-', linewidth=2)
ax1.set_xlabel("Tiempo")
ax1.set_ylabel(" $|\Delta\langle S^z \rangle|$ ")
ax1.set_title("Error en magnetización")
ax1.grid(True, alpha=0.3)
ax1.set_yscale("log")

ax2.plot(config.t_list, fidelity, 'b-', linewidth=2)

```

```

ax2.plot(config.t_list, fidelity, 'b-', linewidth=2)
ax2.set_xlabel("Tiempo")
ax2.set_ylabel("Fidelidad")
ax2.set_title("Fidelidad durante la evolución")
ax2.grid(True, alpha=0.3)
ax2.set_ylim(0, 1.05)

plt.tight_layout()
if config.save_figs:
    plt.savefig('error_analysis_ricci.png', dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

# 11. Métricas finales
print("\n=== MÉTRICAS FINALES ===")
print(f"Fidelidad inicial (proyección): {ricci_results['best_fidelity']:.6e}")
print(f"Fidelidad promedio (evolución): {np.mean(fidelity):.6e}")
print(f"Error máximo en  $\langle S^z \rangle$ : {np.max(error_sz):.6e}")
print(f"Dimensión reducida: {P_optimal.shape[1]}/{dim} "
      f"({100.0*P_optimal.shape[1]/dim:.2f}%)")

print("\n=== PROCESO COMPLETADO ===")

return {
    'ricci_results': ricci_results,
    'states_proj': states_proj,
    'states_full': states_full,
    'exp_proj': exp_proj,
    'exp_full': exp_full,
    'fidelity': fidelity
}

if __name__ == "__main__":
    results = main_integrated()

```

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import eigh
from scipy.sparse.linalg import expm_multiply
import scipy.sparse as sp
import time

def lambda_plus(beta, h, J=1.0):
    bJ = beta * J
    bh = beta * h
    e2bJ = np.exp(2 * bJ)
    term = e2bJ * np.sinh(bh)**2 + np.exp(-2 * bJ)
    return np.exp(bJ) * np.cosh(bh) + np.sqrt(term)

def magnetization_1d_analytic(T_vals, J=1.0, h=0.0, kB=1.0):
    T = np.array(T_vals, dtype=float)
    beta = 1.0 / (kB * T)
    bh = beta * h
    bJ = beta * J
    e2bJ = np.exp(2 * bJ)
    sqrt_arg = e2bJ * np.sinh(bh)**2 + np.exp(-2 * bJ)
    lam = np.exp(bJ) * np.cosh(bh) + np.sqrt(sqrt_arg)
    # derivada de lam respecto a (beta*h): dL_dbh
    dl_dbh = np.exp(bJ) * np.sinh(bh) + (e2bJ * np.sinh(bh) * np.cosh(bh)) / np.sqrt(sqrt_arg)
    m = dl_dbh / lam
    return m

def free_energy_1d_per_site(T_vals, J=1.0, h=0.0, kB=1.0):
    T = np.array(T_vals, dtype=float)
    beta = 1.0 / (kB * T)
    lam = lambda_plus(beta, h, J)
    f = - (1.0 / beta) * np.log(lam)
    return f

def magnetization_2d_onsager(T_vals, J=1.0, kB=1.0):
    T = np.array(T_vals, dtype=float)
    beta = 1.0 / (kB * T)
    x = np.sinh(2 * beta * J)
    m = np.zeros_like(T)
    # condición  $T < T_c \Leftrightarrow x > 1$ 
    mask = x > 1.0
    with np.errstate(divide='ignore', invalid='ignore'):
        val = 1.0 - x**(-4)
        val[val < 0] = 0.0
        m[mask] = val[mask]**(1.0/8.0)
    return m

def Tc_2d(J=1.0, kB=1.0):
    return 2.0 * J / np.log(1.0 + np.sqrt(2.0))

# -----
# 2) Transfer-matrix 1D (numérico) for validation
# -----

def transfer_matrix_T(J=1.0, h=0.0, beta=1.0):
    """Construye la matriz de transferencia 2x2 para Ising 1D."""

```

```

    def transfer_matrix_T(J=1.0, h=0.0, beta=1.0):
        """Construye la matriz de transferencia 2x2 para Ising 1D."""
        bJ = beta * J
        bh = beta * h
        Tm = np.array([[np.exp(bJ + bh), np.exp(-bJ)],
                       [np.exp(-bJ), np.exp(bJ - bh)]], dtype=float)
        return Tm

    def magnetization_1d_transfermatrix(T_vals, J=1.0, h=0.0, kB=1.0):
        T_vals = np.array(T_vals, dtype=float)
        m = np.zeros_like(T_vals)
        for i, Tval in enumerate(T_vals):
            beta = 1.0 / (kB * Tval)
            Tm = transfer_matrix_T(J, h, beta)
            vals, vecs = np.linalg.eig(Tm)
            idx = np.argmax(np.real(vals))
            lam = vals[idx]
            dh = 1e-8
            lam_plus = lambda_plus(beta, h + dh, J)
            lam_minus = lambda_plus(beta, h - dh, J)
            dln = (np.log(lam_plus) - np.log(lam_minus)) / (2 * dh)
            m[i] = dln
        return m

    def mc_ising2d(L, T, J=1.0, n_steps=2000, equilibrate=1000, seed=None):
        rng = np.random.default_rng(seed)
        spins = rng.choice([-1, 1], size=(L, L))
        beta = 1.0 / T
        N = L * L

        def local_energy(i, j):
            s = spins[i, j]
            nb = spins[(i+1)%L, j] + spins[(i-1)%L, j] + spins[i, (j+1)%L] + spins[i, (j-1)%L]
            return -J * s * nb

        for _ in range(equilibrate):
            i = rng.integers(0, L)
            j = rng.integers(0, L)
            deltaE = -2 * local_energy(i, j)
            if deltaE <= 0 or rng.random() < np.exp(-beta * deltaE):
                spins[i, j] *= -1

        mags = []
        for _ in range(n_steps):
            for _ in range(N):
                i = rng.integers(0, L)
                j = rng.integers(0, L)
                deltaE = -2 * local_energy(i, j)
                if deltaE <= 0 or rng.random() < np.exp(-beta * deltaE):
                    spins[i, j] *= -1
            mags.append(np.abs(np.sum(spins)) / N)
        return np.mean(mags), np.std(mags) / np.sqrt(len(mags))

    def tensor_power(vec, N):
        if N == 0:

```

```

def tensor_power(vec, N):
    if N == 0:
        return np.array([1.0], dtype=vec.dtype)
    def _pow(v, n):
        if n == 1:
            return v
        if n % 2 == 0:
            half = _pow(v, n//2)
            return np.kron(half, half)
        else:
            return np.kron(v, _pow(v, n-1))
    return _pow(vec, N)

def build_product_coherent_same_z(z_array, N):

    S = len(z_array)
    dim = 2**N
    Data = np.zeros((dim, S), dtype=complex)
    for j, z in enumerate(z_array):
        ket = np.array([1.0, z], dtype=complex)
        ket /= np.linalg.norm(ket)
        Data[:, j] = tensor_power(ket, N)
    return Data

def main_compare_and_plot():
    J = 1.0
    kB = 1.0

    T_min, T_max = 0.01, 5.0
    T_vals = np.linspace(T_min, T_max, 300)

    hs = [0.0, 0.05, 0.2]
    plt.figure(figsize=(8,5))
    for h in hs:
        m_analytic = magnetization_1d_analytic(T_vals, J=J, h=h, kB=kB)
        plt.plot(T_vals, m_analytic, label=f'Analytic 1D, h={h}')
    plt.xlabel('Temperature T (J/k_B)')
    plt.ylabel('Magnetization per site m (1D)')
    plt.title('Ising 1D - Magnetization (analytic)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('ising1d_analytic.png', dpi=200)
    print("Saved ising1d_analytic.png")
    plt.show()

    print("Computing transfer-matrix numeric validation (1D, h=0) ...")
    t0 = time.time()
    m_tm = magnetization_1d_transfermatrix(T_vals, J=J, h=0.0, kB=kB)
    print(f"Transfer-matrix computed in {time.time()-t0:.2f}s")
    plt.figure(figsize=(8,5))
    plt.plot(T_vals, magnetization_1d_analytic(T_vals, J=J, h=0.0, kB=kB), label='Analytic 1D (h=0)')
    plt.plot(T_vals, m_tm, '--', label='Transfer-matrix numeric', linewidth=1)
    plt.xlabel('Temperature T (J/k_B)')
    plt.ylabel('Magnetization per site m (1D)')
    plt.title('Ising 1D: analytic vs transfer-matrix (validation)')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('ising1d_validate_transfer.png', dpi=200)
    print("Saved ising1d_validate_transfer.png")
    plt.show()

    Ls = [16, 32]
    nT_mc = 20
    T_mc_vals = np.linspace(0.1, 4.0, nT_mc)
    mc_results = {L: {'T': [], 'm': [], 'err': []} for L in Ls}

    for L in Ls:
        print(f"MC for L={L}")
        for T in T_mc_vals:
            m_avg, m_err = mc_ising2d(L, T, J=J, n_steps=400, equilibrate=200, seed=1234)
            mc_results[L]['T'].append(T)
            mc_results[L]['m'].append(m_avg)
            mc_results[L]['err'].append(m_err)

    m2_analytic = magnetization_2d_onsager(T_vals, J=J, kB=kB)
    Tc = Tc_2d(J=J, kB=kB)
    print(f"2D critical temperature Tc = {Tc:.6f} (J/kB={J})")

    plt.figure(figsize=(8,5))
    plt.plot(T_vals, m2_analytic, label='Onsager analytic (2D, h=0)')
    for L in Ls:
        plt.errorbar(mc_results[L]['T'], mc_results[L]['m'], yerr=mc_results[L]['err'],
                    fmt='o-', label=f'MC L={L}', capsize=3)
    plt.axvline(Tc, linestyle='--', label=f'Tc={Tc:.4f}')
    plt.xlabel('Temperature T (J/k_B)')
    plt.ylabel('Magnetization per site m (2D)')
    plt.title('Ising 2D - Onsager analytic vs Monte Carlo')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig('ising2d_analytic_vs_mc.png', dpi=200)
    print("Saved ising2d_analytic_vs_mc.png")
    plt.show()

    plt.figure(figsize=(8,5))
    for L in Ls:
        T_arr = np.array(mc_results[L]['T'])
        m_arr = np.array(mc_results[L]['m'])
        m_ons_mc = np.interp(T_arr, T_vals, m2_analytic)
        diff = m_arr - m_ons_mc
        plt.plot(T_arr, diff, 'o-', label=f'L={L}')
    plt.axhline(0, color='k', linestyle='--')
    plt.xlabel('Temperature T (J/k_B)')
    plt.ylabel('MC - Onsager (m)')
    plt.title('Residual: MC - Onsager')
    plt.legend()
    plt.grid(True)
    plt.tight_layout()

```

```
plt.tight_layout()
plt.savefig('ising2d_mc_minus_analytic.png', dpi=200)
print("Saved ising2d_mc_minus_analytic.png")
plt.show()

if __name__ == "__main__":
    main_compare_and_plot()
```

```

class SystemConfig:
    """Configuración unificada para Ising + Flujo de Ricci + Energía"""
    def __init__(self):
        # Parámetros físicos (Ising)
        self.N = 8                # número de espines
        self.J = 1.0              # acoplamiento ZZ
        self.h = 0.8              # campo transversal (Sx)
        self.periodic = False     # condiciones periódicas

        # Muestreo y proyección
        self.S = 500              # número de estados coherentes
        self.M_trunc = 50         # dimensión del subespacio proyectado

        # Evolución temporal (Chebyshev)
        self.T = 8.0              # tiempo final
        self.nt = 161             # número de puntos temporales
        self.t_list = np.linspace(0.0, self.T, self.nt)
        self.cheb_order = 120    # orden de Chebyshev

        # Flujo de Ricci
        self.BETA_MAX = 5.0       # máximo parámetro de escala
        self.n_beta = 15         # número de puntos en beta

        # Análisis de energía
        self.compute_energy_components = True # calcular componentes de energía
        self.plot_energy_analysis = True      # graficar análisis de energía

        # Visualización
        self.plot_dpi = 150
        self.colormap = 'viridis'
        self.save_figs = True
        self.plot_ricci_flow = True
        self.plot_comparison = True

np.random.seed(42)
config = SystemConfig()
plt.rcParams["figure.dpi"] = config.plot_dpi

# Operadores de Pauli
sx = np.array([[0.0, 1.0], [1.0, 0.0]], dtype=complex)
sy = np.array([[0.0, -1j], [1j, 0.0]], dtype=complex)
sz = np.array([[1.0, 0.0], [0.0, -1.0]], dtype=complex)
id2 = np.eye(2, dtype=complex)

def kron_list_sparse(mats):
    """Producto de Kronecker para matrices sparse"""
    out = mats[0]
    for A in mats[1:]:
        out = sp.kron(out, A, format='csr')
    return out

def kron_list_dense(mats):
    """Producto de Kronecker para matrices densas"""
    out = mats[0]
    for A in mats[1:]:
        out = np.kron(out, A)

```

```

    out = mats[0]
    for A in mats[1:]:
        out = np.kron(out, A)
    return out

def build_Sz_total_sparse(N):
    """Operador total de S^z"""
    dim = 2**N
    Sz_total = sp.csr_matrix((dim, dim), dtype=complex)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        Sz_total = Sz_total + kron_list_sparse(ops)
    return Sz_total

def build_Sx_total_sparse(N):
    """Operador total de S^x"""
    dim = 2**N
    Sx_total = sp.csr_matrix((dim, dim), dtype=complex)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sx)
        Sx_total = Sx_total + kron_list_sparse(ops)
    return Sx_total

def build_Ising_Hamiltonian_components(N, J=1.0, h=0.5, periodic=False):
    """
    Construye componentes del Hamiltoniano de Ising por separado
    Retorna: H_ZZ (interacción), H_X (campo), H_total
    """
    dim = 2**N
    H_ZZ = sp.csr_matrix((dim, dim), dtype=complex) # Parte de interacción
    H_X = sp.csr_matrix((dim, dim), dtype=complex)  # Parte del campo

    n_bonds = N if periodic else (N - 1)

    # Parte de interacción ZZ
    for i in range(n_bonds):
        j = (i + 1) % N
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        ops[j] = sp.csr_matrix(sz)
        H_ZZ = H_ZZ - J * kron_list_sparse(ops)

    # Parte del campo transversal
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sx)
        H_X = H_X - h * kron_list_sparse(ops)

    H_total = H_ZZ + H_X

    return H_ZZ, H_X, H_total

def build_local_energy_operators(N, J=1.0, h=0.5, periodic=False):
    """
    Construye operadores para energías locales
    Retorna: h_local_zz, h_local_x, h_local_total (lista de operadores locales)
    """

```

```

return H_ZZ, H_X, H_total

def build_local_energy_operators(N, J=1.0, h=0.5, periodic=False):
    """
    Construye operadores para energías locales
    Retorna: h_local_zz, h_local_x, h_local_total (lista de operadores locales)
    """
    h_local_zz = [] # Energía de interacción por enlace
    h_local_x = [] # Energía de campo por sitio
    h_local_total = [] # Hamiltoniano local por sitio

    n_bonds = N if periodic else (N - 1)

    # Operadores de energía de enlace (interacción ZZ)
    for i in range(n_bonds):
        j = (i + 1) % N
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sz)
        ops[j] = sp.csr_matrix(sz)
        h_bond = -J * kron_list_sparse(ops)
        h_local_zz.append(h_bond)

    # Operadores de energía de sitio (campo X)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sx)
        h_site = -h * kron_list_sparse(ops)
        h_local_x.append(h_site)

    # Hamiltoniano local por sitio (aproximación)
    for i in range(N):
        ops = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
        ops[i] = sp.csr_matrix(sx)
        h_local = -h * kron_list_sparse(ops)

        # Añadir contribuciones de enlace adyacentes
        if i < N-1 or periodic:
            j = (i + 1) % N
            ops_zz = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
            ops_zz[i] = sp.csr_matrix(sz)
            ops_zz[j] = sp.csr_matrix(sz)
            h_local = h_local - J * kron_list_sparse(ops_zz) / 2.0

        if i > 0 or periodic:
            j = (i - 1) % N
            ops_zz = [sp.eye(2, dtype=complex, format='csr') for _ in range(N)]
            ops_zz[i] = sp.csr_matrix(sz)
            ops_zz[j] = sp.csr_matrix(sz)
            h_local = h_local - J * kron_list_sparse(ops_zz) / 2.0

        h_local_total.append(h_local)

    return h_local_zz, h_local_x, h_local_total

def ising_hamiltonian_sparse(N, J=1.0, h=0.5, periodic=False, use_quimb=True):
    """Hamiltoniano de Ising sparse (versión original para compatibilidad)"""
    H_ZZ, H_X, H_total = build_Ising_Hamiltonian_components(N, J, h, periodic)
    return H_total

```

```

def ising_hamiltonian_sparse(N, J=1.0, h=0.5, periodic=False, use_quimb=True):
    """Hamiltoniano de Ising sparse (versión original para compatibilidad)"""
    H_ZZ, H_X, H_total = build_Ising_Hamiltonian_components(N, J, h, periodic)
    return H_total

class EnergyAnalyzer:
    """Analizador de energía para el sistema de Ising"""

    @staticmethod
    def compute_energy_evolution(states_list, H):
        """
        Calcula la energía total para cada estado en states_list
        states_list: array (nt x dim) o lista de estados
        H: Hamiltoniano (sparse o dense)
        Retorna: array de energías (nt,)
        """
        energies = np.zeros(len(states_list), dtype=float)

        for i, state in enumerate(states_list):
            if sp.issparse(H):
                energies[i] = np.real(np.vdot(state, H.dot(state)))
            else:
                energies[i] = np.real(np.vdot(state, H @ state))

        return energies

    @staticmethod
    def compute_energy_components_evolution(states_list, H_ZZ, H_X):
        """
        Calcula componentes de energía por separado
        Retorna: (energía_ZZ, energía_X, energía_total)
        """
        nt = len(states_list)
        E_ZZ = np.zeros(nt, dtype=float)
        E_X = np.zeros(nt, dtype=float)

        for i, state in enumerate(states_list):
            if sp.issparse(H_ZZ):
                E_ZZ[i] = np.real(np.vdot(state, H_ZZ.dot(state)))
            else:
                E_ZZ[i] = np.real(np.vdot(state, H_ZZ @ state))

            if sp.issparse(H_X):
                E_X[i] = np.real(np.vdot(state, H_X.dot(state)))
            else:
                E_X[i] = np.real(np.vdot(state, H_X @ state))

        E_total = E_ZZ + E_X
        return E_ZZ, E_X, E_total

    @staticmethod
    def compute_local_energy_density(states_list, h_local_list):
        """
        Calcula densidad de energía local (energía por sitio)

```

```

def compute_local_energy_density(states_list, h_local_list):
    """
    Calcula densidad de energía local (energía por sitio)
    h_local_list: lista de operadores locales
    Retorna: array (nt x N) con energía local por sitio
    """
    nt = len(states_list)
    N = len(h_local_list)
    local_energy = np.zeros((nt, N), dtype=float)

    for i, state in enumerate(states_list):
        for j, h_loc in enumerate(h_local_list):
            if sp.issparse(h_loc):
                local_energy[i, j] = np.real(np.vdot(state, h_loc.dot(state)))
            else:
                local_energy[i, j] = np.real(np.vdot(state, h_loc @ state))

    return local_energy

@staticmethod
def compute_energy_variance(states_list, H):
    """
    Calcula varianza de energía  $\Delta E^2 = \langle H^2 \rangle - \langle H \rangle^2$ 
    Retorna: varianza de energía para cada tiempo
    """
    nt = len(states_list)
    variance = np.zeros(nt, dtype=float)

    # Precalcular  $H^2$  si es posible
    if sp.issparse(H):
        H_squared = H.dot(H)
    else:
        H_squared = H @ H

    for i, state in enumerate(states_list):
        #  $\langle H \rangle$ 
        if sp.issparse(H):
            H_exp = np.real(np.vdot(state, H.dot(state)))
        else:
            H_exp = np.real(np.vdot(state, H @ state))

        #  $\langle H^2 \rangle$ 
        if sp.issparse(H_squared):
            H2_exp = np.real(np.vdot(state, H_squared.dot(state)))
        else:
            H2_exp = np.real(np.vdot(state, H_squared @ state))

        variance[i] = H2_exp - H_exp**2

    return variance

@staticmethod
def compute_energy_conservation_error(energies, initial_energy):
    """
    Calcula error en conservación de energía
    """

```

```

@staticmethod
def compute_energy_conservation_error(energies, initial_energy):
    """
    Calcula error en conservación de energía
    Retorna: error absoluto y relativo
    """
    abs_error = np.abs(energies - initial_energy)
    rel_error = abs_error / (np.abs(initial_energy) + 1e-12)
    return abs_error, rel_error

class EnergyVisualizer:
    """Visualizador para análisis de energía"""

    @staticmethod
    def plot_energy_evolution(t_list, energies_full, energies_proj=None, title="Evolución de Energía"):
        """
        Grafica energía vs tiempo para evolución completa y proyectada
        """
        plt.figure(figsize=(12, 8))

        # Subplot 1: Energía total
        plt.subplot(2, 2, 1)
        plt.plot(t_list, energies_full, 'b-', linewidth=2, label='Completo')
        if energies_proj is not None:
            plt.plot(t_list, energies_proj, 'r--', linewidth=2, label='Proyectado')
        plt.xlabel('Tiempo')
        plt.ylabel('Energía total')
        plt.title(f'{title} - Energía Total')
        plt.legend()
        plt.grid(True, alpha=0.3)

        # Subplot 2: Error en energía
        if energies_proj is not None:
            energy_error = np.abs(energies_full - energies_proj)
            plt.subplot(2, 2, 2)
            plt.plot(t_list, energy_error, 'g-', linewidth=2)
            plt.xlabel('Tiempo')
            plt.ylabel('|ΔE|')
            plt.title('Error en Energía Total')
            plt.grid(True, alpha=0.3)
            plt.yscale('log')

        # Subplot 3: Conservación de energía (error relativo)
        if energies_proj is not None:
            initial_energy_full = energies_full[0]
            initial_energy_proj = energies_proj[0] if energies_proj[0] is not None else energies_proj[1]

            abs_error_full, rel_error_full = EnergyAnalyzer.compute_energy_conservation_error(
                energies_full, initial_energy_full)
            abs_error_proj, rel_error_proj = EnergyAnalyzer.compute_energy_conservation_error(
                energies_proj, initial_energy_proj)

            plt.subplot(2, 2, 3)
            plt.plot(t_list, rel_error_full, 'b-', linewidth=2, label='Completo')
            plt.plot(t_list, rel_error_proj, 'r--', linewidth=2, label='Proyectado')

```

```

plt.subplot(2, 2, 3)
plt.plot(t_list, rel_error_full, 'b-', linewidth=2, label='Completo')
plt.plot(t_list, rel_error_proj, 'r--', linewidth=2, label='Proyectado')
plt.xlabel('Tiempo')
plt.ylabel('Error relativo')
plt.title('Error de Conservación de Energía')
plt.legend()
plt.grid(True, alpha=0.3)
plt.yscale('log')

# Subplot 4: Varianza de energía
plt.subplot(2, 2, 4)
# Esta será calculada después y añadida
plt.text(0.5, 0.5, 'Varianza de energía\n(se calculará después)',
        horizontalalignment='center', verticalalignment='center',
        transform=plt.gca().transAxes, fontsize=12)
plt.axis('off')

plt.tight_layout()
if config.save_figs:
    plt.savefig(f'{title.lower().replace(" ", "_")}_evolution.png',
                dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

@staticmethod
def plot_energy_components(t_list, E_ZZ, E_X, E_total, title="Componentes de Energía"):
    """
    Grafica componentes de energía por separado
    """
    plt.figure(figsize=(14, 10))

    # Subplot 1: Todas Las componentes
    plt.subplot(2, 2, 1)
    plt.plot(t_list, E_total, 'k-', linewidth=3, label='Total')
    plt.plot(t_list, E_ZZ, 'b-', linewidth=2, label='Interacción ZZ')
    plt.plot(t_list, E_X, 'r-', linewidth=2, label='Campo X')
    plt.xlabel('Tiempo')
    plt.ylabel('Energía')
    plt.title(f'{title} - Componentes')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Subplot 2: Proporciones relativas
    plt.subplot(2, 2, 2)
    with np.errstate(divide='ignore', invalid='ignore'):
        ratio_ZZ = E_ZZ / (np.abs(E_total) + 1e-12)
        ratio_X = E_X / (np.abs(E_total) + 1e-12)

    plt.plot(t_list, ratio_ZZ, 'b-', linewidth=2, label='Fracción ZZ')
    plt.plot(t_list, ratio_X, 'r-', linewidth=2, label='Fracción X')
    plt.xlabel('Tiempo')
    plt.ylabel('Fracción de energía total')
    plt.title('Proporción de Componentes')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.ylim(-1.5, 1.5)

    # Subplot 3: Derivadas (potencia)
    dt = t_list[1] - t_list[0]
    dE_dt = np.gradient(E_total, dt)
    dE_ZZ_dt = np.gradient(E_ZZ, dt)
    dE_X_dt = np.gradient(E_X, dt)

    plt.subplot(2, 2, 3)
    plt.plot(t_list, dE_dt, 'k-', linewidth=2, label='dE/dt total')
    plt.plot(t_list, dE_ZZ_dt, 'b--', linewidth=1.5, label='dE_ZZ/dt')
    plt.plot(t_list, dE_X_dt, 'r--', linewidth=1.5, label='dE_X/dt')
    plt.xlabel('Tiempo')
    plt.ylabel('dE/dt')
    plt.title('Derivada Temporal de Energía (Potencia)')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Subplot 4: Suma de componentes vs total (verificación)
    E_sum = E_ZZ + E_X
    diff = np.abs(E_sum - E_total)
    plt.plot(t_list, diff, 'g-', linewidth=2)
    plt.xlabel('Tiempo')
    plt.ylabel('|(E_ZZ+E_X) - E_total|')
    plt.title('Error en Suma de Componentes')
    plt.grid(True, alpha=0.3)
    plt.yscale('log')

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_components.png',
                    dpi=config.plot_dpi, bbox_inches='tight')
    plt.show()

@staticmethod
def plot_local_energy_density(t_list, local_energy, title="Densidad de Energía Local"):
    """
    Grafica mapa de calor de energía local por sitio
    """
    plt.figure(figsize=(12, 8))

    # Mapa de calor
    plt.imshow(local_energy.T, aspect='auto', origin='lower',
               extent=[t_list[0], t_list[-1], 0, local_energy.shape[1]],
               cmap='RdBu_r', interpolation='bilinear')
    plt.colorbar(label='Energía local')
    plt.xlabel('Tiempo')
    plt.ylabel('Sitio')
    plt.title(f'{title} - Mapa de Calor')

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_heatmap.png',
                    dpi=config.plot_dpi, bbox_inches='tight')
    plt.show()

# Perfiles en tiempos seleccionados
selected_times = [0, len(t_list)//4, len(t_list)//2, 3*len(t_list)//4, -1]

```

```

plt.tight_layout()

# Subplot 3: Derivadas (potencia)
dt = t_list[1] - t_list[0]
dE_dt = np.gradient(E_total, dt)
dE_ZZ_dt = np.gradient(E_ZZ, dt)
dE_X_dt = np.gradient(E_X, dt)

plt.subplot(2, 2, 3)
plt.plot(t_list, dE_dt, 'k-', linewidth=2, label='dE/dt total')
plt.plot(t_list, dE_ZZ_dt, 'b--', linewidth=1.5, label='dE_ZZ/dt')
plt.plot(t_list, dE_X_dt, 'r--', linewidth=1.5, label='dE_X/dt')
plt.xlabel('Tiempo')
plt.ylabel('dE/dt')
plt.title('Derivada Temporal de Energía (Potencia)')
plt.legend()
plt.grid(True, alpha=0.3)

# Subplot 4: Suma de componentes vs total (verificación)
plt.subplot(2, 2, 4)
E_sum = E_ZZ + E_X
diff = np.abs(E_sum - E_total)
plt.plot(t_list, diff, 'g-', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('|(E_ZZ+E_X) - E_total|')
plt.title('Error en Suma de Componentes')
plt.grid(True, alpha=0.3)
plt.yscale('log')

plt.tight_layout()
if config.save_figs:
    plt.savefig(f'{title.lower().replace(" ", "_")}_components.png',
                dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

@staticmethod
def plot_local_energy_density(t_list, local_energy, title="Densidad de Energía Local"):
    """
    Grafica mapa de calor de energía local por sitio
    """
    plt.figure(figsize=(12, 8))

    # Mapa de calor
    plt.imshow(local_energy.T, aspect='auto', origin='lower',
               extent=[t_list[0], t_list[-1], 0, local_energy.shape[1]],
               cmap='RdBu_r', interpolation='bilinear')
    plt.colorbar(label='Energía local')
    plt.xlabel('Tiempo')
    plt.ylabel('Sitio')
    plt.title(f'{title} - Mapa de Calor')

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_heatmap.png',
                    dpi=config.plot_dpi, bbox_inches='tight')
    plt.show()

# Perfiles en tiempos seleccionados
selected_times = [0, len(t_list)//4, len(t_list)//2, 3*len(t_list)//4, -1]

```

```

# Perfiles en tiempos seleccionados
selected_times = [0, len(t_list)//4, len(t_list)//2, 3*len(t_list)//4, -1]

fig, axes = plt.subplots(1, len(selected_times), figsize=(15, 5))
if len(selected_times) == 1:
    axes = [axes]

for idx, t_idx in enumerate(selected_times):
    axes[idx].bar(range(local_energy.shape[1]), local_energy[t_idx, :])
    axes[idx].set_xlabel('Sitio')
    axes[idx].set_ylabel('Energía local')
    axes[idx].set_title(f't = {t_list[t_idx]:.2f}')
    axes[idx].grid(True, alpha=0.3)

plt.tight_layout()
if config.save_figs:
    plt.savefig(f'{title.lower().replace(" ", "_")}_profiles.png',
                dpi=config.plot_dpi, bbox_inches='tight')

plt.show()

@staticmethod
def plot_energy_variance(t_list, variance_full, variance_proj=None, title="Varianza de Energía"):
    """
    Grafica varianza de energía vs tiempo
    """
    plt.figure(figsize=(12, 5))

    plt.subplot(1, 2, 1)
    plt.plot(t_list, variance_full, 'b-', linewidth=2, label='Completo')
    if variance_proj is not None:
        plt.plot(t_list, variance_proj, 'r--', linewidth=2, label='Proyectado')
    plt.xlabel('Tiempo')
    plt.ylabel('ΔE²')
    plt.title(f'{title} - Varianza')
    plt.legend()
    plt.grid(True, alpha=0.3)

    if variance_proj is not None:
        plt.subplot(1, 2, 2)
        variance_ratio = variance_proj / (variance_full + 1e-12)
        plt.plot(t_list, variance_ratio, 'g-', linewidth=2)
        plt.axhline(y=1, color='k', linestyle='--', alpha=0.5)
        plt.xlabel('Tiempo')
        plt.ylabel('Varianza(proj) / Varianza(full)')
        plt.title('Ratio de Varianzas')
        plt.grid(True, alpha=0.3)
        plt.yscale('log')

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_variance.png',
                    dpi=config.plot_dpi, bbox_inches='tight')

    plt.show()

@staticmethod
def plot_energy_summary(t_list, energies_full, energies_proj,
                        E_ZZ_full, E_X_full, E_ZZ_proj=None, E_X_proj=None,
                        title="Resumen de Energía"):
    """
    Gráfica de resumen completo de energía
    """
    fig = plt.figure(figsize=(16, 12))

    # 1. Energía total comparativa
    ax1 = plt.subplot(3, 3, 1)
    ax1.plot(t_list, energies_full, 'b-', linewidth=2, label='Completo')
    if energies_proj is not None:
        ax1.plot(t_list, energies_proj, 'r--', linewidth=2, label='Proyectado')
    ax1.set_xlabel('Tiempo')
    ax1.set_ylabel('Energía total')
    ax1.set_title('Energía Total')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # 2. Error en energía total
    ax2 = plt.subplot(3, 3, 2)
    if energies_proj is not None:
        energy_error = np.abs(energies_full - energies_proj)
        ax2.plot(t_list, energy_error, 'g-', linewidth=2)
        ax2.set_yscale('log')
    ax2.set_xlabel('Tiempo')
    ax2.set_ylabel('|ΔE|')
    ax2.set_title('Error en Energía Total')
    ax2.grid(True, alpha=0.3)

    # 3. Componentes de energía (full)
    ax3 = plt.subplot(3, 3, 3)
    ax3.plot(t_list, E_ZZ_full, 'b-', linewidth=2, label='E_ZZ')
    ax3.plot(t_list, E_X_full, 'r-', linewidth=2, label='E_X')
    ax3.set_xlabel('Tiempo')
    ax3.set_ylabel('Energía')
    ax3.set_title('Componentes (Completo)')
    ax3.legend()
    ax3.grid(True, alpha=0.3)

    # 4. Componentes de energía (proj) si existen
    ax4 = plt.subplot(3, 3, 4)
    if E_ZZ_proj is not None and E_X_proj is not None:
        ax4.plot(t_list, E_ZZ_proj, 'b--', linewidth=2, label='E_ZZ (proj)')
        ax4.plot(t_list, E_X_proj, 'r--', linewidth=2, label='E_X (proj)')
    ax4.set_xlabel('Tiempo')
    ax4.set_ylabel('Energía')
    ax4.set_title('Componentes (Proyectado)')
    if E_ZZ_proj is not None:
        ax4.legend()
    ax4.grid(True, alpha=0.3)

    # 5. Error en componentes
    ax5 = plt.subplot(3, 3, 5)
    if E_ZZ_proj is not None:
        error_ZZ = np.abs(E_ZZ_full - E_ZZ_proj)
        error_X = np.abs(E_X_full - E_X_proj)
        ax5.plot(t_list, error_ZZ, 'g-', linewidth=2, label='Error E_ZZ')
        ax5.plot(t_list, error_X, 'g-', linewidth=2, label='Error E_X')
        ax5.set_xlabel('Tiempo')
        ax5.set_ylabel('Error')
        ax5.set_title('Error en Componentes')
        ax5.legend()
        ax5.grid(True, alpha=0.3)

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_summary.png',
                    dpi=config.plot_dpi, bbox_inches='tight')

    plt.show()

```

```

@staticmethod
def plot_energy_summary(t_list, energies_full, energies_proj,
                        E_ZZ_full, E_X_full, E_ZZ_proj=None, E_X_proj=None,
                        title="Resumen de Energía"):
    """
    Gráfica de resumen completo de energía
    """
    fig = plt.figure(figsize=(16, 12))

    # 1. Energía total comparativa
    ax1 = plt.subplot(3, 3, 1)
    ax1.plot(t_list, energies_full, 'b-', linewidth=2, label='Completo')
    if energies_proj is not None:
        ax1.plot(t_list, energies_proj, 'r--', linewidth=2, label='Proyectado')
    ax1.set_xlabel('Tiempo')
    ax1.set_ylabel('Energía total')
    ax1.set_title('Energía Total')
    ax1.legend()
    ax1.grid(True, alpha=0.3)

    # 2. Error en energía total
    ax2 = plt.subplot(3, 3, 2)
    if energies_proj is not None:
        energy_error = np.abs(energies_full - energies_proj)
        ax2.plot(t_list, energy_error, 'g-', linewidth=2)
        ax2.set_yscale('log')
    ax2.set_xlabel('Tiempo')
    ax2.set_ylabel('|ΔE|')
    ax2.set_title('Error en Energía Total')
    ax2.grid(True, alpha=0.3)

    # 3. Componentes de energía (full)
    ax3 = plt.subplot(3, 3, 3)
    ax3.plot(t_list, E_ZZ_full, 'b-', linewidth=2, label='E_ZZ')
    ax3.plot(t_list, E_X_full, 'r-', linewidth=2, label='E_X')
    ax3.set_xlabel('Tiempo')
    ax3.set_ylabel('Energía')
    ax3.set_title('Componentes (Completo)')
    ax3.legend()
    ax3.grid(True, alpha=0.3)

    # 4. Componentes de energía (proj) si existen
    ax4 = plt.subplot(3, 3, 4)
    if E_ZZ_proj is not None and E_X_proj is not None:
        ax4.plot(t_list, E_ZZ_proj, 'b--', linewidth=2, label='E_ZZ (proj)')
        ax4.plot(t_list, E_X_proj, 'r--', linewidth=2, label='E_X (proj)')
    ax4.set_xlabel('Tiempo')
    ax4.set_ylabel('Energía')
    ax4.set_title('Componentes (Proyectado)')
    if E_ZZ_proj is not None:
        ax4.legend()
    ax4.grid(True, alpha=0.3)

    # 5. Error en componentes
    ax5 = plt.subplot(3, 3, 5)
    if E_ZZ_proj is not None:
        error_ZZ = np.abs(E_ZZ_full - E_ZZ_proj)
        error_X = np.abs(E_X_full - E_X_proj)
        ax5.plot(t_list, error_ZZ, 'g-', linewidth=2, label='Error E_ZZ')
        ax5.plot(t_list, error_X, 'g-', linewidth=2, label='Error E_X')
        ax5.set_xlabel('Tiempo')
        ax5.set_ylabel('Error')
        ax5.set_title('Error en Componentes')
        ax5.legend()
        ax5.grid(True, alpha=0.3)

    plt.tight_layout()
    if config.save_figs:
        plt.savefig(f'{title.lower().replace(" ", "_")}_summary.png',
                    dpi=config.plot_dpi, bbox_inches='tight')

    plt.show()

```

```

# 5. Error en componentes
ax5 = plt.subplot(3, 3, 5)
if E_ZZ_proj is not None:
    error_ZZ = np.abs(E_ZZ_full - E_ZZ_proj)
    error_X = np.abs(E_X_full - E_X_proj)
    ax5.plot(t_list, error_ZZ, 'b-', linewidth=2, label='Error E_ZZ')
    ax5.plot(t_list, error_X, 'r-', linewidth=2, label='Error E_X')
    ax5.set_yscale('log')
    ax5.legend()
ax5.set_xlabel('Tiempo')
ax5.set_ylabel('|\Delta E|')
ax5.set_title('Error en Componentes')
ax5.grid(True, alpha=0.3)

# 6. Proporciones de energía
ax6 = plt.subplot(3, 3, 6)
with np.errstate(divide='ignore', invalid='ignore'):
    ratio_ZZ_full = E_ZZ_full / (np.abs(energies_full) + 1e-12)
    ratio_X_full = E_X_full / (np.abs(energies_full) + 1e-12)

ax6.plot(t_list, ratio_ZZ_full, 'b-', linewidth=2, label='ZZ/Total')
ax6.plot(t_list, ratio_X_full, 'r-', linewidth=2, label='X/Total')
ax6.set_xlabel('Tiempo')
ax6.set_ylabel('Fracción')
ax6.set_title('Proporción de Energía (Completo)')
ax6.legend()
ax6.grid(True, alpha=0.3)
ax6.set_ylim(-2, 2)

# 7. Conservación de energía (error relativo)
ax7 = plt.subplot(3, 3, 7)
if energies_proj is not None:
    initial_energy_full = energies_full[0]
    initial_energy_proj = energies_proj[0]

    _, rel_error_full = EnergyAnalyzer.compute_energy_conservation_error(
        energies_full, initial_energy_full)
    _, rel_error_proj = EnergyAnalyzer.compute_energy_conservation_error(
        energies_proj, initial_energy_proj)

    ax7.plot(t_list, rel_error_full, 'b-', linewidth=2, label='Completo')
    ax7.plot(t_list, rel_error_proj, 'r--', linewidth=2, label='Proyectado')
    ax7.set_yscale('log')
    ax7.legend()
ax7.set_xlabel('Tiempo')
ax7.set_ylabel('Error relativo')
ax7.set_title('Error de Conservación')
ax7.grid(True, alpha=0.3)

# 8. Energía por sitio promedio
ax8 = plt.subplot(3, 3, 8)
energy_per_site_full = energies_full / config.N
ax8.plot(t_list, energy_per_site_full, 'b-', linewidth=2, label='Completo')
if energies_proj is not None:
    energy_per_site_proj = energies_proj / config.N
    ax8.plot(t_list, energy_per_site_proj, 'r--', linewidth=2, label='Proyectado')
ax8.set_xlabel('Tiempo')

```

```

energy_per_site_proj = energies_proj / config.N
ax8.plot(t_list, energy_per_site_proj, 'r--', linewidth=2, label='Proyectado')
ax8.set_xlabel('Tiempo')
ax8.set_ylabel('Energía por sitio')
ax8.set_title('Energía por Sitio')
ax8.legend()
ax8.grid(True, alpha=0.3)

# 9. Espectro de Fourier de La energía
ax9 = plt.subplot(3, 3, 9)
# Transformada de Fourier de La energía
dt = t_list[1] - t_list[0]
freqs = np.fft.fftfreq(len(t_list), dt)
fft_energy = np.fft.fft(energies_full - np.mean(energies_full))
power_spectrum = np.abs(fft_energy)**2

# Solo frecuencias positivas
pos_mask = freqs > 0
ax9.plot(freqs[pos_mask], power_spectrum[pos_mask], 'b-', linewidth=2)
ax9.set_xlabel('Frecuencia')
ax9.set_ylabel('Potencia')
ax9.set_title('Espectro de Fourier de Energía')
ax9.set_xscale('log')
ax9.set_yscale('log')
ax9.grid(True, alpha=0.3)

plt.suptitle(title, fontsize=16, y=1.02)
plt.tight_layout()
if config.save_figs:
    plt.savefig(f'{title.lower().replace(" ", "_")}_summary.png',
                dpi=config.plot_dpi, bbox_inches='tight')

plt.show()

```

```

class RicciFlowOptimizer:
    """Optimiza la proyección usando flujo de Ricci"""

    def __init__(self, config):
        self.config = config

    def get_thermal_weights(self, H, Data, beta):
        """Calcula pesos térmicos para la proyección ponderada"""
        if beta < 1e-8:
            return np.ones(Data.shape[1], dtype=float) / Data.shape[1]

        H_dense = H.todense() if sp.issparse(H) else H
        P_op = expm(-beta * H_dense)

        weights = np.zeros(Data.shape[1], dtype=float)
        for k in range(Data.shape[1]):
            Zk = Data[:, k]
            weights[k] = np.real(Zk.conj().T @ (P_op @ Zk))

        weights = np.maximum(weights, 1e-12)
        weights /= np.sum(weights)

        return weights

    def build_weighted_projection(self, Data, M, weights, regularization=1e-8):

```

```

return weights

def build_weighted_projection(self, Data, M, weights, regularization=1e-8):
    """Proyección ponderada con regularización"""
    sqrt_weights = np.sqrt(weights)
    D_tilde = Data * sqrt_weights[np.newaxis, :]

    G_tilde = D_tilde.conj().T @ D_tilde
    G_tilde += regularization * np.eye(G_tilde.shape[0])

    evals, evecs = eigh(G_tilde)
    idx = np.argsort(evals)[::-1]
    evals, evecs = evals[idx], evecs[:, idx]

    M_eff = min(M, len(evals))

    P = np.zeros((Data.shape[0], M_eff), dtype=complex)
    for m in range(M_eff):
        v = evecs[:, m]
        lam = evals[m]
        P[:, m] = (D_tilde @ v) / np.sqrt(lam)

    return P, evals, M_eff

def calculate_fidelity(self, P, psi0):
    """Calcula fidelidad de la proyección"""
    psi0_proj = P.conj().T @ psi0
    psi0_reconstructed = P @ psi0_proj
    overlap = np.vdot(psi0_reconstructed, psi0)
    return np.abs(overlap)**2

def optimize_projection(self, H, psi0, Data, beta_list=None):
    """Encuentra la proyección óptima variando beta"""
    if beta_list is None:
        beta_list = np.linspace(0.01, self.config.BETA_MAX, self.config.n_beta)

    fidelities = []
    projections = []
    weights_list = []

    print("Optimizando proyección con flujo de Ricci...")
    for beta in tqdm(beta_list, desc="β"):
        weights = self.get_thermal_weights(H, Data, beta)
        weights_list.append(weights)

        P, evals, M_eff = self.build_weighted_projection(Data, self.config.M_trunc, weights)
        projections.append(P)

        fid = self.calculate_fidelity(P, psi0)
        fidelities.append(fid)

    best_idx = np.argmax(fidelities)
    best_beta = beta_list[best_idx]
    best_P = projections[best_idx]
    best_fidelity = fidelities[best_idx]

    HP = H.dot(best_P) if sp.issparse(H) else H @ best_P
    H_eff = best_P.conj().T @ HP

```

```

HP = H.dot(best_P) if sp.issparse(H) else H @ best_P
H_eff = best_P.conj().T @ HP

```

```

results = {
    'beta_list': beta_list,
    'fidelities': np.array(fidelities),
    'weights_list': weights_list,
    'best_beta': best_beta,
    'best_fidelity': best_fidelity,
    'best_P': best_P,
    'best_H_eff': H_eff,
    'best_idx': best_idx
}

return results

```

```
def main_with_energy_analysis():
```

```

# 1. Inicialización
N = config.N
dim = 2**N
print(f"N = {N} → dim = {dim}")

# 2. Construir Hamiltoniano y componentes
print("Construyendo Hamiltoniano y componentes de energía...")
H_ZZ, H_X, H_total = build_Ising_Hamiltonian_components(N, config.J, config.h, config.periodic)
h_local_zz, h_local_x, h_local_total = build_local_energy_operators(N, config.J, config.h, config.periodic)

# 3. Estado inicial
psi0 = np.zeros(dim, dtype=complex)
psi0[0] = 1.0 # |00...0>

# 4. Muestreo de estados coherentes
print("Muestreando estados coherentes...")
# Para mantener compatibilidad con el flujo de Ricci
from scipy.linalg import eigh as scipy_eigh

# Función de muestreo rápida
def sample_cp1_fibonacci(S):
    indices = np.arange(0, S, dtype=float)
    phi = 2 * np.pi * indices * (1.0 / ((1.0 + np.sqrt(5.0)) / 2.0))
    theta = np.arccos(1.0 - 2.0 * (indices + 0.5) / S)
    z = np.tan(theta / 2.0) * np.exp(1j * phi)
    return z, theta, phi

# Construir estados coherentes
def build_product_coherent_vectors(N, z_list):
    S = len(z_list)
    dim = 2**N
    Data = np.zeros((dim, S), dtype=complex)
    for j, z in enumerate(z_list):
        single = np.array([1.0, z], dtype=complex) / np.sqrt(1.0 + np.abs(z)**2)
        psi = single
        for _ in range(1, N):
            psi = np.kron(psi, single)

```

```

psi = single
for _ in range(1, N):
    psi = np.kron(psi, single)
Data[:, j] = psi
return Data

z_list, theta, phi = sample_cp1_fibonacci(config.S)
Data = build_product_coherent_vectors(N, z_list)

# 5. Optimización con Flujo de Ricci
print("\n=== OPTIMIZACIÓN CON FLUJO DE RICCI ===")
ricci_optimizer = RicciFlowOptimizer(config)

beta_list = np.linspace(0.01, config.BETA_MAX, config.n_beta)
ricci_results = ricci_optimizer.optimize_projection(H_total, psi0, Data, beta_list)

print(f"\nMejor  $\beta$  encontrado: {ricci_results['best_beta']:.4f}")
print(f"Fidelidad óptima: {ricci_results['best_fidelity']:.6f}")
print(f"Dimensión efectiva: {ricci_results['best_P'].shape[1]}/{dim} "
      f"({100.0*ricci_results['best_P'].shape[1]/dim:.2f}%)")

# 6. Evolución temporal con Chebyshev
print("\n=== EVOLUCIÓN TEMPORAL CON CHEBYSHEV ===")

# Clase ChebyshevPropagator (del script original)
class ChebyshevPropagator:
    def __init__(self, H, M=80):
        self.H = H if sp.issparse(H) else sp.csr_matrix(H)
        self.M = int(M)
        self._prepare_rescaled_hamiltonian()

    def _prepare_rescaled_hamiltonian(self):
        k = 2
        try:
            emin_vals = spla.eigsh(self.H, k=k, which='SA', return_eigenvalues=False)
            emax_vals = spla.eigsh(self.H, k=k, which='LA', return_eigenvalues=False)
            emin = float(np.min(emin_vals))
            emax = float(np.max(emax_vals))
        except Exception:
            H_arr = self.H.toarray()
            evals = np.linalg.eigvalsh(H_arr)
            emin, emax = float(evals.min()), float(evals.max())

        if emax <= emin:
            emax = emin + 1e-10

        self.emin = emin
        self.emax = emax
        self.a = (emax - emin) / 2.0
        self.b = (emax + emin) / 2.0

        I = sp.eye(self.H.shape[0], format='csr')
        self.H_rescaled = (self.H - self.b * I) * (1.0 / self.a)

    def evolve(self, psi0, t):
        M = self.M
        coeff = np.empty(M + 1, dtype=complex)
        z = self.a * t
        pref = np.exp(-1j * self.b * t)
        coeff[0] = pref * special.jv(0, z)

        for n in range(1, M + 1):
            coeff[n] = 2.0 * pref * special.jv(n, z)

        T0 = psi0
        T1 = self.H_rescaled.dot(psi0)
        psi_t = coeff[0] * T0 + coeff[1] * T1

        for n in range(2, M + 1):
            Tn = 2.0 * self.H_rescaled.dot(T1) - T0
            psi_t = psi_t + coeff[n] * Tn
            T0, T1 = T1, Tn

        return psi_t

# Función de evolución con proyección
def project_and_evolve_chebyshev(H, P, psi0, t_list, cheb_order=80):
    HP = H.dot(P) if sp.issparse(H) else H @ P
    H_eff = P.conj().T @ HP

    psi0_proj = P.conj().T @ psi0
    nrm = np.linalg.norm(psi0_proj)
    if nrm > 0:
        psi0_proj = psi0_proj / nrm

    prop_eff = ChebyshevPropagator(sp.csr_matrix(H_eff), M=cheb_order)
    prop_full = ChebyshevPropagator(H if sp.issparse(H) else sp.csr_matrix(H), M=cheb_order)

    states_proj_sub = []
    states_full = []

    for t in tqdm(t_list, desc="Evolución temporal"):
        psi_proj_t = prop_eff.evolve(psi0_proj, t)
        states_proj_sub.append(psi_proj_t.copy())
        psi_full_t = prop_full.evolve(psi0, t)
        states_full.append(psi_full_t.copy())

    states_proj_sub = np.array(states_proj_sub)
    states_full = np.array(states_full)
    states_proj_lifted = np.array([P @ v for v in states_proj_sub])

    return states_proj_lifted, states_full, H_eff, psi0_proj

# Evolución con la proyección óptima
P_optimal = ricci_results['best_P']
print("Evolucionando con Chebyshev...")
states_proj, states_full, H_eff, psi0_proj = project_and_evolve_chebyshev(
    H_total, P_optimal, psi0, config.t_list, cheb_order=config.cheb_order
)

```

```

def evolve(self, psi0, t):
    M = self.M
    coeff = np.empty(M + 1, dtype=complex)
    z = self.a * t
    pref = np.exp(-1j * self.b * t)
    coeff[0] = pref * special.jv(0, z)

    for n in range(1, M + 1):
        coeff[n] = 2.0 * pref * special.jv(n, z)

    T0 = psi0
    T1 = self.H_rescaled.dot(psi0)
    psi_t = coeff[0] * T0 + coeff[1] * T1

    for n in range(2, M + 1):
        Tn = 2.0 * self.H_rescaled.dot(T1) - T0
        psi_t = psi_t + coeff[n] * Tn
        T0, T1 = T1, Tn

    return psi_t

# Función de evolución con proyección
def project_and_evolve_chebyshev(H, P, psi0, t_list, cheb_order=80):
    HP = H.dot(P) if sp.issparse(H) else H @ P
    H_eff = P.conj().T @ HP

    psi0_proj = P.conj().T @ psi0
    nrm = np.linalg.norm(psi0_proj)
    if nrm > 0:
        psi0_proj = psi0_proj / nrm

    prop_eff = ChebyshevPropagator(sp.csr_matrix(H_eff), M=cheb_order)
    prop_full = ChebyshevPropagator(H if sp.issparse(H) else sp.csr_matrix(H), M=cheb_order)

    states_proj_sub = []
    states_full = []

    for t in tqdm(t_list, desc="Evolución temporal"):
        psi_proj_t = prop_eff.evolve(psi0_proj, t)
        states_proj_sub.append(psi_proj_t.copy())
        psi_full_t = prop_full.evolve(psi0, t)
        states_full.append(psi_full_t.copy())

    states_proj_sub = np.array(states_proj_sub)
    states_full = np.array(states_full)
    states_proj_lifted = np.array([P @ v for v in states_proj_sub])

    return states_proj_lifted, states_full, H_eff, psi0_proj

# Evolución con la proyección óptima
P_optimal = ricci_results['best_P']
print("Evolucionando con Chebyshev...")
states_proj, states_full, H_eff, psi0_proj = project_and_evolve_chebyshev(
    H_total, P_optimal, psi0, config.t_list, cheb_order=config.cheb_order
)

```

```

)

# 7. ANÁLISIS DE ENERGÍA
print("\n=== ANÁLISIS COMPLETO DE ENERGÍA ===")

# 7.1 Energía total
print("Calculando energía total...")
energies_full = EnergyAnalyzer.compute_energy_evolution(states_full, H_total)
energies_proj = EnergyAnalyzer.compute_energy_evolution(states_proj, H_total)

# 7.2 Componentes de energía
print("Calculando componentes de energía...")
E_ZZ_full, E_X_full, E_total_full = EnergyAnalyzer.compute_energy_components_evolution(
    states_full, H_ZZ, H_X)
E_ZZ_proj, E_X_proj, E_total_proj = EnergyAnalyzer.compute_energy_components_evolution(
    states_proj, H_ZZ, H_X)

# 7.3 Densidad de energía local
print("Calculando densidad de energía local...")
local_energy_full = EnergyAnalyzer.compute_local_energy_density(states_full, h_local_total)
local_energy_proj = EnergyAnalyzer.compute_local_energy_density(states_proj, h_local_total)

# 7.4 Varianza de energía
print("Calculando varianza de energía...")
variance_full = EnergyAnalyzer.compute_energy_variance(states_full, H_total)
variance_proj = EnergyAnalyzer.compute_energy_variance(states_proj, H_total)

# 8. VISUALIZACIONES DE ENERGÍA
if config.plot_energy_analysis:
    print("\n=== GENERANDO GRÁFICAS DE ENERGÍA ===")

    # 8.1 Evolución de energía total
    EnergyVisualizer.plot_energy_evolution(
        config.t_list, energies_full, energies_proj,
        title="Evolución de Energía Total"
    )

    # 8.2 Componentes de energía
    EnergyVisualizer.plot_energy_components(
        config.t_list, E_ZZ_full, E_X_full, energies_full,
        title="Componentes de Energía (Completo)"
    )

    if E_ZZ_proj is not None:
        EnergyVisualizer.plot_energy_components(
            config.t_list, E_ZZ_proj, E_X_proj, energies_proj,
            title="Componentes de Energía (Proyectado)"
        )

    # 8.3 Densidad de energía local
    EnergyVisualizer.plot_local_energy_density(
        config.t_list, local_energy_full,
        title="Densidad de Energía Local (Completo)"
    )

    EnergyVisualizer.plot_local_energy_density(
        config.t_list, local_energy_proj,
        title="Densidad de Energía Local (Proyectado)"
    )

```

```

EnergyVisualizer.plot_local_energy_density(
    config.t_list, local_energy_proj,
    title="Densidad de Energía Local (Proyectado)"
)

# 8.4 Varianza de energía
EnergyVisualizer.plot_energy_variance(
    config.t_list, variance_full, variance_proj,
    title="Varianza de Energía"
)

# 8.5 Resumen completo de energía
EnergyVisualizer.plot_energy_summary(
    config.t_list, energies_full, energies_proj,
    E_ZZ_full, E_X_full, E_ZZ_proj, E_X_proj,
    title="Resumen Completo de Energía"
)

print("\n Energía")

# Error en energía
energy_error = np.abs(energies_full - energies_proj)
max_energy_error = np.max(energy_error)
mean_energy_error = np.mean(energy_error)

# Error relativo en energía
energy_rel_error = energy_error / (np.abs(energies_full) + 1e-12)
max_energy_rel_error = np.max(energy_rel_error)

# Conservación de energía
initial_energy_full = energies_full[0]
initial_energy_proj = energies_proj[0]

abs_error_full, rel_error_full = EnergyAnalyzer.compute_energy_conservation_error(
    energies_full, initial_energy_full)
abs_error_proj, rel_error_proj = EnergyAnalyzer.compute_energy_conservation_error(
    energies_proj, initial_energy_proj)

max_conservation_error_full = np.max(abs_error_full)
max_conservation_error_proj = np.max(abs_error_proj)

print(f"Energía inicial (completo): {initial_energy_full:.6f}")
print(f"Energía inicial (proyectado): {initial_energy_proj:.6f}")
print(f"Error máximo en energía: {max_energy_error:.6e}")
print(f"Error medio en energía: {mean_energy_error:.6e}")
print(f"Error relativo máximo en energía: {max_energy_rel_error:.6e}")
print(f"Error máximo de conservación (completo): {max_conservation_error_full:.6e}")
print(f"Error máximo de conservación (proyectado): {max_conservation_error_proj:.6e}")

# 10. ANÁLISIS ADICIONAL
print("\n=== ANÁLISIS ADICIONAL ===")

# Correlación entre energía y magnetización
Sz_total = build_Sz_total_sparse(N)

def compute_expectation_series(states, operator):
    if sp.issparse(operator):
        return np.array([np.vdot(s, operator.dot(s)).real for s in states])

```

```

def compute_expectation_series(states, operator):
    if sp.issparse(operator):
        return np.array([np.vdot(s, operator.dot(s)).real for s in states])
    else:
        return np.array([np.vdot(s, operator @ s).real for s in states])

magnetization_full = compute_expectation_series(states_full, Sz_total) / N
magnetization_proj = compute_expectation_series(states_proj, Sz_total) / N

# Gráfica de energía vs magnetización
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

axes[0].scatter(magnetization_full, energies_full, c=config.t_list,
                cmap='viridis', alpha=0.6, s=30)
axes[0].set_xlabel(r'$\langle S^z \rangle$ por sitio')
axes[0].set_ylabel('Energía total')
axes[0].set_title('Energía vs Magnetización (Completo)')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(magnetization_proj, energies_proj, c=config.t_list,
                cmap='viridis', alpha=0.6, s=30)
axes[1].set_xlabel(r'$\langle S^z \rangle$ por sitio')
axes[1].set_ylabel('Energía total')
axes[1].set_title('Energía vs Magnetización (Proyectado)')
axes[1].grid(True, alpha=0.3)

plt.colorbar(axes[1].collections[0], ax=axes[1], label='Tiempo')
plt.tight_layout()
if config.save_figs:
    plt.savefig('energy_vs_magnetization.png', dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

# Gráfica de energía por sitio en función del tiempo
plt.figure(figsize=(10, 6))
energy_per_site_full = energies_full / N
energy_per_site_proj = energies_proj / N

plt.plot(config.t_list, energy_per_site_full, 'b-', linewidth=2, label='Completo')
plt.plot(config.t_list, energy_per_site_proj, 'r--', linewidth=2, label='Proyectado')
plt.xlabel('Tiempo')
plt.ylabel('Energía por sitio')
plt.title('Energía por Sitio vs Tiempo')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
if config.save_figs:
    plt.savefig('energy_per_site.png', dpi=config.plot_dpi, bbox_inches='tight')
plt.show()

print("\n=== PROCESO COMPLETADO ===")

return {
    'ricci_results': ricci_results,
    'states_proj': states_proj,
    'states_full': states_full,
    'energies_full': energies_full,
    'energies_proj': energies_proj,

```

```

    return {
        'ricci_results': ricci_results,
        'states_proj': states_proj,
        'states_full': states_full,
        'energies_full': energies_full,
        'energies_proj': energies_proj,
        'E_ZZ_full': E_ZZ_full,
        'E_X_full': E_X_full,
        'E_ZZ_proj': E_ZZ_proj,
        'E_X_proj': E_X_proj,
        'local_energy_full': local_energy_full,
        'local_energy_proj': local_energy_proj,
        'variance_full': variance_full,
        'variance_proj': variance_proj,
        'magnetization_full': magnetization_full,
        'magnetization_proj': magnetization_proj,
        'energy_metrics': {
            'max_energy_error': max_energy_error,
            'mean_energy_error': mean_energy_error,
            'max_energy_rel_error': max_energy_rel_error,
            'max_conservation_error_full': max_conservation_error_full,
            'max_conservation_error_proj': max_conservation_error_proj,
            'energy_per_site_full': energy_per_site_full,
            'energy_per_site_proj': energy_per_site_proj
        }
    }

if __name__ == "__main__":
    # Ejecutar el análisis completo
    results = main_with_energy_analysis()

    # Acceder a Los resultados
    print("\nPara acceder a los resultados:")
    print(" - Energías completas: results['energies_full']")
    print(" - Energías proyectadas: results['energies_proj']")
    print(" - Componentes ZZ: results['E_ZZ_full'], results['E_ZZ_proj']")
    print(" - Componentes X: results['E_X_full'], results['E_X_proj']")
    print(" - Métricas de energía: results['energy_metrics']")

    # Configurar parámetros
    config = SystemConfig()
    config.N = 10
    config.h = 0.3
    config.plot_energy_analysis = True

    # Ejecutar análisis completo
    results = main_with_energy_analysis()

    # Acceder a datos específicos
    tiempos = config.t_list
    energia_completa = results['energies_full']
    energia_proyectada = results['energies_proj']
    energia_por_sitio = results['energy_metrics']['energy_per_site_full']

    # Graficar adicionalmente si es necesario
    plt.figure(figsize=(10, 6))
    plt.plot(tiempos, energia_completa, label='Completa')
    plt.plot(tiempos, energia_proyectada, label='Proyectada', linestyle='--')
    plt.xlabel('Tiempo')

```

```

plt.plot(tiempos, energia_proyectada, label='Proyectada', linestyle='--')
plt.xlabel('Tiempo')
plt.ylabel('Energía')
plt.legend()
plt.show()

```

Gráfica: magnetización 1D vs temperatura

A continuación generamos la gráfica de $m(T)$ para varios campos externos. Cambiá `J`, `hs` o el rango `T_vals` si querés experimentar.

In [8]:

```
import numpy as np
from typing import Tuple, Optional
import matplotlib.pyplot as plt
from scipy.sparse import diags, kron, identity
from scipy.sparse.linalg import eigsh
```

In [9]:

```
# Funciones analíticas para Ising 1D (transfer matrix)
import numpy as np

def lambda_plus(beta, h, J=1.0):
    bJ = beta * J
    bh = beta * h
    e2bJ = np.exp(2 * bJ)
    term = e2bJ * np.sinh(bh)**2 + np.exp(-2 * bJ)
    return np.exp(bJ) * np.cosh(bh) + np.sqrt(term)

def magnetization_1d(beta, h, J=1.0):
    # derivada analítica de lambda_plus respecto a (beta*h)
    bJ = beta * J
    bh = beta * h
    e2bJ = np.exp(2 * bJ)
    sqrt_arg = e2bJ * np.sinh(bh)**2 + np.exp(-2 * bJ)
    lam = lambda_plus(beta, h, J)
    dl_dbh = np.exp(bJ) * np.sinh(bh) + (e2bJ * np.sinh(bh) * np.cosh(bh)) / np.sqrt(sqrt_arg)
    return dl_dbh / lam

# Probar funciones
beta = 2.0
for h in [0.0, 0.1, 0.5]:
    print(f"beta={beta}, h={h} -> m_1D = {magnetization_1d(beta,h):.6f}")
```

```
import matplotlib.pyplot as plt
J = 1.0
kB = 1.0
T_vals = np.linspace(0.01, 5.0, 400)
hs = [0.0, 0.05, 0.2, 0.5, 1.0]

plt.figure(figsize=(8,5))
for h in hs:
    m_vals = np.array([magnetization_1d(1.0/(kB*T), h, J) for T in T_vals])
    plt.plot(T_vals, m_vals, label=f"h={h}")
plt.xlabel("Temperatura T (J/k_B)")
plt.ylabel("Magnetización por sitio m (1D)")
plt.title("Ising 1D – magnetización exacta (transfer matrix)")
plt.legend()
plt.grid(True)
plt.show()
```

```

def magnetization_2d(T, J=1.0, kB=1.0):
    T = np.array(T, dtype=float)
    beta = 1.0 / (kB * T)
    x = np.sinh(2 * beta * J)
    m = np.zeros_like(T)
    with np.errstate(divide='ignore', invalid='ignore'):
        val = 1.0 - x**(-4)
        val[val < 0] = 0.0
        m[x > 1.0] = val[x > 1.0]**(1.0/8.0)
    return m

# Parámetros y gráfico
J = 1.0
kB = 1.0
T_vals = np.linspace(0.01, 5.0, 400)
m2 = magnetization_2d(T_vals, J, kB)
Tc = 2.0 * J / np.log(1.0 + np.sqrt(2.0))

import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.plot(T_vals, m2, label="Magnetización 2D (Onsager, h=0)")
plt.axvline(Tc, linestyle='--', label=f"Tc = {Tc:.4f}")
plt.xlabel("Temperatura T (J/k_B)")
plt.ylabel("Magnetización por sitio m (2D)")
plt.title("Ising 2D - Magnetización espontánea (Onsager)")
plt.legend()
plt.grid(True)
plt.show()

```

Este es para correr los datos de los archivos csv generados con c++

Sebastián Alí Sacasa Céspedes 09/11/2025 - 03/12/2025

```
] : import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Para Cargar datos
config = pd.read_csv('config.csv')
time = pd.read_csv('time.csv')
energy = pd.read_csv('energy_results.csv')
magnetization = pd.read_csv('magnetization.csv')

# Graficamos La energía
plt.figure(figsize=(10, 6))
plt.plot(time['time'], energy['energy_full'], label='Completo', linewidth=2)
plt.plot(time['time'], energy['energy_proj'], label='Proyectado', linestyle='--', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('Energía total')
plt.title('Evolución de Energía')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('energy_evolution.png', dpi=150)
plt.show()

# Graficamos componentes de energía
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.plot(time['time'], energy['E_ZZ_full'], label='E_ZZ', linewidth=2)
plt.plot(time['time'], energy['E_X_full'], label='E_X', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('Energía')
plt.title('Componentes de Energía (Completo)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 2)
plt.plot(time['time'], energy['E_ZZ_proj'], label='E_ZZ (proj)', linewidth=2)
plt.plot(time['time'], energy['E_X_proj'], label='E_X (proj)', linewidth=2)
plt.xlabel('Tiempo')
```

```

# Graficamos componentes de energía
plt.figure(figsize=(12, 8))
plt.subplot(2, 2, 1)
plt.plot(time['time'], energy['E_zz_full'], label='E_zz', linewidth=2)
plt.plot(time['time'], energy['E_x_full'], label='E_x', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('Energía')
plt.title('Componentes de Energía (Completo)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 2)
plt.plot(time['time'], energy['E_zz_proj'], label='E_zz (proj)', linewidth=2)
plt.plot(time['time'], energy['E_x_proj'], label='E_x (proj)', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('Energía')
plt.title('Componentes de Energía (Proyectado)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 3)
plt.plot(time['time'], magnetization['magnetization_full'], label='Completo', linewidth=2)
plt.plot(time['time'], magnetization['magnetization_proj'], label='Proyectado', linestyle='--', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel(r'$\langle S^z \rangle$ por sitio')
plt.title('Magnetización')
plt.legend()
plt.grid(True, alpha=0.3)

plt.subplot(2, 2, 4)
fidelity = pd.read_csv('fidelity_evolution.csv')
plt.plot(time['time'], fidelity['fidelity'], 'g-', linewidth=2)
plt.xlabel('Tiempo')
plt.ylabel('Fidelidad')
plt.title('Fidelidad durante la evolución')
plt.grid(True, alpha=0.3)
plt.ylim(0, 1.05)

plt.tight_layout()
plt.savefig('summary_plots.png', dpi=150)
plt.show()

```